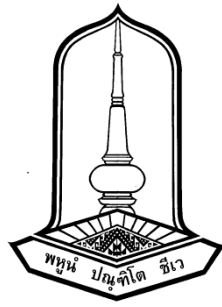


๑๑ มี



ศูนย์รภัยรหัสผ่านแบบออนไลน์
Organized Online Password Safe (OOPS)

โครงการปริญญานิพนธ์
ของ

มลธิชา ศรีดาจันทร์

เสนอต่อมหาวิทยาลัยมหาสารคาม เพื่อเป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
ปริญญาวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์

ปีการศึกษา ๒๕๖๗ ๖ ๒๕๖๘.

ลิขสิทธิ์เป็นของมหาวิทยาลัยมหาสารคาม
คณะวิทยาการสารสนเทศ มหาวิทยาลัยมหาสารคาม

↓ ชิดลา ศด.

คู่มือรหัสผ่านแบบออนไลน์
Organized Online Password Safe (OOPS)

โครงการปริญญานิพนธ์

ของ

มลธิชา ศรีดาจันทร์

เสนอต่อมหาวิทยาลัยมหาสารคาม เพื่อเป็นส่วนหนึ่งของการศึกษาตามหลักสูตร

ปริญญาวิทยาศาสตรบัณฑิต สาขาวิทยาการคอมพิวเตอร์

ปีการศึกษา 2567

ลิขสิทธิ์เป็นของมหาวิทยาลัยมหาสารคาม

คณะวิทยาการสารสนเทศ มหาวิทยาลัยมหาสารคาม

* ၆၆၈။ ၁၁၇။ ၁၁၈။ ၁၁၉။ ၁၂၀။

9.

ประธานสอบ

$(\quad)$

→ ၂၂၀၀၀၀၀၀

กิจกรรมการ

ที่ปรึกษาโครงการปริญญานิพนธ์หลัก

(ผู้ช่วยศาสตราจารย์ ดร.สมนึก พ่วงพรพิทักษ์)

หลักสูตรวิทยาการคอมพิวเตอร์อนุมัติให้รับโครงการปริญญาโทฉบับนี้ เป็นส่วนหนึ่งของ
การศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาการ
สารสนเทศ มหาวิทยาลัยมหาสารคาม

(อาจารย์พชร พฤษะสิทธิ์)

(ผู้ช่วยศาสตราจารย์พิมลรัตน์ อ้วนศรีเมือง)

07/07/21 05.

อาจารย์ผู้ประสานงานวิชาโครงการปริญญานิพนธ์

วันที่ [ใช้วันที่สอบ] เดือน [ชื่อเดือน] พ.ศ. [ปีพ.ศ.]

Q. 21C H₂O₂ 24!

↑ ชัดเจน.

บทคัดย่อ

ชื่อโครงการ	คู่มือการรหัสผ่านแบบออนไลน์
ผู้จัดทำ	65011212194 นางสาวมลธิชา ศรีดาจันทร์
อาจารย์ที่ปรึกษา	ผู้ช่วยศาสตราจารย์ ดร.สมนึก พ่วงพรพิทักษ์
หลักสูตร	วิทยาศาสตร์บัณฑิต (สาขาวิชาวิทยาการคอมพิวเตอร์)
คณะ	วิทยาการสารสนเทศ
มหาวิทยาลัย	มหาวิทยาลัยมหาสารคาม
ปีที่พิมพ์	[ปีที่ส่งเล่มสมบูรณ์] ๕๕๕๕

Organized Online Password Safe หรือ OOPS! เป็นแพลตฟอร์มออนไลน์ที่ช่วยจัดเก็บชื่อผู้ใช้รหัสผ่าน และข้อมูลสำคัญอื่นๆได้อย่างปลอดภัยด้วยการเข้ารหัสทำให้ปลอดภัยจากการโจมตีของแฮกเกอร์ และจากแอดมินระบบด้วยเช่นกัน พร้อมฟังก์ชันประเมินความแข็งแกร่งของรหัสผ่านด้วยการคำนวณค่า Entropy และการประเมินความเสี่ยงในการถูกโจมตี เพื่อช่วยผู้ใช้งานจัดการจัดเก็บและปรับปรุงรหัสผ่านให้มีความปลอดภัยยิ่งขึ้น

คำสำคัญ: [คลิกพิมพ์รายละเอียด] [คลิกพิมพ์รายละเอียด] (ไม่ควรเกิน 5 คำ)

กิตติกรรมประกาศ

อาจารย์ ดร.

โครงการปริญญานิพนธ์ฉบับนี้สำเร็จสมบูรณ์ได้ด้วยความรู้และความช่วยเหลืออย่างสูงยิ่งจาก ผู้ช่วยศาสตราจารย์ ดร.สมนึก พ่วงพรพิทักษ์ อาจารย์ที่ปรึกษาโครงการปริญญานิพนธ์ อาจารย์พระ พฤกษ์ ศรี และ ผู้ช่วยศาสตราจารย์พิมลรัตน์ อ้วนศรีเมือง กรรมการควบคุมโครงการปริญญานิพนธ์ [ชื่อประธาน กรรมการสอบ] ประธานกรรมการสอบ และ [ชื่อกรรมการสอบ]

กรรมการสอบ

ขอขอบพระคุณ [ชื่อผู้เชี่ยวชาญ] (ขอบคุณผู้เชี่ยวชาญที่ช่วยตรวจ หรือช่วยให้คำแนะนำ ถ้ามี)
ขอขอบพระคุณ (ขอบคุณผู้ให้การช่วยเหลือสนับสนุนอื่นๆ ถ้ามี)

มลธิชา ศรีดาจันทร์ →

ศึกษา

สารบัญ

สอ.ก.น.อ. (ทอ.ก.น.อ.)

← บทคัดย่อ.....	ข
← กิตติกรรมประกาศ	ค
← สารบัญ	ง
← สารบัญตาราง	ช
← สารบัญภาพประกอบ	ซ
← บทที่ 1.....	1
1.1 หลักการและเหตุผล	1 →
1.2 วัตถุประสงค์ของโครงการ	2 →
1.3 ขอบเขตของโครงการ	2 →
1.3.1 สิทธิ์การใช้งานระบบ	2
1.3.2 การตรวจสอบความปลอดภัยของรหัสผ่าน	4
1.3.3 ตรวจสอบความเสี่ยงจากการถูกโจมตี.....	5
1.3.4 การสมัครเข้าใช้งานระบบ (Sign-up).....	5
1.3.5 การเข้าใช้งานระบบ (Log-in).....	5
1.3.6 การเก็บข้อมูลความลับของ Member ในระบบ.....	6
1.3.7 การเข้ารหัสข้อมูลอ่อนไหวที่เก็บในระบบ	6
1.3.8 การ reset M-OTP.....	7
1.4 ภาพรวมของระบบ.....	7 →
1.5 ตัวอย่างโปรแกรม OOPS!.....	8 →
1.6 ประโยชน์ที่คาดว่าจะได้รับ	16 →
1.7 อุปกรณ์และเครื่องมือที่ใช้ดำเนินการ	16 →
1.8 แผนการดำเนินการ	17 →
← บทที่ 2.....	18
2.1 งานวิจัยและทฤษฎีที่เกี่ยวข้อง.....	18
2.1.1 การยืนยันตัวตน 2 ชั้น (2FA).....	18
2.1.2 M-OTP	18
2.1.3 การโจมตีรหัสผ่าน	19

07/22

2.1.4 Entropy.....	19
2.1.5 Rockyou.txt	20
2.1.6 Rockyou2024	21
2.1.7 HIBP.....	22
2.1.8 Scrypt Hash Function : AES256 bits	22
2.1.9 GPU (Graphics Processing Unit).....	23
2.1.10 โปรเซสเซอร์ Intel core	24
2.1.11 การ์ดจอ	24
ประเภทของการ์ดจอ.....	24
2.1.12 GeForce RTX 4090	26
2.2 Software ที่เกี่ยวข้อง	26
Visual Studio.....	26
Visual Studio Code (VS Code)	27
Google Authenticator.....	27
บทที่ 3.....	29
3.1 กรอบการดำเนินการ (ภาพรวมของระบบ).....	29
3.1.1 User Registration Flow	30
3.1.2 Login & Authentication Flow	31
3.1.3 Session ManagementFlow	32
3.1.4 Secret Management Flow	32
3.2 ฐานข้อมูล (ดาต้าเบส).....	34
3.3 Data Store	43
3.4 External Entity Description	43
3.5 Data Flow Description and Data Structure.....	44
3.6 การพัฒนาระบบ	47
3.6.1 การติดตั้ง Node.js	47
3.6.2 การใช้งาน Node.js	47
3.6.3 การติดตั้ง Angular	48
3.6.4 การใช้งาน Angular.....	50
3.6.5 การสมัครสมาชิก	51
3.6.6 การเข้าสู่ระบบ.....	58

3.6.7	การรีเซ็ต M-OTP	62
3.6.8	ตรวจสอบความแข็งแกร่งของรหัสผ่านด้วยค่า Entropy	67
3.6.9	Dictionary Attack	70
3.6.10	GPU Attack.....	75
3.6.11	AES 256 GCM.....	79
	เอกสารอ้างอิง.....	91

สารบัญตาราง

I 1 ตารางต่อ

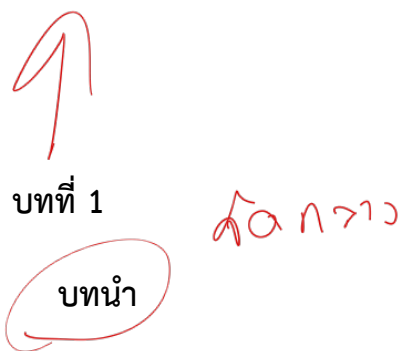
ตารางที่ 1.1 แผนการดำเนินงาน.....	17
ตารางที่ 3.1 User table.....	35
ตารางที่ 3.3 Usersecret table.....	36
ตารางที่ 3.4 Dictionary.....	36
ตารางที่ 3.5 Otpcodes	37
ตารางที่ 3.6 GPU	37
ตารางที่ 3.7 Data Store Description	43
ตารางที่ 3.8 External Entity Description.....	43
ตารางที่ 3.9 Data Flow Description and Data Structure	45

สารบัญภาพประกอบ

ภาพประกอบที่ 1.1 ภาพรวมของระบบ	7
ภาพประกอบที่ 1.2 ตัวอย่างหน้าเริ่มต้นของเว็บไซต์.....	8
ภาพประกอบที่ 1.3 สมัครเข้าใช้งานระบบ.....	8
ภาพประกอบที่ 1.4 ล็อกอินครั้งแรก.....	9
ภาพประกอบที่ 1.5 ยืนยันตัวตน.....	10
ภาพประกอบที่ 1.6 เชื่อม Authenticator.....	10
ภาพประกอบที่ 1.7 หน้าแอปพลิเคชันทั้งหมด.....	11
ภาพประกอบที่ 1.8 หน้าสื่อสังคมออนไลน์.....	11
ภาพประกอบที่ 1.9 หน้าธุรกรรม.....	12
ภาพประกอบที่ 1.10 หน้าความบันเทิง.....	12
ภาพประกอบที่ 1.11 หน้าการศึกษา.....	13
ภาพประกอบที่ 1.12 หน้าหนังสือ.....	13
ภาพประกอบที่ 1.13 หน้าเกม.....	14
ภาพประกอบที่ 1.14 หน้าอื่นๆ.....	14
ภาพประกอบที่ 1.15 ตรวจสอบรหัสผ่าน.....	15
ภาพประกอบที่ 1.16 รหัสผ่านไม่ปลอดภัย.....	15
ภาพประกอบที่ 1.17 รหัสผ่านปลอดภัย.....	16
ภาพประกอบที่ 2.1 ตัวอย่างการใช้งานแอปพลิเคชัน Authenticator.....	19
ภาพประกอบที่ 2.2 ตัวอย่างการใช้งานแอปพลิเคชัน Authy.....	19
ภาพประกอบที่ 2.3 Rocky.txt	21
ภาพประกอบที่ 2.4 ตัวอย่างการจำลองฮาร์ดแวร์.....	24
ภาพประกอบที่ 2.5 ตัวอย่างการจำลองเล่นเกม.....	25
ภาพประกอบที่ 2.6 ตัวอย่างการจำลองกราฟิก.....	26
ภาพประกอบที่ 2.7 GEFORCE RTX 4090	26
ภาพประกอบที่ 2.8 Visual Studio & VS Code	27
ภาพประกอบที่ 2.9 Google Authenticator.....	28
ภาพประกอบที่ 3.1 ภาพรวมของระบบ	29
ภาพประกอบที่ 3.2 Registration Flow	30
ภาพประกอบที่ 3.3 Login & Authentication Flow.....	31
ภาพประกอบที่ 3.4 Session ManagementFlow	32

ภาพประกอบที่ 3.5 Secret Management Flow	32
ภาพประกอบที่ 3.6 Database	34
ภาพประกอบที่ 3.7 Context Diagram	38
ภาพประกอบที่ 3.8 แผนภาพกระแสข้อมูลระดับที่ 1	39
ภาพประกอบที่ 3.9 แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการเข้าสู่ระบบและยืนยันตัวตน ...	40
ภาพประกอบที่ 3.10 แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการจัดการข้อมูลลับ	41
ภาพประกอบที่ 3.11 แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการตรวจสอบความปลอดภัยรหัสผ่าน	42
ภาพประกอบที่ 3.12 การตรวจสอบหมายเลขเวอร์ชันและการติดตั้ง npm	47
ภาพประกอบที่ 3.13 การติดตั้ง Angular	48
ภาพประกอบที่ 3.14 การตรวจสอบเวอร์ชัน Angular CLI ด้วยคำสั่ง ng version	49
ภาพประกอบที่ 3.15 การใช้ Angular ในการออกแบบหน้าแรกของเว็บไซต์	50
ภาพประกอบที่ 3.16 ฟอรัมสมัครสมาชิก	51
ภาพประกอบที่ 3.17 การ Import & การกำหนดคุณสมบัติผ่าน Component	52
ภาพประกอบที่ 3.18 การตรวจสอบการสมัครสมาชิก	53
ภาพประกอบที่ 3.19 กรอกอีเมล OTP	54
ภาพประกอบที่ 3.20 ตรวจสอบ Email-OTP	55
ภาพประกอบที่ 3.21 ยืนยันตัวตนด้วย Google Authenticator	56
ภาพประกอบที่ 3.22 การกระบวนการยืนยันตัวตนของผู้ใช้งาน	57
ภาพประกอบที่ 3.23 กรอกข้อมูลสำหรับการเข้าสู่ระบบ	58
ภาพประกอบที่ 3.24 หน้าเข้าสู่ระบบ	59
ภาพประกอบที่ 3.25 คลาส Login	60
ภาพประกอบที่ 3.26 การยืนยันตัวตนแบบ Two-Factor Authentication (2FA)	61
ภาพประกอบที่ 3.27 Service สำหรับจัดการกระบวนการเข้าสู่ระบบ	61
ภาพประกอบที่ 3.28 กรอกอีเมลเพื่อขอรีเซ็ตรหัสยืนยันตัวตน	62
ภาพประกอบที่ 3.29 คลาสรีเซ็ตอีเมล	63
ภาพประกอบที่ 3.30 คลาสรีเซ็ต OTP สำหรับตรวจสอบรหัสในขั้นตอนการรีเซ็ต M-OTP	64
ภาพประกอบที่ 3.31 คลาสรีเซ็ต New M-OTP	64
ภาพประกอบที่ 3.32 ฟังก์ชัน confirm สำหรับตรวจสอบรหัส OTP จาก Google Authenticator ..	65
ภาพประกอบที่ 3.33 Service สำหรับจัดการกระบวนการรีเซ็ตรหัส M-OTP	66
ภาพประกอบที่ 3.34 คลาส Entropy	67
ภาพประกอบที่ 3.35 ฟังก์ชัน calculateEntropy()	67

ภาพประกอบที่ 3.36 คลาส Dictionary	72
ภาพประกอบที่ 3.37 ฟังก์ชัน LoadRockyou	73
ภาพประกอบที่ 3.38 ฟังก์ชัน dictionaryCheck()	73
ภาพประกอบที่ 3.39 คลาส GpuAttack	75
ภาพประกอบที่ 3.40 GPU_RATES	76
ภาพประกอบที่ 3.41 ฟังก์ชัน poolSize & formatDuration	77
ภาพประกอบที่ 3.42 ฟังก์ชันสำหรับคำนวณระยะเวลาที่คาดว่าจะใช้ในการแคร็กรหัสผ่าน	78
ภาพประกอบที่ 3.43 ฟังก์ชัน getKey	80
ภาพประกอบที่ 3.44 ฟังก์ชัน encrypt&decrypt	81



1.1 หลักการและเหตุผล

ในปัจจุบัน รหัสผ่าน (Password) ถือเป็นปัจจัยในการยืนยันตัวตนที่สำคัญที่สุดในระบบดิจิทัล โดยทำหน้าที่เป็นเครื่องมือสำคัญในการปกป้องข้อมูลส่วนบุคคลและทรัพยากรอื่นๆ ที่สำคัญในระบบออนไลน์ หากรหัสผ่านได้รับการตั้งค่าอย่างเหมาะสมและมีความปลอดภัย จะสามารถลดความเสี่ยงจากการโจมตีทางไซเบอร์และช่วยรักษาความลับของข้อมูลได้อย่างมีประสิทธิภาพ

อย่างไรก็ตาม ในยุคดิจิทัลที่บริการออนไลน์ (Online Services) มีการพัฒนาและเพิ่มจำนวนมากขึ้น ผู้ใช้งานจำเป็นต้องสร้างบัญชีผู้ใช้สำหรับการเข้าถึงบริการต่างๆ ซึ่งทำให้ผู้ใช้งานต้องสร้างและจดจำรหัสผ่านจำนวนมาก การใช้รหัสผ่านที่แตกต่างกันในแต่ละบัญชีเพื่อความปลอดภัยอาจกลายเป็นความท้าทาย เนื่องจากจำนวนบัญชีที่เพิ่มขึ้นส่งผลให้การจดจำรหัสผ่านทั้งหมดเป็นเรื่องยาก

ปัจจุบันมีซอฟต์แวร์ประเภท Password Manager ที่ช่วยในการจัดเก็บชื่อผู้ใช้และรหัสผ่านสำหรับการเข้าถึงบริการออนไลน์หรือเว็บไซต์ เช่น Dashlane [1], 1Password [2], NordPass [3], RoboForm [4] เป็นต้น โดยซอฟต์แวร์เหล่านี้ช่วยลดปัญหาการจดจำรหัสผ่านของผู้ใช้งาน แต่กระบวนการติดตั้งซอฟต์แวร์มักมีความซับซ้อน และต้องใช้ทรัพยากรของอุปกรณ์ เช่น พื้นที่เก็บข้อมูลในจำนวนมาก ผู้ใช้งานส่วนมากจึงไม่นิยมติดตั้งใช้งาน

จากปัญหาดังกล่าว ทีมงานจึงพัฒนาเว็บไซต์ "OOPS!" ซึ่งเป็นแพลตฟอร์มออนไลน์ที่ออกแบบมาเพื่อช่วยจัดเก็บชื่อผู้ใช้และรหัสผ่านของผู้ใช้งาน โดยมีการเข้ารหัส (Encryption) เพื่อเพิ่มความปลอดภัยของข้อมูล นอกจากนี้ เว็บไซต์ยังมีฟังก์ชันในการประเมินความแข็งแกร่งของรหัสผ่านเพื่อช่วยผู้ใช้งานปรับปรุงรหัสผ่านให้มีความปลอดภัยยิ่งขึ้น

โครงการปริญญาโทฉบับนี้จึงได้สร้างเว็บไซต์ที่ชื่อว่า "OOPS!" ขึ้นมา โดยมีจุดมุ่งหมายเพื่อแก้ปัญหาลืมรหัสผ่านและลดความยุ่งยากในการจัดการรหัสผ่านของผู้ใช้งาน นอกจากนี้ยังมีบริการตรวจสอบความแข็งแกร่งของรหัสผ่านโดยคำนวณจากค่า Entropy และคำนวณความเสี่ยงในการถูกโจมตีว่าจะถูกโจมตีได้ในระยะเวลาเท่าใด

1.2 วัตถุประสงค์ของโครงการ

เพื่อพัฒนาเว็บไซต์ "OOPS!" ซึ่งเป็นแพลตฟอร์มออนไลน์สำหรับช่วยจัดเก็บและจัดการข้อมูลผู้ใช้รหัสผ่าน และมีข้อมูลสำคัญอื่น ๆ เช่น PIN, รหัสบัตรเครดิต, OTP secret โดยเน้นความสะดวกในการใช้งาน มั่นคงและความปลอดภัยของข้อมูล ผ่านการใช้เทคนิคการเข้ารหัส

นอกจากนี้ยังมีฟังก์ชันการประเมินความแข็งแกร่งของรหัสผ่าน โดยอ้างอิงจากค่า Entropy และระยะเวลาที่คาดว่าจะถูกโจมตี เพื่อช่วยให้ผู้ใช้งานปรับปรุงรหัสผ่านให้มีความปลอดภัยยิ่งขึ้น อีกทั้งยังช่วยลดปัญหาการลืมรหัสผ่าน พร้อมทั้งอำนวยความสะดวกในการจัดการรหัสผ่านของผู้ใช้งาน เพื่อลดภาระในการจดจำรหัสผ่านที่หลากหลายโดยไม่ต้องพึ่งพาซอฟต์แวร์ที่ต้องติดตั้งในอุปกรณ์

1.3 ขอบเขตของโครงการ

1.3.1 สิทธิการใช้งานระบบ

1) ผู้ใช้ทั่วไป (No login)

- สามารถใช้งานฟังก์ชันตรวจสอบความแข็งแกร่งของรหัสผ่านได้
- ไม่สามารถเลือก Dictionary ที่ใช้ตรวจสอบ รุ่นของ GPU Attack และไม่สามารถเก็บ

ข้อมูลในระบบได้

2) สมาชิก (Member)

- สามารถใช้งานฟังก์ชันตรวจสอบความแข็งแกร่งของรหัสผ่านได้
- สามารถเลือก Dictionary ในการตรวจสอบรหัสผ่านได้
- สามารถเลือกรุ่นของ GPU เพื่อตรวจสอบระยะเวลาในการโจมตีได้
- สามารถเก็บข้อมูลในระบบได้
- สามารถเพิ่มข้อมูล(insert), ลบข้อมูล>Delete), แก้ไขข้อมูล>Edit) ของตนเองได้
- ไม่สามารถจัดการข้อมูลของผู้ใช้คนอื่นหรือผู้จัดการระบบได้
- สามารถขอยกเลิกการใช้ระบบและลบข้อมูลที่เก็บไว้ในระบบทั้งหมดได้
- การ login ของ Member จะไม่ต้องอาศัยรหัสผ่าน แต่ email เป็น username และ ใช้

M-OTP หรือ เลข 6 หลักที่เป็น One Time Password (OTP) ที่แสดงใน Google

Authenticator หรือ OTP Application อื่นๆ ที่เป็นไปตามมาตรฐาน Timer-based

OTP เป็นรหัสผ่านแบบใช้ได้ครั้งเดียว

- ทั้งนี้ Member จะต้องทำการ Sign Up (ลงทะเบียน)

- โดยการลงทะเบียน email

- โดยการยืนยัน email จะใช้ Email OTP ในการยืนยัน

บรรทัด 2
จุดที่ 14, 15, 16
*เรดิเคส 2 ไร่ 2 เมตร
พื้นที่

- จากนั้น จะให้ member แต่ละคน สร้าง OTP Seed (หรือเรียกว่า OTP Secret) ซึ่งเป็นค่าเริ่มต้นสำหรับการสร้าง M-OTP ของตัวเอง ที่จะเก็บไว้ใน Member profile ในฐานข้อมูล
- และ Member แต่ละคน ต้องติดตั้ง Google Authenticator App บน โทรศัพท์มือถือ เพื่อ scan ค่า OTP Seed นี้ ผ่าน QR code แล้วนำไปสู่การสร้าง M-OTP ตามเวลาสำหรับ Member นี้ต่อไป

3) Admin

- สามารถทำฟังก์ชันของสมาชิก ในข้อ 2) ได้
- มีสิทธิ์ในการตรวจสอบและควบคุมกิจกรรมที่เกิดขึ้นในระบบ
- มีสิทธิ์ในการจัดการผู้ใช้ทั้งหมด และระงับการใช้งานชั่วคราว (disable user)
- สามารถตั้งค่าระบบเกี่ยวกับการตรวจสอบความปลอดภัยของรหัสผ่าน และการเข้าถึงฟังก์ชันต่าง ๆ บนเว็บไซต์ได้
- สามารถเพิ่ม ลบ แก้ไข ประเภทการดจอเพื่อใช้ในการตรวจสอบ GPU Attack ได้
- สามารถแก้ไข อัปเดต Dictionary ได้
- ผู้ใช้กลุ่ม Admin จะถูกสร้าง โดย Superadmin ต่างจาก Member ที่ไม่มีการลงทะเบียน (Sign Up)
- การ login ของผู้ใช้กลุ่ม Admin เป็น Two Factor Authentication (2FA) โดยการ login สำหรับแต่ละ username จะใช้ รหัสผ่าน และ M-OTP ที่ดูจาก Google authenticator App
- การ สร้างและลงทะเบียน M-OTP Seed จะทำในครั้งแรกของการ login ดังนี้
 - หลังได้รับรหัสผ่านที่กำหนดโดย Superadmin เพื่อ login ครั้งแรก
 - Admin จะต้องทำการตั้งรหัสผ่านใหม่ของตัวเอง
 - จากนั้น Generate และลงทะเบียน M-OTP Seed เพื่อเก็บเข้า database ของระบบ
 - และใช้ Google Authenticator App เพื่อ scan QR code
 - การลงทะเบียน M-OTP นี้ต้องแล้วเสร็จ ก่อนที่จะสามารถทำฟังก์ชัน อื่นๆ ได้
- ทั้งนี้ รหัสผ่านของ Admin จะจัดเก็บโดยการ hash ที่เหมาะสม ในระบบ

4) Superadmin

- สามารถทำฟังก์ชันต่างๆ ของทั้ง Admin และ Member ในข้อ 2) และ 3) ได้
- สามารถเพิ่ม ลบ แก้ไข และ disable (ระงับการใช้งาน) Admin ได้

- Superadmin เป็น user ที่มีอำนาจสูงสุด ให้มาพร้อมระบบ สำหรับเจ้าของระบบเป็นผู้ถือครอง และไม่สามารถถูกลบได้
- Superadmin มีขบวนการ สร้างและลงทะเบียน M-OTP Seed จะทำในครั้งแรกของการ login เช่นเดียวกันกับ Admin จากนั้น การ login ก็จะทำโดย 2FA คือ ใช้ email เป็น username และ ทำ authentication ด้วย รหัสผ่าน และ M-OTP

1.3.2 การตรวจสอบความปลอดภัยของรหัสผ่าน

1) ตรวจสอบโอกาสในการโดน brute-force attack โดยการคำนวณค่า Entropy

คำนวณค่า Entropy จากสูตรการคำนวณของ Claude Shannon ในปี 1948 ซึ่งมีสูตรการคำนวณ

คือ $H = L * \log_2 N$

โดย

H หมายถึง ค่า Entropy (หน่วยเป็นบิต)

L หมายถึง ความยาวของรหัสผ่าน (จำนวนตัวอักษร)

N หมายถึง จำนวนตัวเลือกที่เป็นไปได้สำหรับแต่ละตัวอักษร (ขนาดของชุดอักขระ)

โดย N = 10 ตัวเลข 0-9

N = 26 ตัวอักษรพิมพ์เล็ก (a-z)

N = 26 ตัวอักษรพิมพ์ใหญ่ (A-Z)

N = 32 สัญลักษณ์พิเศษ

ระบบจะนำประเภทของอักขระที่ปรากฏภายในรหัสผ่านมาคำนวณร่วมกัน เพื่อหาค่าความเป็นไปได้ทั้งหมดของชุดรหัสผ่าน และนำไปใช้ประเมินความแข็งแกร่งของรหัสผ่านว่ามีความปลอดภัยเพียงพอต่อการป้องกันการโจมตีแบบ Brute-force หรือไม่

แนวคิดดังกล่าวอ้างอิงจากงานวิจัยเรื่อง “An Analysis of Password Security Risk Against Dictionary Attacks” ซึ่งเผยแพร่ผ่านเว็บไซต์ IEEE Xplore

2) ตรวจสอบโอกาสในการโดน dictionary attack

- ตรวจสอบว่ารหัสผ่านที่ผู้ใช้กำหนดมีอยู่ในฐานข้อมูล Dictionary หรือรหัสผ่านที่เคยรั่วไหลมาก่อนหรือไม่ หากตรวจพบว่ารหัสผ่านตรงกับข้อมูลในฐานข้อมูล ระบบจะแจ้งเตือนให้ผู้ใช้เปลี่ยนรหัสผ่านใหม่
- ใช้ฐานข้อมูลจากไฟล์ RockYou.txt ซึ่งเป็นชุดข้อมูลรหัสผ่านที่ถูกเปิดเผยจากเหตุการณ์รั่วไหลของข้อมูลในอดีต และถูกนำมาใช้เป็นฐานข้อมูลสำหรับการโจมตีแบบ Dictionary Attack

- ใช้วิธีการเรียกใช้บริการ API จาก Have I Been Pwned (HIBP) มาใช้ในการตรวจสอบรหัสผ่านที่เคยปรากฏอยู่ในฐานข้อมูลการรั่วไหลของข้อมูล (Data Breach) จากแหล่งข้อมูลจริงทั่วโลก

1.3.3 ตรวจสอบความเสี่ยงจากการถูกโจมตี

1) ตรวจสอบจาก GPU

- คำนวณโดยเปรียบเทียบจาก GPU รุ่นล่าสุดว่าจะใช้เวลานานแค่ไหนในการโจมตีรหัสผ่าน
- สามารถเพิ่ม GPU และกำลังการคำนวณได้

1.3.4 การสมัครใช้งานระบบ (Sign-up)

- 1) ชื่อ-นามสกุลจริง : ผู้ใช้จะต้องกรอกชื่อจริงและนามสกุลจริง หากผู้ใช้เกิดลืมรหัสและติดต่อผ่าน Form แอดมินจะทำการตรวจสอบว่าผู้ที่ร้องขอมานั้นข้อมูลต่างๆเหมือนกันหรือไม่
- 2) อีเมล (Email) : ผู้ใช้จะต้องกรอกที่อยู่อีเมลที่สามารถติดต่อได้และใช้ในการสมัครใช้งานระบบ อีเมลนี้จะถูกใช้สำหรับการยืนยันการสมัคร การส่งข้อมูลสำคัญเกี่ยวกับบัญชีผู้ใช้งาน และอีเมลจะถูกนำไปใช้เป็น username ของระบบเพื่อป้องกันการตั้งชื่อซ้ำกัน

2.1) Email-OTP

ระบบจะส่ง Email-OTP ไปยังอีเมลที่ใช้สมัคร จากนั้นผู้ใช้นำ Email-OTP มายืนยันจึงจะถือว่าสมัครสำเร็จ

- 3) หมายเลขโทรศัพท์ : ผู้ใช้จะต้องกรอกหมายเลขโทรศัพท์ที่สามารถติดต่อได้ โดยหมายเลขโทรศัพท์นี้จะใช้สำหรับการยืนยันตัวตนและเป็นช่องทางการติดต่อในกรณีที่ระบบต้องการติดต่อผู้ใช้งาน
- 4) CAPTCHA : ใช้ CAPTCHA ในการป้องกัน Bot , Script อัตโนมัติไม่ให้ flood สมัครบัญชี ทำให้ระบบปลอดภัยและลดการใช้ทรัพยากรเกินจำเป็น
- 5) รหัสผ่าน (Password) : สำหรับ admin และ superadmin เท่านั้น

1.3.5 การใช้งานระบบ (Log-in)

1) สมาชิก

1.1) ใช้งานครั้งที่ 1

- กรอก Email เพื่อรับ Email-OTP
- กรอกรหัส Email-OTP
- เมื่อเข้าสู่เว็บไซต์แล้วระบบจะบังคับให้ผู้ใช้งานผูกบัญชีอีเมลกับแอปพลิเคชัน Authenticator

1.2) เข้าใช้งานครั้งต่อไป

- กรอก Email ที่สมัครใช้งาน
- เปิดแอปพลิเคชัน Google Authenticator เพื่อรับรหัส M-OTP
- กรอกรหัส M-OTP เพื่อเข้าใช้งาน

2) Admin และ Superadmin

- กรอก Email
- กรอกรหัสผ่าน
- กรอก M-OTP

1.3.6 การเก็บข้อมูลความลับของ Member ในระบบ

เมื่อผู้ใช้ล็อกอินเข้าสู่ระบบแล้ว ผู้ใช้สามารถบันทึกข้อมูลที่ต้องการได้ โดยมีข้อมูลหลักที่สามารถเก็บได้ดังนี้

- Username
- Password
- PIN
- QR Code
- ชื่อระบบที่เข้าใช้งาน
- ข้อมูลอื่นๆ ระบุ

หมายเหตุ : การเก็บข้อมูล Password, Pin และ QR Code ระบบจะจัดเก็บข้อมูลเหล่านี้อย่างปลอดภัยในรูปแบบเข้ารหัสเพื่อป้องกันการเข้าถึงจากบุคคลภายนอก

1.3.7 การเข้ารหัสข้อมูลอ่อนไหวที่เก็บในระบบ

เพื่อความปลอดภัยของข้อมูลของผู้ใช้เราจึงทำการเข้ารหัสข้อมูลที่สำคัญ เพื่อป้องกันการถูกเข้าถึงจากบุคคลอื่น

- ให้ member กำหนด passphrase และต้องจำไว้โดยไม่เก็บในระบบ
- ใช้ passphrase ดังกล่าว สร้าง AES Key ขนาด 256 bit เพื่อทำการเข้ารหัสข้อมูลสำคัญ ก่อนเก็บลงฐานข้อมูล
- การเก็บข้อมูลในระบบนี้จึงปลอดภัยจากการ hack และจาก admin ด้วยเช่นกัน
- มีเงื่อนไขว่า member จะต้องไม่ลืม passphrase ที่ตั้ง มิฉะนั้นจะเข้าถึงข้อมูลในระบบไม่ได้
- มีระบบช่วย reset passphrase ได้ โดยจะต้องจำ passphrase เดิมได้โดยจะทำการ re-encrypt ข้อมูลสำคัญที่เข้ารหัสไว้ใหม่ทั้งหมดให้กับ member ที่สั่ง reset passphrase

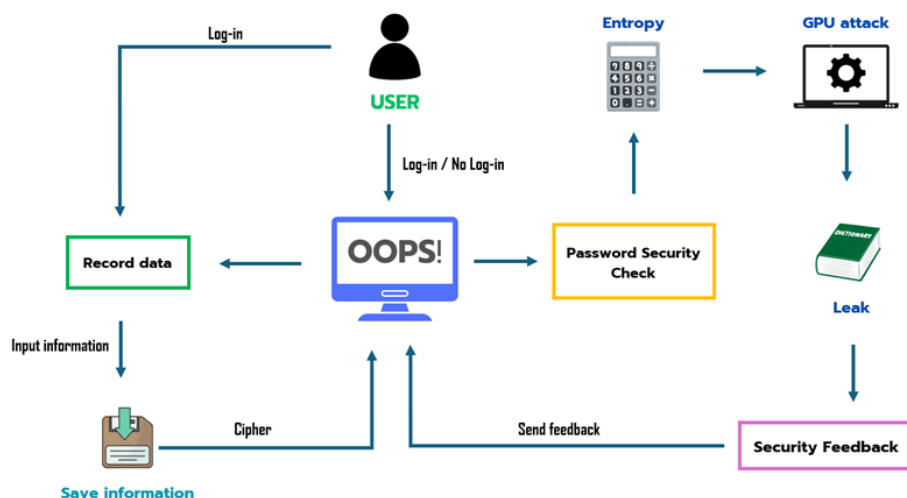
1.3.8 การ reset M-OTP

ในกรณีที่ผู้ใช้ (ทั้งระดับ Member, Admin และ Superadmin) ทำโทรศัพท์มือถือหาย ทำให้ถึง Google Authenticator เพื่อดู M-OTP ได้ หรือมีเหตุต้องเปลี่ยน M-OTP Seed ผู้ใช้สามารถดำเนินการตามขั้นตอนต่างๆ ที่กำหนดไว้เพื่อทำการรีเซ็ตรหัส M-OTP และเข้าสู่ระบบใหม่ได้โดยมีวิธีการดำเนินการดังนี้

หากผู้ใช้ลืมรหัส M-OTP ระบบจะมีตัวเลือกให้ส่งรหัสรีเซ็ตไปยัง อีเมล ที่ได้ทำการลงทะเบียนไว้ในระบบ ผู้ใช้สามารถทำตามขั้นตอนดังนี้

- ผู้ใช้กรอกอีเมลที่ใช้ในการสมัครเข้าใช้งานระบบ
- ระบบจะส่งรหัสรีเซ็ตไปยังอีเมลที่กรอกไว้
- นำรหัสที่ได้รับจากอีเมลกรอกลงในฟอร์มเพื่อทำการยืนยันตัวตน
- หลังจากยืนยันตัวตนสำเร็จระบบจะให้ผู้ใช้ตั้งรหัส M-OTP ใหม่และทำการล็อกอินเข้าสู่ระบบด้วยรหัสที่ตั้งใหม่

1.4 ภาพรวมของระบบ

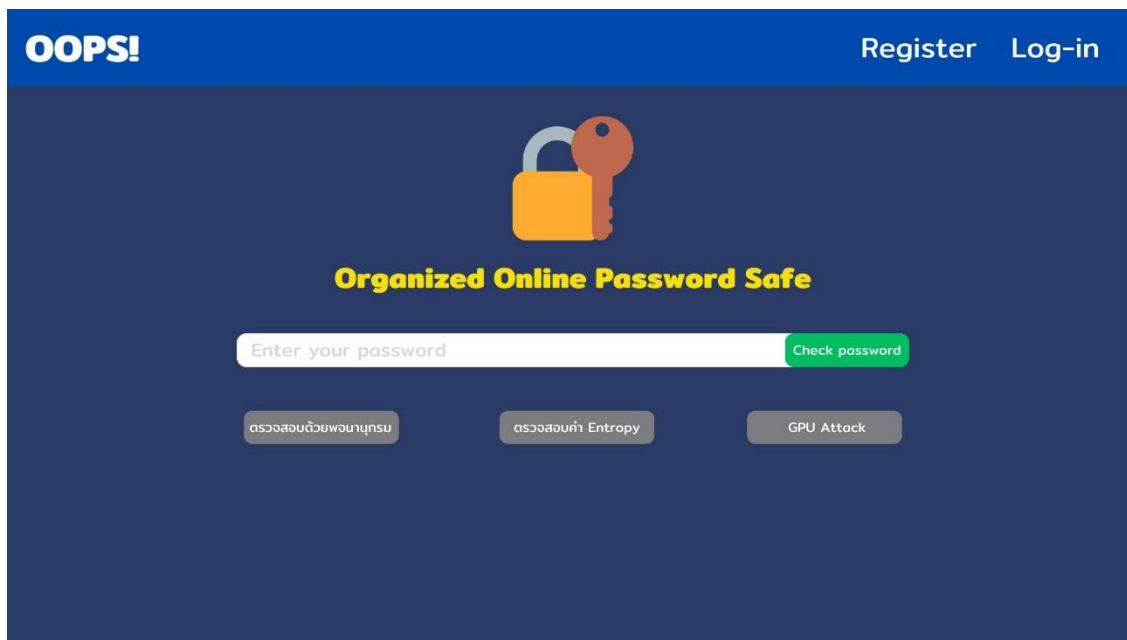


ภาพประกอบที่ 1.1 ภาพรวมของระบบ

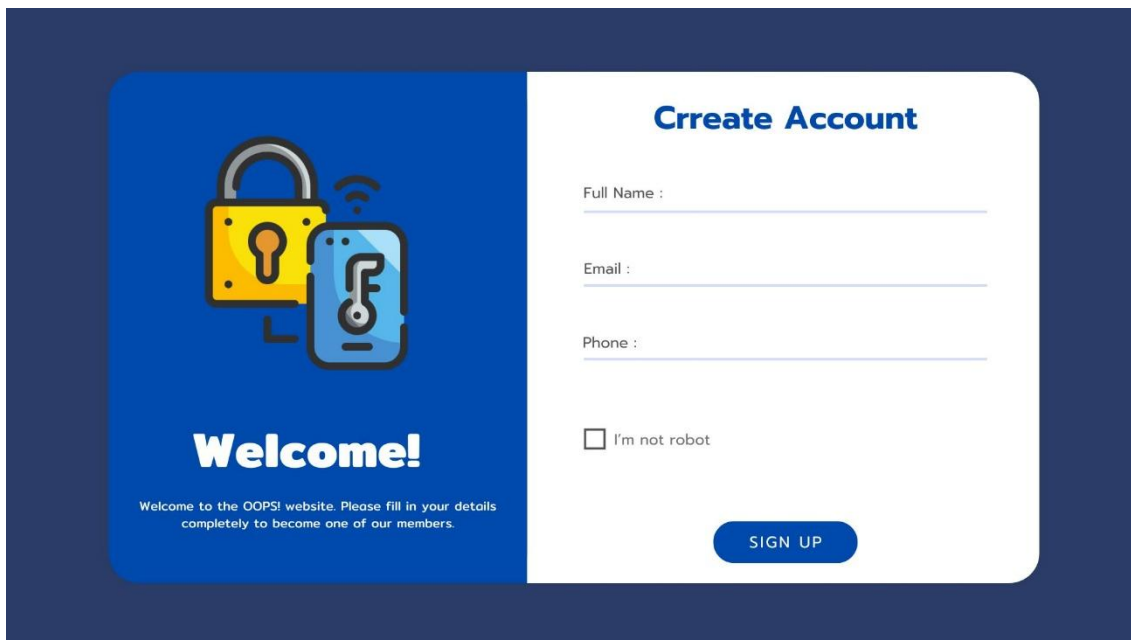
OOPS! เป็นแพลตฟอร์มออนไลน์ที่ออกแบบมาเพื่อช่วยจัดเก็บข้อมูลสำคัญต่างๆ บนสื่อออนไลน์ ไม่ว่าจะเป็นชื่อผู้รหัสผ่านหรือข้อมูลสำคัญอื่นๆ ของผู้ใช้งานโดยจะมีการเข้ารหัสเพื่อเพิ่มความปลอดภัยข้อมูลของผู้ใช้ซึ่งจะมีแต่ผู้ใช้นั้นจะสามารถดูรหัสได้นอกจากนี้ยังมีฟังก์ชันประเมินความปลอดภัยของรหัสผ่านโดยจะมีการตรวจสอบจากค่า Entropy และการคำนวณการถูกโจมตีโดย GPU และจะมีการตรวจสอบไปยังเว็บไซต์ต่างๆ เช่น Rockyou ว่ารหัสผ่านของผู้ใช้เคยรั่วไหลหรือไม่

รูป หน้า ล็อกอิน

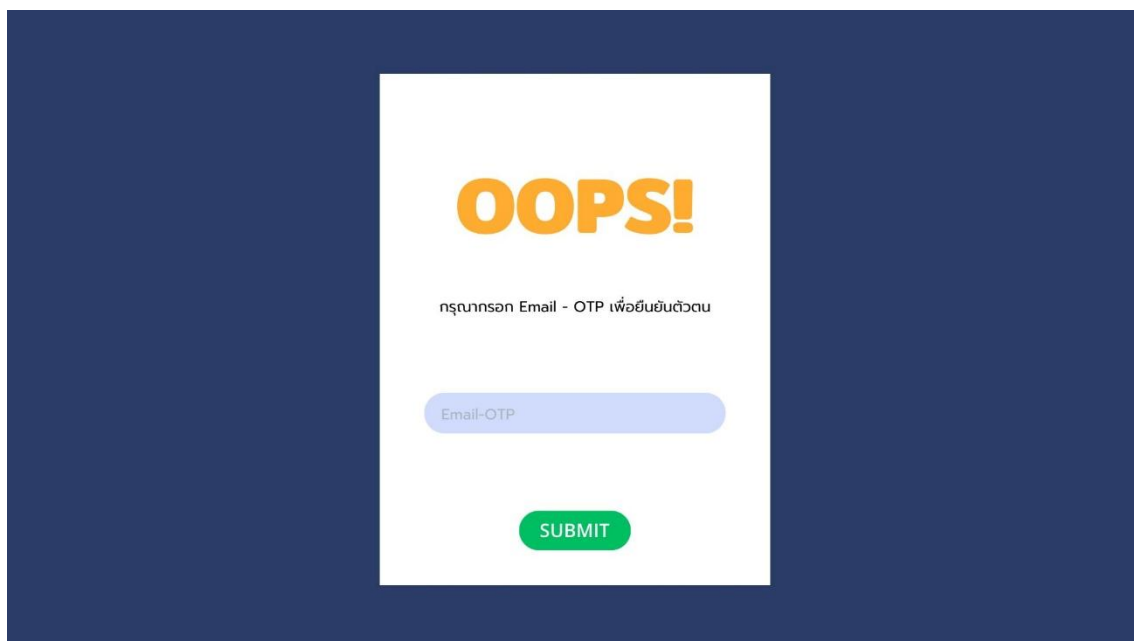
1.5 ตัวอย่างโปรแกรม OOPS!



ภาพประกอบที่ 1.2 ตัวอย่างหน้าเริ่มต้นของเว็บไซต์



ภาพประกอบที่ 1.3 สมัครเข้าใช้งานระบบ



The image shows a login interface with a dark blue background. In the center is a white card. At the top of the card, the word "OOPS!" is written in large, bold, orange letters. Below it, the text "กรุณาทกรอก Email - OTP เพื่อยืนยันตัวตน" (Please enter Email - OTP to verify identity) is displayed in black. There is a light blue rounded rectangular input field containing the placeholder text "Email-OTP". Below the input field is a green rounded rectangular button with the word "SUBMIT" in white capital letters.

ภาพประกอบที่ 1.4 ล็อกอินครั้งแรก



หน้าจอ ล็อกอินสำเร็จ

Log-in

OOPS!

Enter email

M-OTP

[Request Email-OTP](#)

SUBMIT

ภาพประกอบที่ 1.5 ยืนยันตัวตน

ผู้ใช้งานกรอกอีเมลเพื่อรับรหัส OTP เพื่อใช้ในการยืนยันตัวตน

Log-in

OOPS!

กรุณาเชื่อมต่ออีเมลกับแอปพลิเคชัน Authenticator

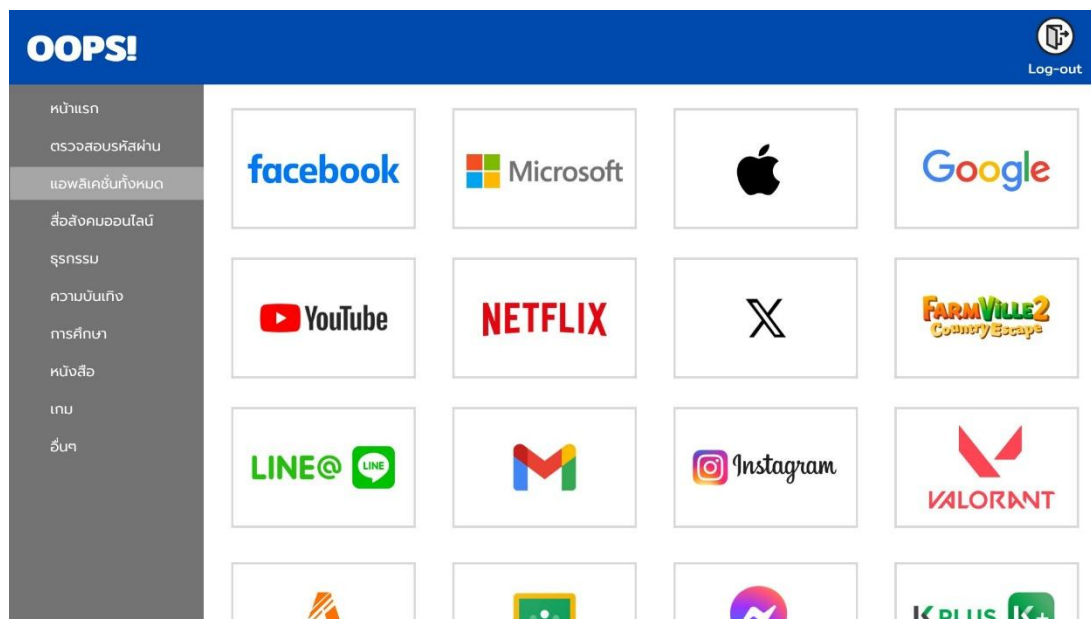
เพื่อรับ M-OTP

กลับไปหน้าแรก

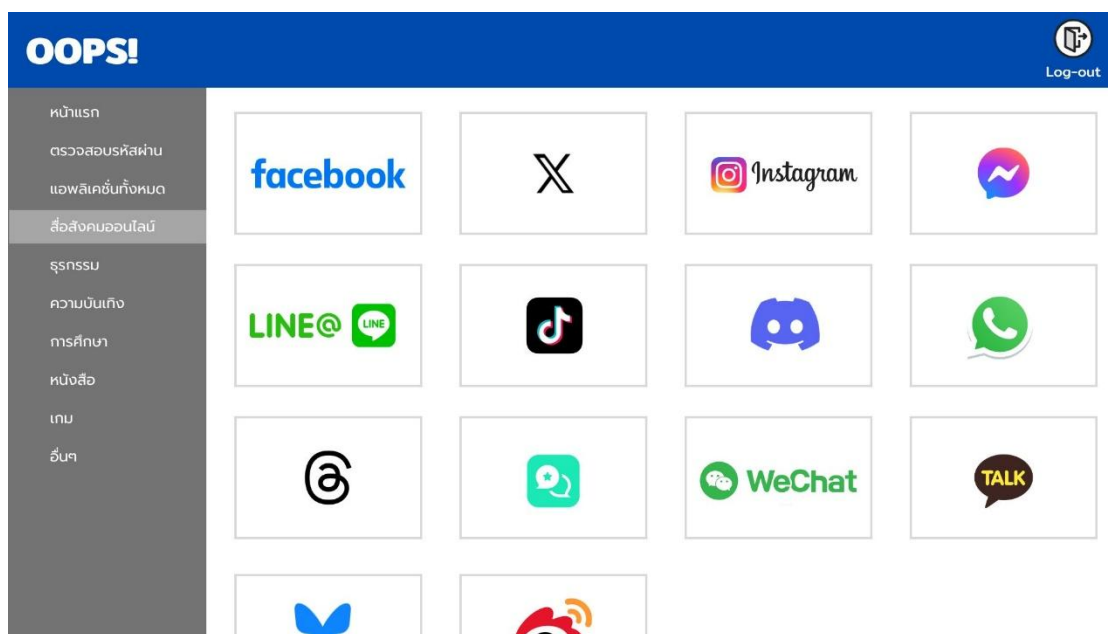
ภาพประกอบที่ 1.6 เชื่อม Authenticator

ระบบจะบังคับให้เชื่อมต่อกับแอปพลิเคชัน Authenticator เพื่อรับรหัส M-OTP มาใช้เข้าสู่ระบบ

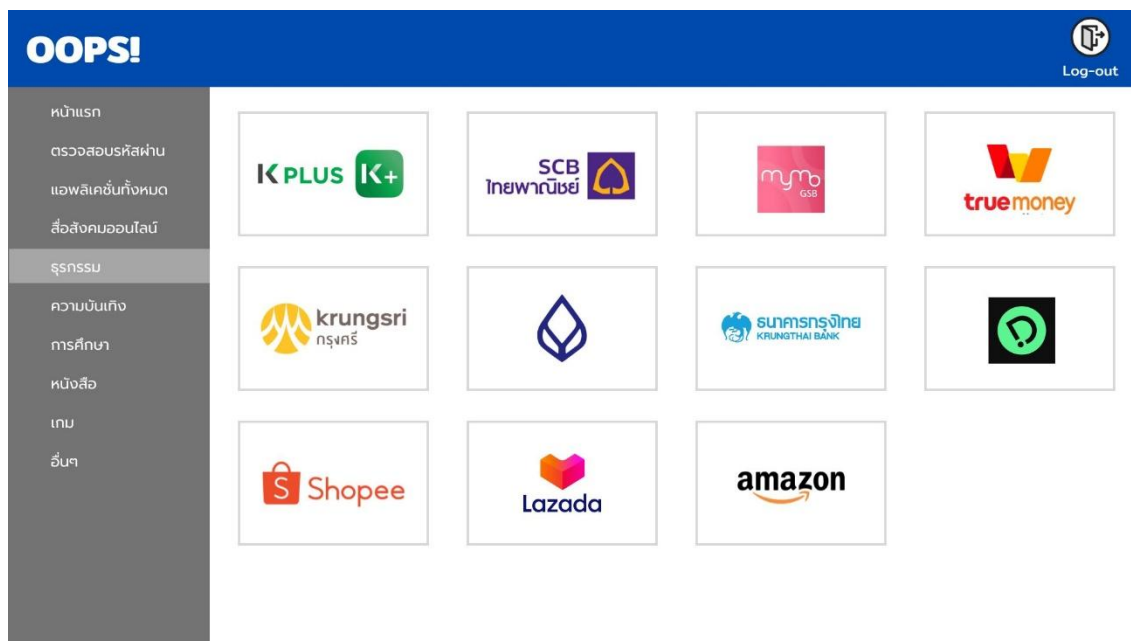
สวัสดี



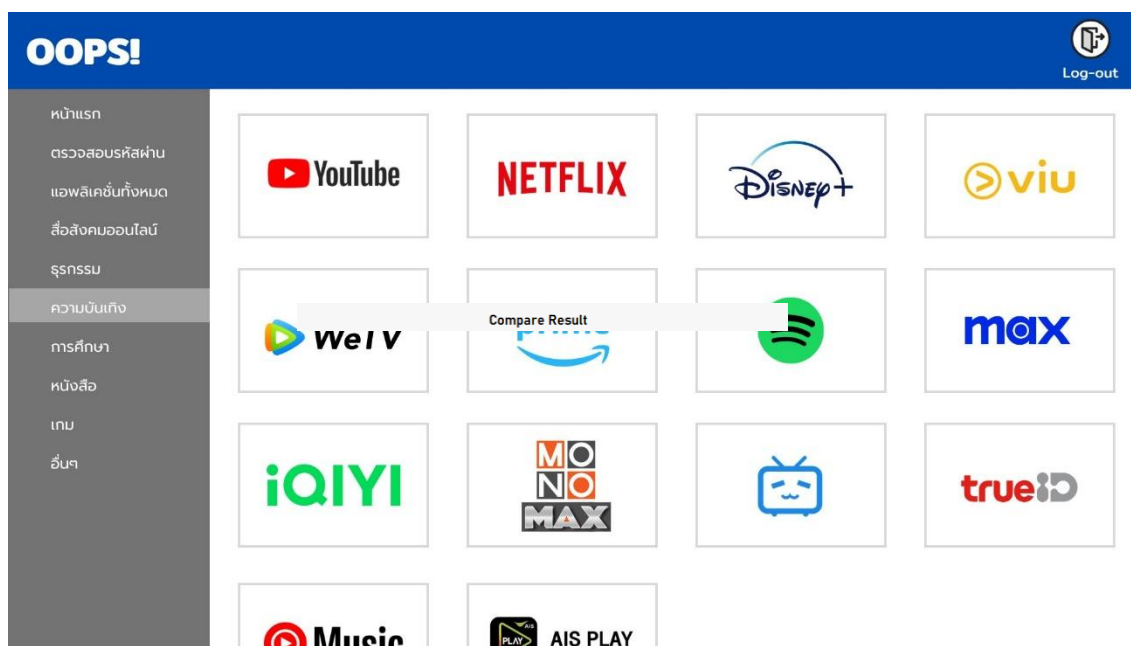
ภาพประกอบที่ 1.7 หน้าแอปพลิเคชันทั้งหมด



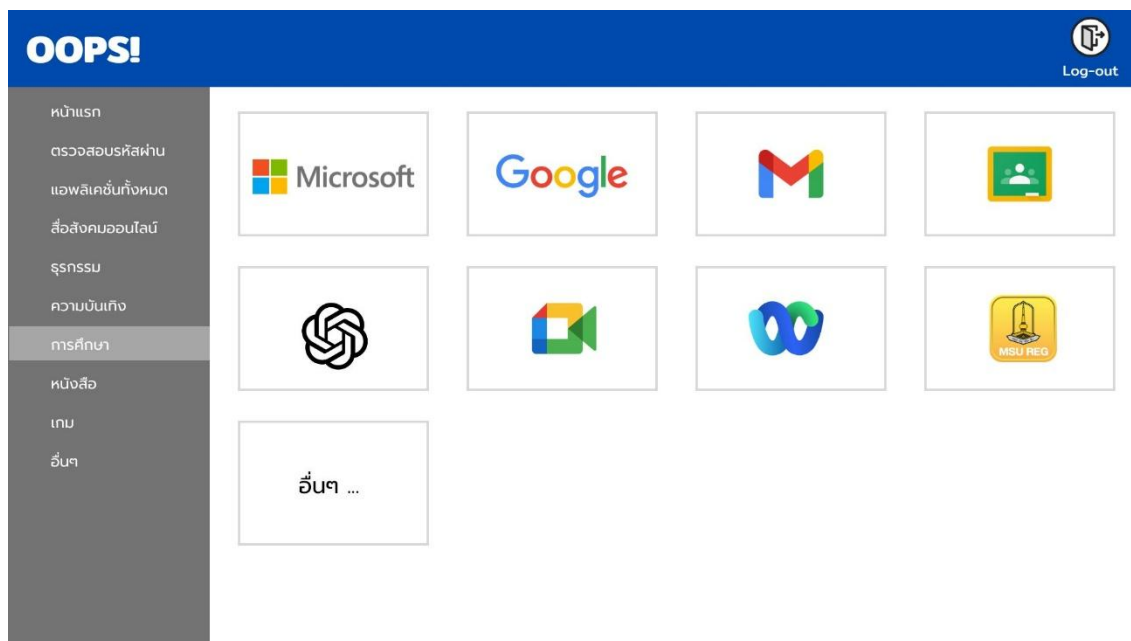
ภาพประกอบที่ 1.8 หน้าสื่อสังคมออนไลน์



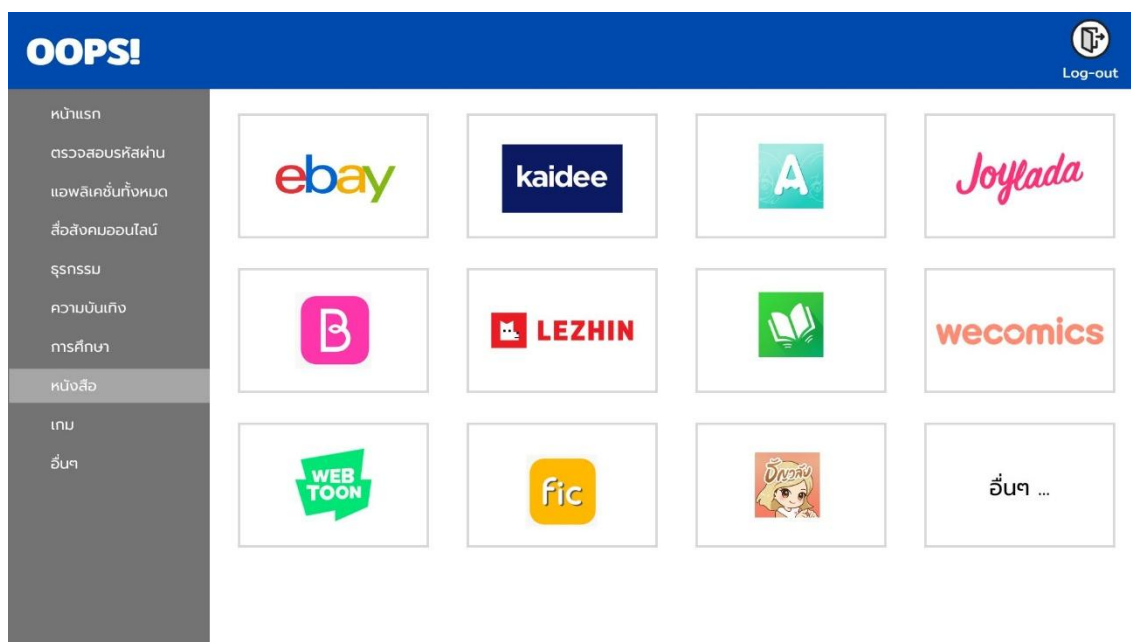
ภาพประกอบที่ 1.9 หน้าธุรกรรม



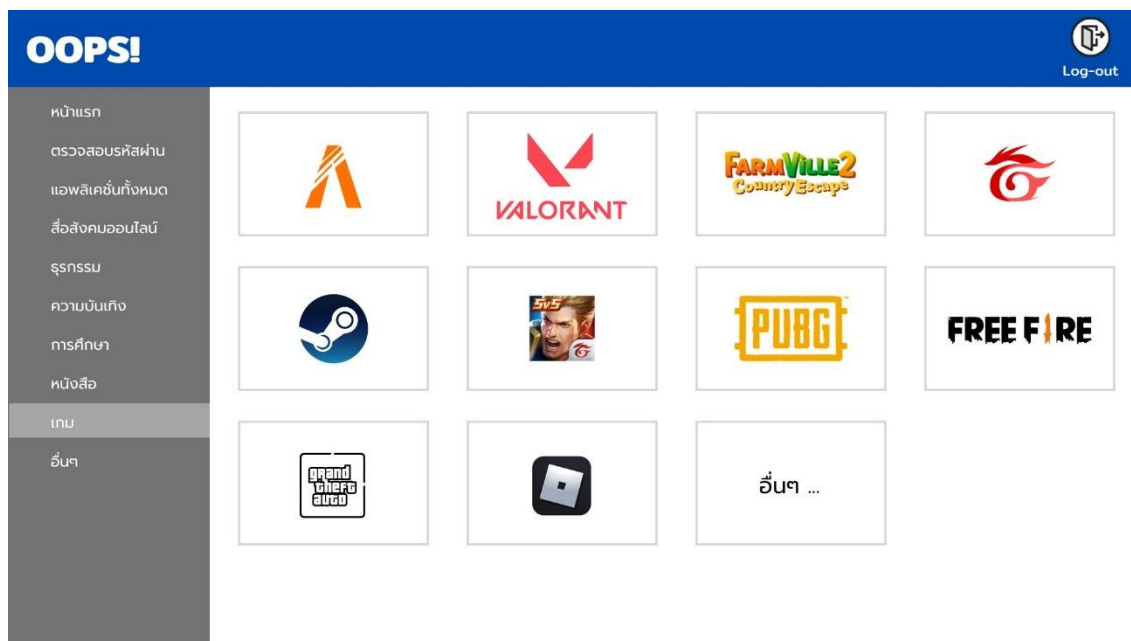
ภาพประกอบที่ 1.10 หน้าความบันเทิง



ภาพประกอบที่ 1.11 หน้าการศึกษา



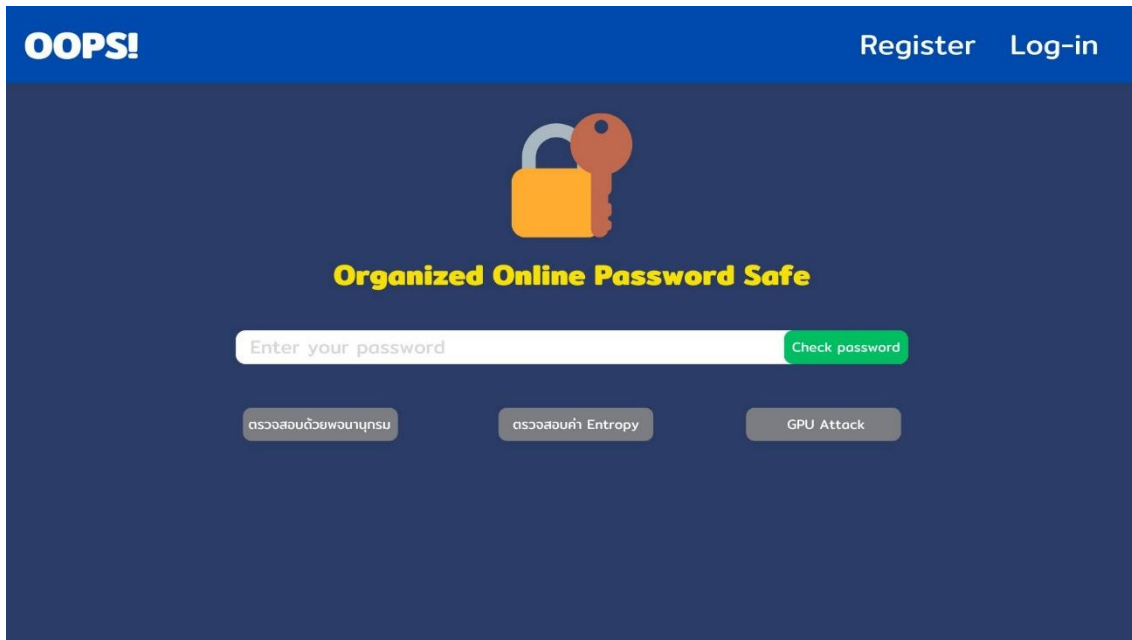
ภาพประกอบที่ 1.12 หน้าหนังสือ



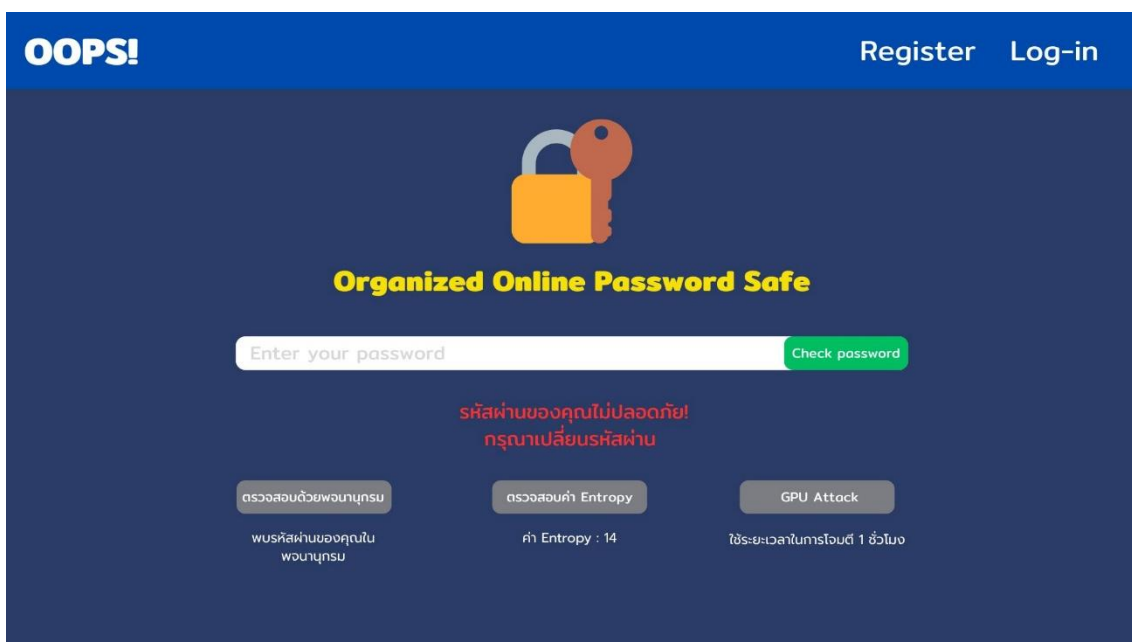
ภาพประกอบที่ 1.13 หน้าเกม



ภาพประกอบที่ 1.14 หน้าอื่นๆ



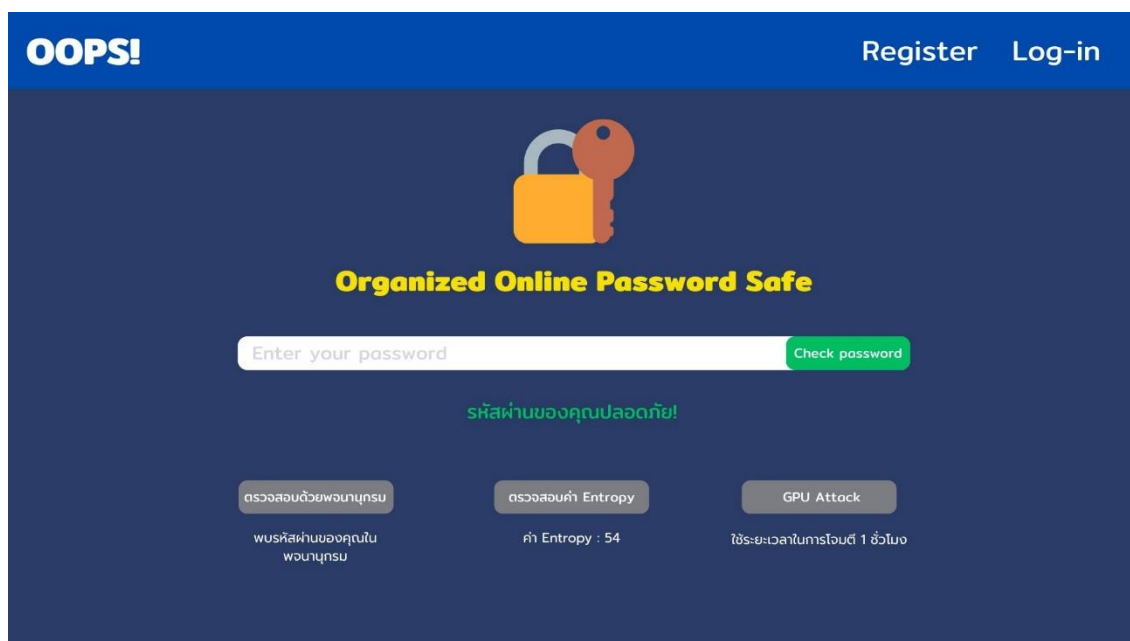
ภาพประกอบที่ 1.15 ตรวจสอบรหัสผ่าน



ภาพประกอบที่ 1.16 รหัสผ่านไม่ปลอดภัย

หากตรวจสอบแล้วพบว่ารหัสผ่านไม่ปลอดภัย จะมีแจ้งค่า Entropy ระยะเวลาที่จะถูกโจมตี และแจ้งเตือนให้เปลี่ยนรหัสผ่านทันที

6 พัดดาภิเษก บทที่ 1.16 รหัสผ่านไม่ปลอดภัย
บทที่ 2 ข้อ 4



ภาพประกอบที่ 1.17 รหัสผ่านปลอดภัย

หากตรวจสอบแล้วพบว่ารหัสผ่านปลอดภัย จะมีแจ้งค่า Entropy ระยะเวลาที่จะถูกโจมตี

1.6 ประโยชน์ที่คาดว่าจะได้รับ

- 1) แอปพลิเคชันจัดเก็บรหัสผ่านด้วยการเข้ารหัสระดับสูง ช่วยป้องกันการเข้าถึงข้อมูลโดยไม่ได้รับอนุญาตมีฟังก์ชันแจ้งเตือนหากพบรหัสผ่านที่มีความเสี่ยงต่อการถูกโจมตี
- 2) แอปพลิเคชันสามารถสร้างรหัสผ่านที่ซับซ้อนและคาดเดาได้ยาก เพื่อเพิ่มความปลอดภัยในการใช้งาน ลดปัญหาการตั้งรหัสผ่านซ้ำหรือการตั้งคำรหัสผ่านที่อ่อนแอ
- 3) ผู้ใช้งานสามารถจัดเก็บและจัดการรหัสผ่านทั้งหมดได้ในที่เดียว โดยไม่ต้องจดจำรหัสผ่านหลายชุด
- 4) แอปพลิเคชัน OOPS! (Organize Online Password Save) เป็นเครื่องมือที่มีประโยชน์สำหรับผู้ใช้งานทุกกลุ่ม ตั้งแต่บุคคลทั่วไปไปจนถึงองค์กร โดยเฉพาะผู้ที่ต้องการเพิ่มความปลอดภัยในการจัดการรหัสผ่านและการเข้าสู่ระบบอย่างสะดวกและมั่นใจในทุกการใช้งานออนไลน์.

1.7 อุปกรณ์และเครื่องมือที่ใช้ดำเนินการ

1.7.1 Hardware

- 1) เครื่องคอมพิวเตอร์ ที่มีคุณสมบัติ ดังนี้
 - เชื่อมต่ออินเทอร์เน็ตได้
 - ใช้เว็บ web browser ได้
- 2) โทรศัพท์มือถือ ที่มีคุณสมบัติ ดังนี้
 - สัมผัสหน้าจอได้

- เชื่อมต่ออินเทอร์เน็ตได้
- มีบัญชีอีเมล

1.7.2 Software

- 1) Visual Studio Code ใช้เขียนโปรแกรม
- 2) Google Authenticator ใช้รหัส M-OTP

1.8 แผนการดำเนินการ

โครงการปริญญานิพนธ์ฉบับนี้ ดำเนินงาน ณ คณะวิทยาการสารสนเทศ มหาวิทยาลัยมหาสารคาม ระหว่างเดือน ตุลาคม 2565 ถึง กันยายน 2566

ตารางที่ 1.1 แผนการดำเนินงาน

กิจกรรม	เดือน											
	ต.ค.	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย.	พ.ค.	มิ.ย.	ก.ค.	ส.ค.	ก.ย.
1. ศึกษาและรวบรวมข้อมูล												
2. วิเคราะห์และกำหนดขอบเขต												
3. ออกแบบระบบ												
4. พัฒนาโปรแกรม												
5. ทดสอบระบบ												
6. ทำรายงานสรุป												
7. นำเสนอโครงการ												

30 ชั่วโมง * ตรงกับที่ผู้เขียน

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

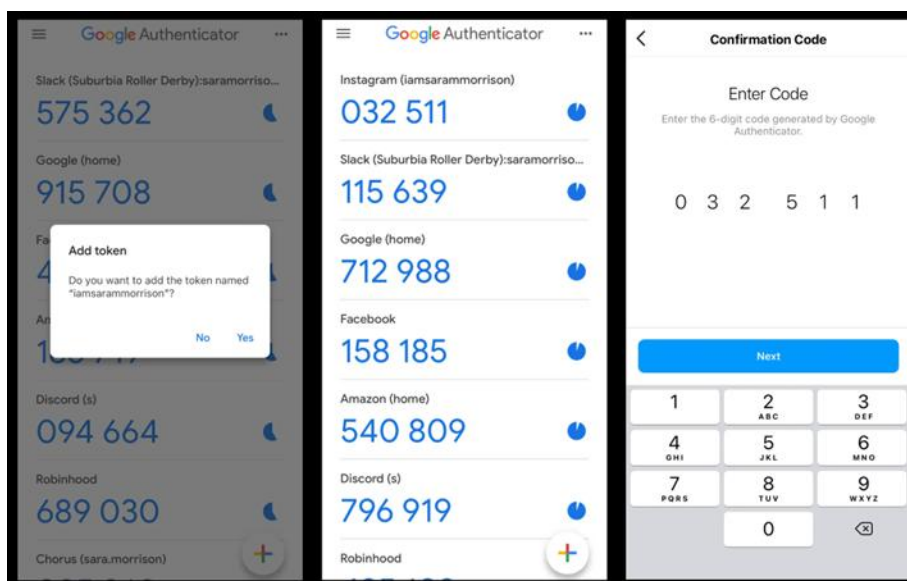
2.1 งานวิจัยและทฤษฎีที่เกี่ยวข้อง

2.1.1 การยืนยันตัวตน 2 ชั้น (2FA)

Two-Factor Authentication หรือ 2FA เป็นกระบวนการรักษาความปลอดภัยที่เสริมการป้องกันข้อมูลส่วนบุคคลหรือบัญชีออนไลน์ โดยผู้ใช้งานต้องผ่านการยืนยันตัวตนในสองขั้นตอนก่อนที่จะสามารถเข้าถึงระบบได้ในการเปิด 2FA การยืนยันตนเองสองขั้นตอนนี้ จะช่วยเพิ่มระดับความปลอดภัยขั้นที่ 2 ให้กับบัญชีของท่านเวลา ล็อกอิน เปลี่ยนรหัส ชื่อขาย ถอนเงิน และถอนเหรียญ ระบบนี้จะช่วยป้องกันบัญชีของท่านจากคนอื่น ๆ ที่มี อีเมล/รหัสผ่านของท่าน

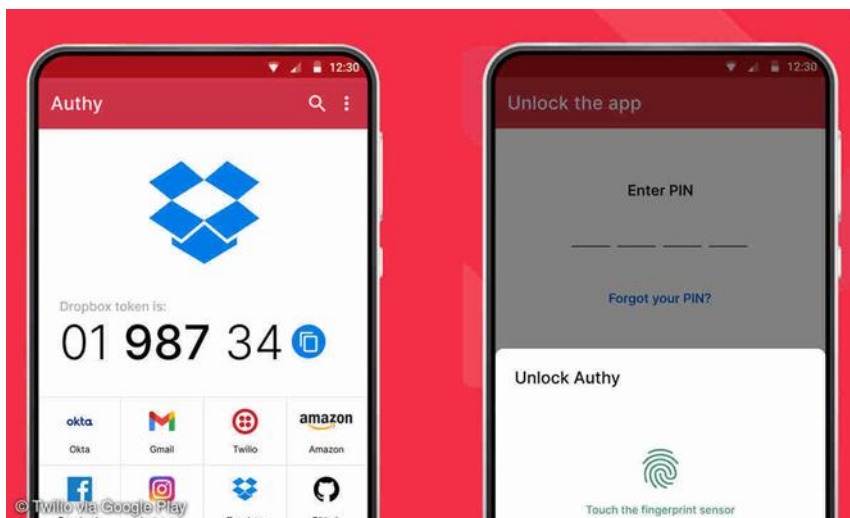
2.1.2 M-OTP

M-OTP (Mobile One-Time Password) คือระบบสร้างรหัสผ่านแบบใช้ครั้งเดียวบนมือถือ โดย M-OTP เป็นส่วนสำคัญของการยืนยันตัวตนแบบสองปัจจัย (2FA) ที่ช่วยให้การเข้าถึงข้อมูลหรือบริการออนไลน์มีความปลอดภัยมากขึ้น โดยรหัสมีอายุการใช้งานสั้นและใช้ได้ครั้งเดียว ลดความเสี่ยงจากการโจมตี เช่น ฟิชซิง และการดักจับข้อมูล ผู้ใช้เพียงติดตั้งแอปที่รองรับและลงทะเบียนกับบัญชี เพื่อใช้สร้างรหัส OTP สำหรับยืนยันตัวตน เหมาะสำหรับธุรกรรมหรือระบบที่ต้องการความปลอดภัยสูง ทั้งสะดวกและปลอดภัยกว่าการใช้รหัสผ่านแบบเดิมหรือ OTP ผ่าน SMS



ได้อัปเกรด ทน cyber attack 100%

ภาพประกอบที่ 2.1 ตัวอย่างการใช้งานแอปพลิเคชัน Authenticator



ภาพประกอบที่ 2.2 ตัวอย่างการใช้งานแอปพลิเคชัน Authy

2.1.3 การโจมตีรหัสผ่าน

Dictionary Attack เป็นการสุ่มเดา Password จากไฟล์ที่มีการรวบรวมคำศัพท์ต่างๆ ที่พบอยู่ใน Dictionary และคำศัพท์ที่สามารถพบได้บ่อยครั้ง ซึ่งจัดว่าเป็นวิธีการที่ถูกนำมาใช้กับ Password ของผู้ใช้งานมากที่สุด นี่ถือว่าเป็นเวกชัน ที่ไม่ต้องใช้ประสบการณ์มากนัก เมื่อเทียบกับเวกชันที่โหดสุดๆ อย่างการโจมตีแบบ Brute Force Attack แต่ก็ยังคงต้องอาศัยแฮกเกอร์ที่ต้องกระหน่ำโจมตีระบบด้วยการทดลองใส่ Password ไปเรื่อยๆ จนกว่าจะสำเร็จ

Brute Force Attack การโจมตีแบบ Brute Force Attack สามารถสืบหาคำศัพท์ที่ไม่ใช่คำศัพท์จากพจนานุกรมได้ เช่น รหัสผ่านที่มีการผสมระหว่างตัวเลขและตัวอักษร ซึ่งก็หมายความว่า รหัสผ่านที่มีความต่อเนื่อง อย่างเช่น "aaa1" หรือ "zzz10" ก็อาจจะมีความเสี่ยงจากการโจมตีด้วยวิธี Brute force Attack ได้มากขึ้น หรือจะเรียกการโจมตีแบบนี้ว่าการเดาก็ได้

Phishing คือการโจมตีทางไซเบอร์ที่ผู้ไม่หวังดีพยายามหลอกลวงเหยื่อให้เปิดเผยข้อมูลสำคัญ เช่น รหัสผ่านหรือข้อมูลทางการเงิน โดยแอบอ้างเป็นองค์กรหรือบุคคลที่น่าเชื่อถือผ่านช่องทางต่างๆ เช่น อีเมล ข้อความ หรือเว็บไซต์ปลอม

2.1.4 Entropy

การประเมินความแข็งแกร่งของรหัสผ่าน (Password Strength) ด้วยแนวคิดเอนโทรปี (Entropy) เป็นวิธีที่ใช้กันอย่างแพร่หลายในงานด้านความมั่นคงปลอดภัยทางสารสนเทศ โดยมีวัตถุประสงค์เพื่อวัดระดับ

ความไม่แน่นอนหรือความยากในการคาดเดารหัสผ่าน อย่างไรก็ตาม วิธีการคำนวณเอนโทรปีมีหลายรูปแบบ ซึ่งให้ผลลัพธ์ที่แตกต่างกันตามสมมติฐานของแบบจำลองที่ใช้

- Shannon Entropy เป็นการวัดความไม่แน่นอนของข้อมูลโดยอาศัยความน่าจะเป็นของตัวอักษรที่ปรากฏในรหัสผ่าน หากรหัสผ่านมีการกระจายตัวของตัวอักษรอย่างสม่ำเสมอและไม่มีรูปแบบซ้ำจะให้ค่าเอนโทรปี ดังนั้นแนวคิดนี้จึงเน้นการวัด “ความสุ่มทางสถิติ” ของข้อมูลเป็นหลัก
- Uniform Random Model เป็นการประเมินโดยสมมติว่าตัวอักษรทุกตัวมีโอกาสถูกเลือกเท่ากัน ความแข็งแกร่งของรหัสผ่านจึงขึ้นอยู่กับความยาวของรหัสผ่านและขนาดของชุดตัวอักษรที่ใช้
- Min-Entropy เป็นการวัดความแข็งแกร่งในกรณีที่เลวร้ายที่สุด โดยพิจารณาจากรหัสผ่านที่มีความน่าจะเป็นถูกเลือกมากที่สุด หากมีรหัสผ่านที่ถูกใช้ซ้ำจำนวนมาก ระบบจะถือว่ามีความเสี่ยงสูง วิธีนี้จึงเหมาะสำหรับการวิเคราะห์ในเชิง worst-case scenario แต่ยังไม่สามารถสะท้อนรูปแบบการโจมตีในทางปฏิบัติได้อย่างครบถ้วน
- Guessing Entropy หรือ Attack-based Model ซึ่งประเมินความแข็งแกร่งของรหัสผ่านจากจำนวนครั้งเฉลี่ยที่ผู้โจมตีต้องใช้ในการคาดเดารหัสผ่านให้ถูกต้อง วิธีนี้สะท้อนกระบวนการโจมตีจริงมากกว่าการวัดความสุ่มของข้อมูลเพียงอย่างเดียว

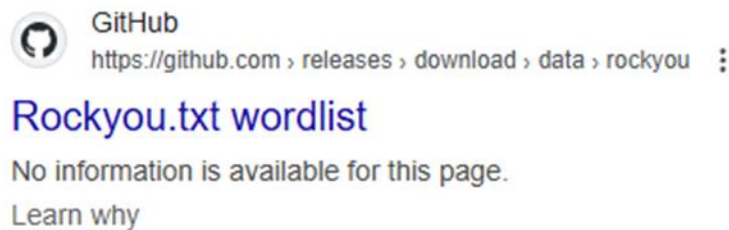
ในการพัฒนาครั้งนี้เลือกใช้แนวทางจากงานวิจัยเรื่อง *An analysis of password security risk against dictionary attacks* ได้เสนอแนวคิดที่แตกต่างจากเอนโทรปีแบบดั้งเดิม โดยเน้นการประเมินความแข็งแกร่งของรหัสผ่านจากความยากในการถูกโจมตีด้วย dictionary attack แทนการวัดความสุ่มของตัวอักษร โดยพิจารณาว่ารหัสผ่านอยู่ในลำดับที่เท่าใดของการเดาภายในชุดพจนานุกรมหรือฐานข้อมูลรหัสผ่านที่ใช้ในการโจมตี แนวทางนี้ทำให้การประเมินความปลอดภัยสอดคล้องกับพฤติกรรมโจมตีในโลกความเป็นจริงมากยิ่งขึ้น

อ้างอิง :

https://www.researchgate.net/publication/366191170_An_analysis_of_password_security_risk_against_dictionary_attacks

2.1.5 Rockyou.txt

RockYou.txt คือไฟล์ที่ประกอบด้วยรหัสผ่านจำนวนมากมหาศาลที่ถูกเปิดเผยจากการโจมตีทางไซเบอร์ครั้งใหญ่ในปี 2009 ต่อบริษัท RockYou ซึ่งเป็นบริษัทที่ให้บริการแอปพลิเคชันออนไลน์และเกมบนโซเชียลมีเดีย การโจมตีดังกล่าวทำให้ข้อมูลผู้ใช้งานรั่วไหลกว่า 14.3 ล้านรหัสผ่าน (14,344,188 รายการ) จาก 32 ล้านบัญชีรั่วไหลออกมาในรูปแบบข้อความธรรมดา (plain text) โดยไม่ได้รับการเข้ารหัส (unencrypted)



ภาพประกอบที่ 2.3 Rockyou.txt

2.1.6 Rockyou2024

RockYou2024 เป็นไฟล์ร่วไหลที่มีรหัสผ่านกว่า 9.9 พันล้านรายการ ถือว่าเป็นการรวบรวมรหัสผ่านที่ใหญ่ที่สุดในประวัติศาสตร์ ไฟล์นี้มาจากข้อมูลที่รั่วไหลทั้งเก่าและใหม่ โดยอาจนำไปใช้ในการโจมตีแบบ brute-force หรือ credential stuffing ซึ่งเสี่ยงต่อการถูกเข้าถึงบัญชีออนไลน์โดยไม่ได้รับอนุญาต นอกจากนี้ยังเพิ่มโอกาสในการโจรกรรมข้อมูลส่วนตัวและการทุจริตทางการเงิน ผู้ใช้งานจึงควรเปลี่ยนรหัสผ่าน เปิดใช้งานการยืนยันตัวตนหลายขั้นตอน (Multi-Factor Authentication: MFA) และใช้โปรแกรมจัดการรหัสผ่าน เพื่อเพิ่มความปลอดภัย [5]

อย่างไรก็ตาม แม้ RockYou2024 จะเป็นฐานข้อมูลรหัสผ่านที่มีประโยชน์สำหรับงานวิจัยด้านความปลอดภัยหรือการทดสอบความแข็งแรงของรหัสผ่าน แต่ไม่เหมาะสำหรับการนำมาใช้เป็น Dictionary ในการค้นหารหัสผ่านด้วยตนเอง เนื่องจากขนาดของไฟล์มีจำนวนข้อมูลมหาศาลเกินกว่า 9.9 พันล้านรายการ ทำให้ต้องใช้ทรัพยากรด้านการประมวลผล หน่วยความจำ และพื้นที่จัดเก็บข้อมูลในระดับสูง

หากยังใช้สถาปัตยกรรมแบบเดิมในการค้นหาข้อมูล เช่น การ Query ตรงผ่านฐานข้อมูล MongoDB หรือการเก็บข้อมูลทั้งหมดไว้ใน Redis จะทำให้ระบบเกิดปัญหาคอขวด (Bottleneck) ได้ทันที เนื่องจากขนาดของข้อมูลและดัชนี (Index) มีขนาดใหญ่เกินกว่าที่หน่วยความจำของเซิร์ฟเวอร์ทั่วไปจะรองรับได้ ตัวอย่างเช่น การสร้าง Index สำหรับข้อมูลระดับ 10,000 ล้านรายการ อาจทำให้ขนาดของ Index เพิ่มขึ้นมากกว่า 200–300 GB หากหน่วยความจำหลัก (RAM) ไม่เพียงพอ MongoDB จะต้องสลับการอ่านข้อมูลไปยังดิสก์อยู่ตลอดเวลา (Disk Thrashing) ส่งผลให้ประสิทธิภาพในการค้นหาลดลงอย่างมาก

ในขณะเดียวกัน การใช้ Redis เพื่อเก็บข้อมูลทั้งหมดไว้บนหน่วยความจำโดยตรงก็ไม่เหมาะสมเช่นกัน เนื่องจากการจัดเก็บ String จำนวนหลายพันล้านรายการจำเป็นต้องใช้ RAM ขนาดหลายร้อยกิกะไบต์ ซึ่งทำให้ต้นทุนของเซิร์ฟเวอร์หรือ Cloud Infrastructure สูงมากและไม่คุ้มค่าต่อการพัฒนาในระดับโครงการทั่วไป

ในขณะที่ระบบ Have I Been Pwned (HIBP) จึงใช้สถาปัตยกรรมแบบ k-Anonymity โดยนำรหัสผ่านทั้งหมดมาแปลงเป็น SHA-1 Hash แล้วใช้เพียง 5 ตัวแรกของ Hash เป็น Key ในการค้นหา ทำให้ลดขนาดข้อมูลที่ต้องค้นหาในแต่ละครั้งจากระดับหลายพันล้านรายการเหลือเพียงหลักร้อยหรือหลักพันรายการ ช่วยเพิ่มประสิทธิภาพและยังปกป้องความเป็นส่วนตัวของผู้ใช้งาน

อย่างไรก็ตาม ระบบลักษณะนี้ยังคงต้องอาศัย Big Data Infrastructure เช่น Distributed Storage, Parallel Processing และระบบ Cache ขนาดใหญ่ เพื่อรองรับข้อมูลมหาศาล ดังนั้นโครงงานนี้จึงเลือกใช้ API ของ HIBP แทนการสร้างระบบ Dictionary ขนาดใหญ่ด้วยตนเอง เนื่องจากช่วยลดต้นทุน ลดภาระของเซิร์ฟเวอร์ และสามารถใช้งานฐานข้อมูลรหัสผ่านทั่วโลกที่มีการอัปเดตอย่างต่อเนื่องได้อย่างมีประสิทธิภาพ

2.1.7 HIBP

Have I Been Pwned (HIBP) ย่อมาจาก “Have I Been Pwned” ซึ่งเป็นเว็บไซต์ที่พัฒนาโดย Troy Hunt นักวิจัยด้านความปลอดภัยไซเบอร์ชาวออสเตรเลีย เว็บไซต์นี้ถูกสร้างขึ้นเพื่อช่วยให้ผู้ใช้งานสามารถตรวจสอบได้ว่าอีเมลหรือรหัสผ่านของตนเคยปรากฏอยู่ในเหตุการณ์ข้อมูลรั่วไหล (Data Breach) หรือไม่ โดยระบบจะเปรียบเทียบข้อมูลของผู้ใช้กับฐานข้อมูลรหัสผ่านและบัญชีที่ถูกเปิดเผยจากการโจมตีทางไซเบอร์หรือการรั่วไหลของข้อมูลจากเว็บไซต์ต่าง ๆ ทั่วโลก นอกจากนี้ HIBP ยังถูกใช้อย่างแพร่หลายในการตรวจสอบความปลอดภัยของบัญชีผู้ใช้งานและลดความเสี่ยงจากการโจมตีแบบ Credential Stuffing และ Brute-force Attack [6]

2.1.8 Script Hash Function : AES256 bits

Script Hash Function คืออัลกอริทึมสำหรับการเข้ารหัสรหัสผ่าน (Password Hashing Algorithm) ที่ถูกออกแบบมาเพื่อเพิ่มความปลอดภัยในการจัดเก็บรหัสผ่านภายในฐานข้อมูล โดยมีคุณสมบัติสำคัญคือเป็น Memory-Hard Function ซึ่งต้องใช้หน่วยความจำ (Memory) จำนวนมากระหว่างกระบวนการประมวลผล ทำให้ยากต่อการโจมตีด้วยฮาร์ดแวร์ประสิทธิภาพสูง เช่น GPU หรือ ASIC

AES หรือ Advanced Encryption Standard เป็นมาตรฐานการเข้ารหัสข้อมูลแบบสมมาตร (Symmetric Encryption) ที่ได้รับการยอมรับและใช้งานอย่างแพร่หลายในระดับสากล เนื่องจากมีประสิทธิภาพสูงและมีความปลอดภัยในการป้องกันการโจมตีข้อมูล ปัจจุบัน AES สามารถทำงานได้หลายโหมดซึ่งแต่ละโหมดมีลักษณะการทำงานแตกต่างกันดังนี้

- ECB (Electronic Codebook) เป็นโหมดพื้นฐาน แต่มีข้อเสียคือข้อมูลที่เหมือนกันจะถูกเข้ารหัสออกมาเหมือนกัน ทำให้ไม่ปลอดภัย
- CBC (Cipher Block Chaining) ช่วยเพิ่มความปลอดภัยมากขึ้นโดยเชื่อมข้อมูลแต่ละบล็อกเข้าด้วยกัน แต่ไม่สามารถตรวจสอบความถูกต้องของข้อมูลได้
- CTR (Counter Mode) มีความรวดเร็วสูงและรองรับการประมวลผลแบบขนาน
- GCM (Galois/Counter Mode) เป็นโหมดที่รวมทั้งการเข้ารหัสข้อมูลและการตรวจสอบความถูกต้องของข้อมูลไว้ในตัวเดียวกัน

ในการพัฒนาระบบ ผู้จัดทำได้เลือกใช้อัลกอริทึมการเข้ารหัสข้อมูลแบบ AES-256-GCM สำหรับป้องกันข้อมูลสำคัญก่อนจัดเก็บลงฐานข้อมูล โดยระบบจะนำ Passphrase ของผู้ใช้งานมาใช้เป็นสารตั้งต้นในการสร้างกุญแจเข้ารหัส (Encryption Key) และนำกุญแจดังกล่าวไปใช้ในการเข้ารหัสข้อมูลก่อนบันทึกลงในฐานข้อมูล เพื่อเพิ่มความปลอดภัยและลดความเสี่ยงจากการเข้าถึงข้อมูลโดยไม่ได้รับอนุญาต

ผู้จัดทำเลือกใช้โหมด GCM (Galois/Counter Mode) เนื่องจากเป็นโหมดที่สามารถให้ทั้งการรักษาความลับของข้อมูล (Confidentiality) และการตรวจสอบความถูกต้องของข้อมูล (Authentication) ได้พร้อมกันภายในกระบวนการเดียว ทำให้สามารถตรวจสอบได้ว่าข้อมูลถูกแก้ไขหรือดัดแปลงระหว่างการจัดเก็บหรือส่งผ่านระบบหรือไม่ ซึ่งมีประสิทธิภาพและความปลอดภัยสูงกว่าโหมด ECB และ CBC ที่รองรับเพียงการเข้ารหัสข้อมูลเท่านั้น

นอกจากนี้ยังเลือกใช้ขนาดกุญแจแบบ 256-bit เนื่องจากมีจำนวนความเป็นไปได้ของกุญแจสูงมาก ส่งผลให้การโจมตีแบบ Brute-force ทำได้ยากและใช้เวลานานมหาศาล แม้จะใช้ GPU หรือ Supercomputer ในการประมวลผลก็ตาม อีกทั้ง AES-256 ยังมีความทนทานต่อแนวคิดการโจมตีจาก Quantum Computing มากกว่า AES-128 เนื่องจากอัลกอริทึมของ Quantum Computer เช่น Grover's Algorithm จะลดระดับความปลอดภัยของคีย์ลงได้เพียงบางส่วน ทำให้ AES-256 ยังคงมีระดับความปลอดภัยเพียงพอสำหรับการใช้งานในปัจจุบันและอนาคต

2.1.9 GPU (Graphics Processing Unit)

หน่วยประมวลผลกราฟิก (GPU) จะมีคอร์ขนาดเล็กกว่าทำหน้าที่พิเศษกว่ามากมาย คอร์เหล่านี้จะให้ประสิทธิภาพขนานใหญ่โดยทำงานร่วมกันและแบ่งงานการประมวลผลไปยังคอร์มากมายพร้อม ๆ กัน (หรือคู่ขนานกัน) GPU นั้นจะเป็นเลิศกับงานแบบคู่ขนานสูง เช่น การเรนเดอร์ภาพแสดงระหว่างเกมการเล่น การจัดแจงข้อมูลวิดีโอระหว่างการสร้างสรรค์เนื้อหา และการคำนวณผลลัพธ์ในเวิร์กโหลด AI แบบเข้มข้น

2.1.10 โปรเซสเซอร์ Intel core

โปรเซสเซอร์ Intel® Core™ Ultra ซึ่งเป็นสถาปัตยกรรมระดับพรีเมียมของเรา จะมอบประสิทธิภาพและประสิทธิภาพการใช้พลังงานสูงสุดให้ โปรเซสเซอร์ Intel® Core™ Ultra จะมี NPU แบบรวม ขณะที่บางข้อเสนอ2 จะมี GPU Intel® Arc™ ในตัว ผลลัพธ์ที่ได้คือโปรเซสเซอร์ตัวเดียวที่ช่วยให้ผู้ใช้เพลิดเพลินกับการเร่งความเร็ว AI ขั้นสูง เกมการเล่นที่สมจริง การใช้สื่อบน 4K

2.1.11 การ์ดจอ

เป็นหนึ่งในอุปกรณ์ที่สำคัญของคอมพิวเตอร์ ซึ่งบางครั้งก็เรียกว่า GPU การ์ดจอ ซึ่งย่อมาจาก Graphic Card นั่นเอง อุปกรณ์ชิ้นนี้จะช่วย CPU ในการประมวลผลพวกชุดคำสั่งต่าง ๆ ให้ออกมาเป็นสัญญาณภาพ ทั้งในรูปแบบของภาพและวิดีโอ การ์ดจอจะคล้ายกับ CPU แต่เหนือกว่ามากในแง่ของจำนวน Core ที่เข้ามาทำงานในเรื่องของความละเอียดภาพ

ประเภทของการ์ดจอ

1. การ์ดจอออนบอร์ด

การ์ดจอ On Board คือ การ์ดจอที่ติดมาพร้อมกับ CPU การ์ดจอประเภทนี้ก็พัฒนาขึ้นมา รองรับการทำงานพื้นฐานได้ดี ดูหนังฟังเพลงเล่นโซเชียลไม่มีสะดุด แต่ถ้าอยากทำงานกราฟิกหนัก ๆ หรือเล่นเกมล้ำ ๆ ก็อาจต้องพิจารณาเพิ่มเติมการ์ดจอเข้าไป หรือมองหา bit การ์ดจอ คือ ยิ่งมีค่าสูงยิ่งประมวลผลได้ดีมากขึ้นเท่านั้น



ภาพประกอบที่ 2.4 ตัวอย่างการ์ดจอออนบอร์ด

2.การ์ดจอเล่นเกม

แน่นอนว่าการ์ดจอเล่นเกมออกแบบมาเพื่อให้การเล่นของคุณลื่นไหลมากขึ้น โดยจะแยกออกมาจาก CPU โดยเฉพาะ ราคาอาจแรง แต่ก็ต้องยอมรับในเรื่องคุณภาพ เพราะมาพร้อมการประมวลผลสุดล้ำ เทคโนโลยีสุดทันสมัย ลดภาพฉีกขาด เล่นเกมลื่นไหล เล่นเกมเพลินที่สุดอย่างแน่นอน



ภาพประกอบที่ 2.5 ตัวอย่างการ์ดจอเล่นเกม

3. การ์ดจอกราฟิก

งานภาพกับการ์ดจอสุดล้ำเป็นของคู่กัน ถ้าเจอการ์ดจอที่ใช้ ไม่ว่าจะเป็นภาพแบบ 2 หรือ 3 มิติก็ประมวลผลให้คุณได้ตามที่ต้องการอย่างแน่นอน ถึงแม้ว่าจะมาพร้อมราคาที่สูงอยู่พอดู แต่ถ้าตอบโจทย์การทำงาน ก็สามารถสร้างรายได้ให้กับคุณ ดังนั้นถือว่าคุ้มค่ากับการลงทุน และยังมีการ์ดจอ Low Profile คือตัวเล็กที่น่าสนใจ เพื่อให้คุณสามารถใส่เคสที่เล็กลง เพิ่มความเท่ให้มูมคอมพิวเตอร์ได้อีกด้วย



ภาพประกอบที่ 2.6 ตัวอย่างการดจอกราฟฟิก

2.1.12 GeForce RTX 4090

เป็นกราฟิกการ์ดรุ่นล่าสุดจาก NVIDIA ในซีรีส์ GeForce RTX 40 ซึ่งใช้สถาปัตยกรรม Ada Lovelace โดยเปิดตัวในปี 2022 เพื่อรองรับการใช้งานในงานกราฟิกที่มีความซับซ้อนสูง เช่น เกม 4K, การเรนเดอร์ 3D, การตัดต่อวิดีโอ, และงานประมวลผล AI ที่ต้องการประสิทธิภาพที่สูงมาก RTX 4090 เป็นกราฟิกการ์ดที่ออกแบบมาให้สามารถรองรับเทคโนโลยี Ray Tracing และ DLSS (Deep Learning Super Sampling) ซึ่งทำให้ภาพกราฟิกมีความสมจริงและคุณภาพสูง โดยไม่กระทบกับประสิทธิภาพมากเกินไป



ภาพประกอบที่ 2.7 GEFORCE RTX 4090

2.2 Software ที่เกี่ยวข้อง

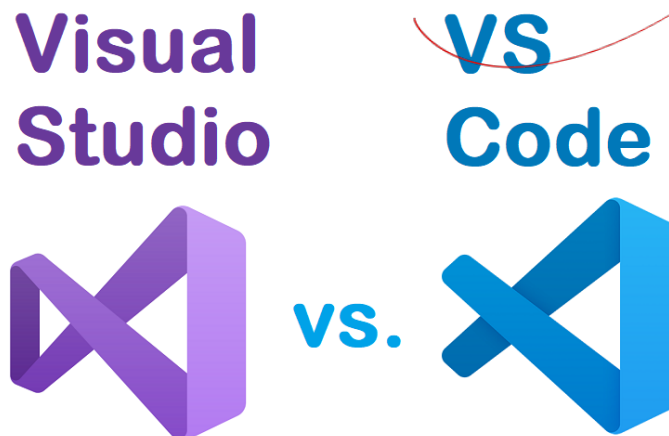
Visual Studio

สิ่งแวดล้อมสำหรับการพัฒนาแบบเบ็ดเสร็จ (อังกฤษ: integrated development environment) หรือ IDE พัฒนาขึ้นโดยไมโครซอฟต์ซึ่งเป็นเครื่องมือที่ช่วยนักพัฒนาซอฟต์แวร์พัฒนาโปรแกรมคอมพิวเตอร์ เว็บไซต์ เว็บแอปพลิเคชัน และ เว็บเซอร์วิส วิศวกรสตูดิโอใช้แพลตฟอร์มต่าง ๆ ของไมโครซอฟต์ เช่น Windows API, Windows Presentation Foundation, Windows Forms, Windows Store และ Microsoft Silverlight วิศวกรสตูดิโอนั้นรองรับภาษาโปรแกรมเป็นจำนวนมาก เช่น C, C++, VB.NET, ภาษาซีชาร์ป, ภาษาเอฟชาร์ป, จาวาสคริปต์, TypeScript, XML, XSLT, HTML, และ CSS

Visual Studio Code (VS Code)

Visual Studio Code (VS Code) เป็นโปรแกรม Code Editor ที่ใช้งานง่ายจากค่าย Microsoft ซึ่งถูกพัฒนาให้เป็น Open Source ทำให้ใช้งานได้ฟรี และยังเป็นเครื่องมือที่เหมาะสมมากสำหรับนักพัฒนาโปรแกรมทุกระดับ ไม่ว่าจะมือใหม่หรือมืออาชีพ

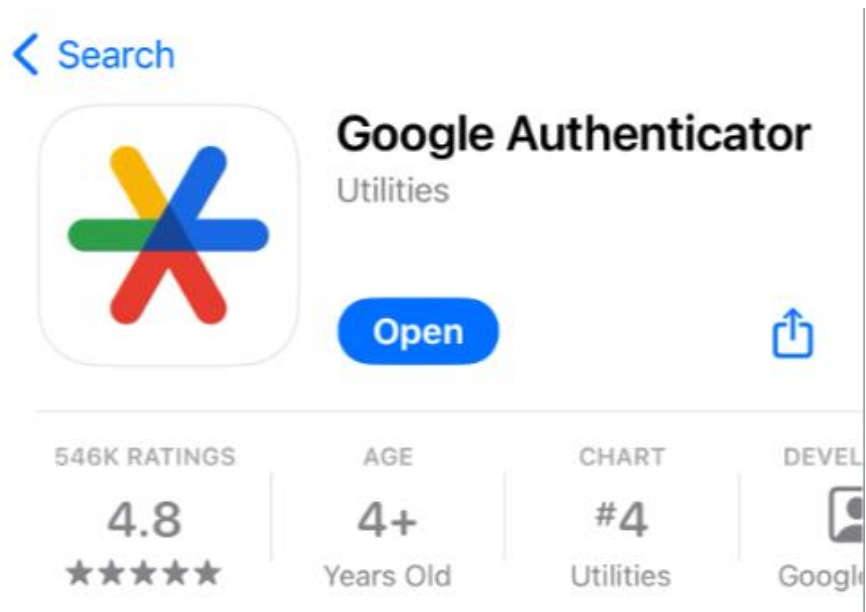
โปรแกรมนี้ออกแบบมาให้รองรับการทำงานข้ามแพลตฟอร์ม ไม่ว่าจะใช้งานบน Windows, macOS หรือ Linux ก็สามารถติดตั้งและใช้งานได้อย่างราบรื่น นอกจากนี้ ยังรองรับภาษาโปรแกรม JavaScript, TypeScript และ Node.js ทำให้ตอบโจทย์การพัฒนาเว็บและแอปพลิเคชันได้เป็นอย่างดี



ภาพประกอบที่ 2.8 Visual Studio & VS Code

Google Authenticator

Google Authenticator คือ แอปพลิเคชันที่ใช้เพื่อยืนยันตัวตน 2 ชั้น ซึ่งหลักการทำงานนั้นคือการสร้างชุดรหัสตัวเลขแบบสุ่ม 6 ตัว โดยชุดรหัสที่สร้างขึ้นในแต่ละครั้ง จะหมดอายุภายใน 30 วินาที และที่สำคัญคือ ชุดรหัสที่ถูกสร้างขึ้นในแต่ละครั้ง สามารถใช้ได้เฉพาะบัญชีที่ผูกไว้เท่านั้น (จะไม่สามารถนำรหัสไปใช้กับบัญชีอื่นได้)



ภาพประกอบที่ 2.9 Google Authenticator

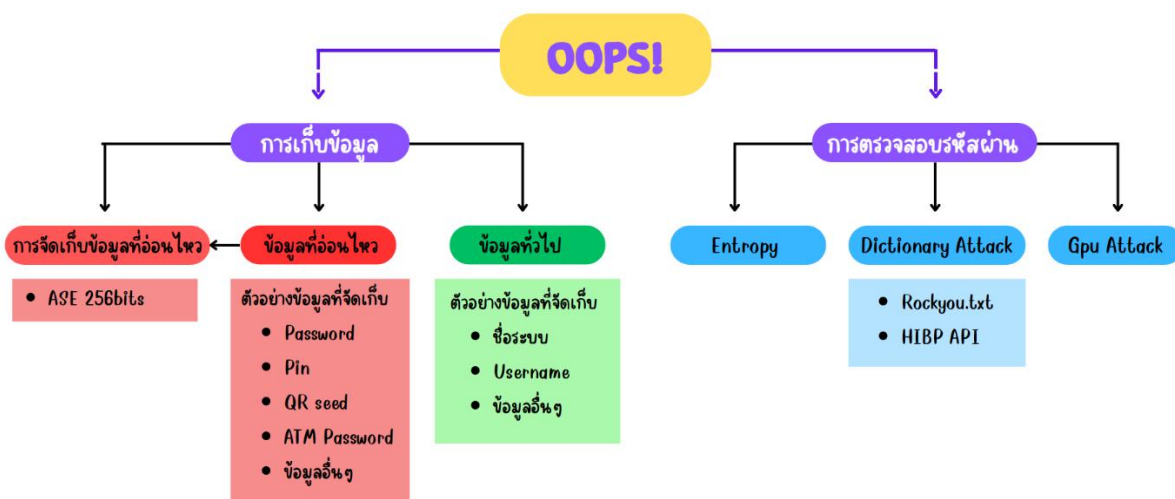
บทที่ 3

I ลบวงจรฟีด.

ขั้นตอนการดำเนินงาน

ในบทนี้จะกล่าวถึงแนวทางการนำข้อมูลจากเว็บไซต์ Have I Been Pwned (HIBP) มาใช้เป็นฐานข้อมูลหลักในการตรวจสอบความปลอดภัยของรหัสผ่านผู้ใช้งาน แทนการใช้ฐานข้อมูล RockYou2024 ที่ได้กล่าวถึงในบทก่อนหน้านี้ การเปลี่ยนมาใช้ HIBP API นั้นมีจุดประสงค์เพื่อลดภาระในการจัดเก็บข้อมูลจำนวนมาก อันเกิดจากการใช้งานฐานข้อมูลรหัสผ่านในรูปแบบดั้งเดิม ซึ่งต้องใช้ทรัพยากรสูง ทั้งในด้านพื้นที่จัดเก็บ (storage) และประสิทธิภาพของระบบประมวลผล (processing power) โดยเฉพาะเมื่อนำไปใช้งานในระบบที่เกี่ยวข้องกับ Big Data หรือระบบเฝ้าระวังความปลอดภัยเชิงลึก โดยมีกรอบการดำเนินการดังนี้

3.1 กรอบการดำเนินการ (ภาพรวมของระบบ)



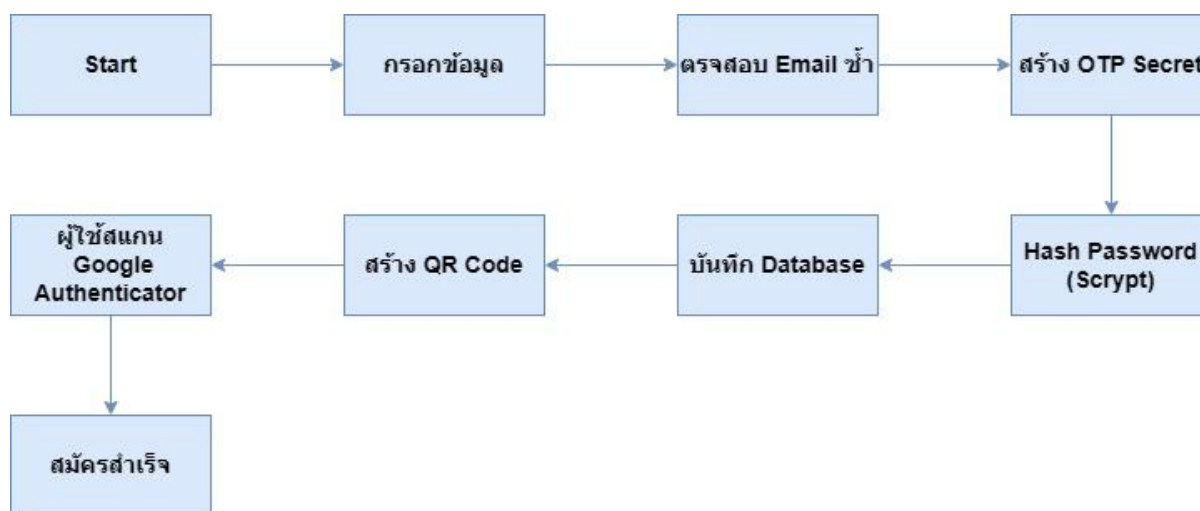
ภาพประกอบที่ 3.1 ภาพรวมของระบบ

ภาพรวมของโครงการปริญญาโทแสดงให้เห็นถึงกรอบการดำเนินงาน ดังนี้

- 1) ศึกษาวิธีการคำนวณค่า Entropy ของรหัสผ่าน เพื่อประเมินระดับความแข็งแกร่งในเชิงปริมาณ และนำค่าที่ได้มาใช้ในการวิเคราะห์ความปลอดภัยเชิงคุณภาพของรหัสผ่าน
- 2) ศึกษาการจัดเก็บข้อมูลสำคัญ เช่น รหัสผ่าน, PIN, QR Seed และข้อมูลลับอื่น ๆ ด้วยกระบวนการเข้ารหัส (Encryption) เพื่อความปลอดภัย
- 3) ศึกษาและทดสอบการเข้ารหัสข้อมูลด้วยอัลกอริธึม AES-256 โดยใช้ Passphrase เป็นกุญแจในการเข้ารหัสและถอดรหัสข้อมูล

- 4) ศึกษาและทดสอบการโจมตีแบบ Dictionary Attack โดยอ้างอิงฐานข้อมูลรหัสผ่านรั่วไหล เช่น Rockyou.txt เพื่อตรวจสอบความเสี่ยงของรหัสผ่านที่ไม่ปลอดภัย
- 5) ศึกษาและทดสอบการใช้งาน HIBP (Have I Been Pwned) API สำหรับตรวจสอบว่ารหัสผ่านเคยปรากฏอยู่ในฐานข้อมูลรั่วไหลสาธารณะหรือไม่
- 6) วิเคราะห์ความเป็นไปได้ของการโจมตีแบบ GPU Attack ซึ่งอาศัยประสิทธิภาพของหน่วยประมวลผลกราฟิกในการถอดรหัสผ่านจำนวนมากภายในระยะเวลาอันสั้น

3.1.1 User Registration Flow

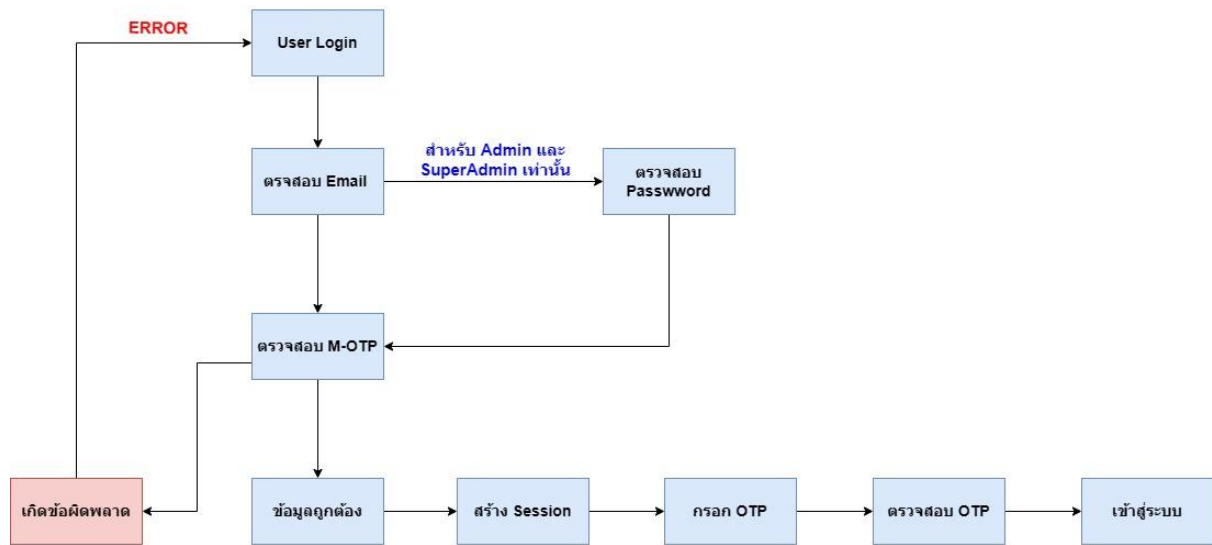


ภาพประกอบที่ 3.2 Registration Flow

ขั้นตอนการทำงาน

- Step 1 : ผู้ใช้กรอกข้อมูลสมัครสมาชิก
- Step 2 : ระบบตรวจสอบ
- Step 3 : ส่ง Password ไปตรวจสอบกับ
- Step 4 : ถ้าผ่าน ระบบเข้ารหัสรหัสผ่านด้วย
- Step 5 : บันทึกข้อมูลลงฐานข้อมูล
- Step 6 : ส่งอีเมลยืนยัน
- Step 7 : สมัครสมาชิกสำเร็จ

3.1.2 Login & Authentication Flow

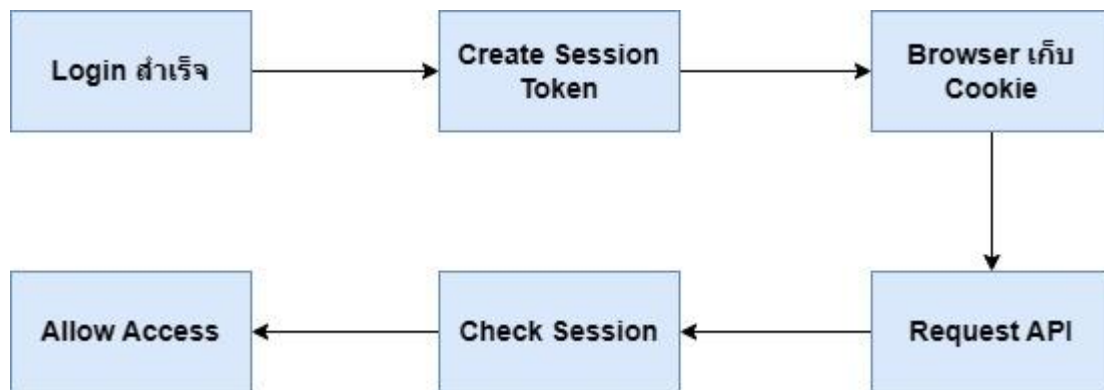


ภาพประกอบที่ 3.3 Login & Authentication Flow

ขั้นตอนการทำงาน

- Step 1 : ผู้ใช้กรอก
 - Email
 - M-OTP
 - Password (เฉพาะ Admin และ SuperAdmin)
- Step 2 : ระบบค้นหาผู้ใช้จากฐานข้อมูล
- Step 3 : นำ Password มา Verify
- Step 4 : หากถูกต้องส่ง OTP ไปยังผู้ใช้
- Step 5 : ตรวจสอบ OTP
- Step 6 : สร้าง Session
- Step 7 : เข้าสู่ Dashboard

3.1.3 Session ManagementFlow



ภาพประกอบที่ 3.4 Session ManagementFlow

ขั้นตอนการทำงาน

- หลังจาก Login สำเร็จระบบจะสร้างSession IDเพื่อระบุตัวตนของผู้ใช้
- ทุกครั้งที่เรียกหน้า
 - Dashboard
 - Vault
 - Profile
- ระบบจะตรวจสอบ Session
- ถ้า Session หมดอายุจะบังคับ Login ใหม่

3.1.4 Secret Management Flow



ภาพประกอบที่ 3.5 Secret Management Flow

ขั้นตอนการเข้ารหัส

- ผู้ใช้กรอกข้อมูลลับ เช่น Facebook Password ,Bank Account, API Key
- ระบบนำข้อมูลไปเข้ารหัส AES-256-GCM
- ได้ผลลัพธ์เป็น Ciphertext , IV และ Auth Tag แล้วบันทึกลงฐานข้อมูล

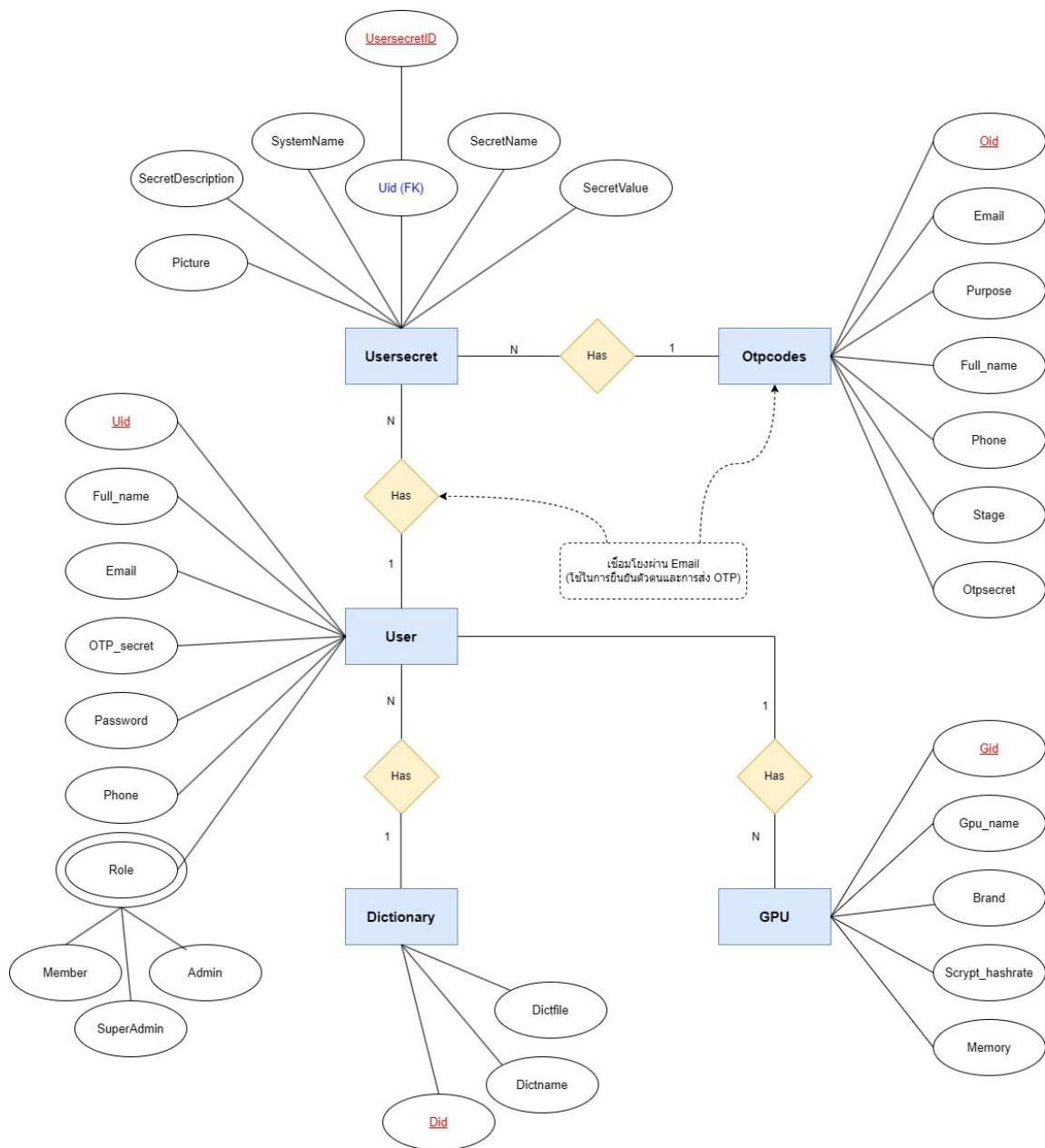
ขั้นตอนการถอดรหัส

- เมื่อผู้ใช้เปิดดูข้อมูลระบบจะดึง Ciphertext IV และ Auth Tag จากฐานข้อมูล
- นำมาถอดรหัส AES-256-GCM Decrypt
- หาก Auth Tag ถูกต้องจะแสดงข้อมูลจริง



3.2 ฐานข้อมูล (ดาต้าเบส)

3.2.1 ER Diagram



ภาพประกอบที่ 3.6 Database

ระบบถูกออกแบบให้สามารถจัดเก็บและจัดการข้อมูลผู้ใช้ (User) ร่วมกับข้อมูลแอปพลิเคชัน (Application) และข้อมูลความลับที่เกี่ยวข้องกับระบบต่าง ๆ (Secret and System) ได้อย่างปลอดภัย โดยมีการแยก Entity ออกเป็นหมวดหลัก ๆ ที่ทำหน้าที่เฉพาะเจาะจง เพื่อให้สามารถปรับขยายหรือบำรุงรักษาระบบได้ง่ายในอนาคต รวมถึงสามารถเชื่อมโยงข้อมูลที่จำเป็นเพื่อสนับสนุนระบบความปลอดภัยและการวิเคราะห์ข้อมูลผ่าน GPU หรือ Dictionary ได้

พอทราบ ครบแล้ว 15:20 * เจ็ดแล้วแต่ยังไม่.

3.2.2 การออกแบบฐานข้อมูล (Data Design)

ตารางที่ 3.1 User table

ID	Column	Type	Description	Example Data	Constraint
1	<u>Uid</u>	INT	รหัสผู้ใช้	1	PK
2	Full_name	STRING	ชื่อผู้ใช้	Monticha	NOT NULL
3	Email	STRING	อีเมล	mtc@gmail.com	UNIQUE, NOT NULL
4	OTP secret	STRING	รหัสที่ได้มาจาก Google Authenticator	123 456	NOT NULL
5	Password	STRING	รหัสผ่านที่ hash ด้วย script algo	OI3XYNYJNNEEBCK	NOT NULL
6	Phone	INT	หมายเลขโทรศัพท์	0911111111	NULL
7	Role	ENUM	Member, Admin, Superadmin)	Member	NOT NULL

เป็นตารางที่ใช้จัดเก็บข้อมูลผู้ใช้งานภายในระบบ ประกอบด้วยข้อมูลสำคัญ เช่น รหัสผู้ใช้ ชื่อผู้ใช้ อีเมล รหัส OTP สำหรับการยืนยันตัวตน รหัสผ่านที่ถูกเข้ารหัสด้วยอัลกอริทึม Script หมายเลขโทรศัพท์ และสิทธิ์การใช้งานของผู้ใช้ เช่น Member, Admin และ Superadmin เพื่อใช้ในการจัดการสิทธิ์และเพิ่มความปลอดภัยในการเข้าสู่ระบบของผู้ใช้งานแต่ละคน

ตารางที่ 3.2 Usersecret table

ID	Column	Type	Description	Example Data	Constraint
1	UserSecretID	INT	รหัสข้อมูลลับ	1	PK
2	UID	c	รหัสผู้ใช้	1	FK
3	SystemName	STRING	ชื่อระบบ	ตู้เซฟที่บ้าน	NOT NULL
4	Secretname	STRING	ชื่อข้อมูล	111111	NOT NULL
5	SecretDescription	STRING	คำอธิบายข้อมูล	รหัสตู้เซฟที่บ้าน	NOT NULL
6	SecretValue	STRING	ผลการเข้ารหัส	X7aP9Lm21Q	NOT NULL
7	Picture	STRING	File รูปภาพ	safe.png	NOT NULL

เป็นตารางที่ใช้จัดเก็บข้อมูลลับของผู้ใช้งานภายในระบบ เช่น ชื่อระบบ ชื่อข้อมูลลับ คำอธิบายข้อมูล และค่าข้อมูลที่ถูกเข้ารหัสเพื่อเพิ่มความปลอดภัย นอกจากนี้ยังมีการจัดเก็บรูปภาพที่เกี่ยวข้องกับข้อมูลลับ เพื่อช่วยให้ผู้ใช้งานสามารถจัดการและค้นหาข้อมูลสำคัญได้สะดวกมากยิ่งขึ้น ภายในระบบ Safebox.

ตารางที่ 3.3 Dictionary

ID	Column	Type	Description	Example Data	Constraint
1	Did	INT	รหัส Dictionary	1	PK
2	Dictname	TEXT	ชื่อ Dictionary ของรหัสผ่าน	Rockyou.txt	NOT NULL
3	Dictfile	STRING	ตำแหน่งไฟล์ Dictionary	C:/dictionary/rockyou.txt	NOT NULL

ใช้สำหรับจัดเก็บข้อมูลเกี่ยวกับไฟล์ Password Dictionary ที่นำมาใช้ในการตรวจสอบความปลอดภัยของรหัสผ่านและการป้องกันการโจมตีแบบ Dictionary Attack โดยภายในตารางจะเก็บข้อมูลรหัส Dictionary ชื่อไฟล์ และตำแหน่งที่จัดเก็บไฟล์ เพื่อให้ระบบสามารถเรียกใช้งานฐานข้อมูลคำศัพท์หรือรหัสผ่านที่นิยมใช้ในการตรวจสอบรหัสผ่านของผู้ใช้งานได้อย่างมีประสิทธิภาพ

ตารางที่ 3.4 Otpcodes

ID	Column	Type	Description	Example Data	Constraint
1	<u>Oid</u>	INT	รหัสข้อมูล OTP	1	PK
2	UID	STRING	รหัสผู้ใช้	1	FK
3	Purpose	STRING	วัตถุประสงค์ของการใช้งาน OTP	Reset_motp	NOT NULL
4	Stage	STRING	ขั้นตอนการยืนยันตัวตน	email	NOT NULL
5	Otpsecret	STRING	รหัสลับสำหรับสร้าง OTP	FFTXAHBACU	NOT NULL

ใช้สำหรับจัดเก็บข้อมูลเกี่ยวกับการยืนยันตัวตนด้วยรหัส OTP ของผู้ใช้งานโดยวัตถุประสงค์ในการใช้งาน OTP และรหัสลับที่ใช้ในการสร้าง OTP เพื่อเพิ่มความปลอดภัยในการเข้าสู่ระบบหรือรีเซตรหัสผ่านของระบบการออกแบบระบบ

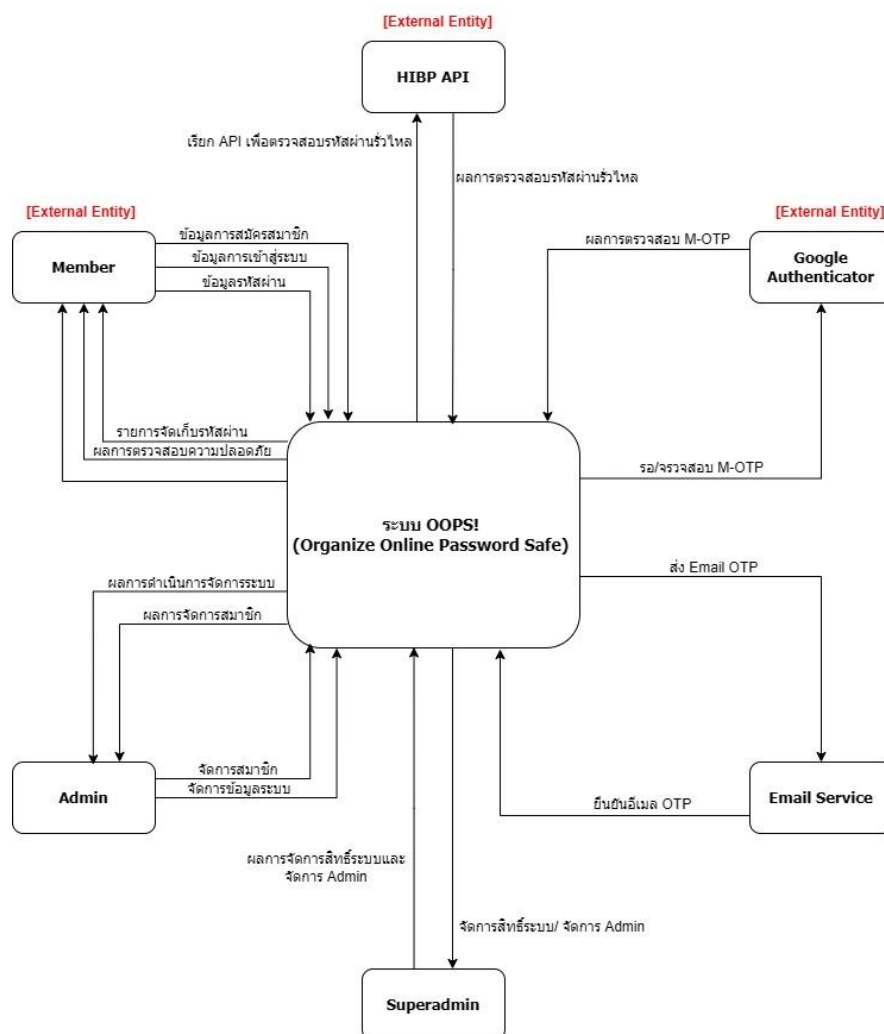
ตารางที่ 3.5 GPU

ID	Column	Type	Description	Example Data	Constraint
1	<u>Gid</u>	INT	รหัส GPU	1	PK
2	Gpu_name	STRING	ชื่อรุ่น GPU	RTX5090	NOT NULL
3	Brand	STRING	ยี่ห้อ	NVIDIA	NOT NULL
4	Script_hashrate	INT	ความเร็วในการประมวลผล	5200000	NOT NULL
5	Memory	INT	ขนาดหน่วยความจำ	24	NOT NULL
6	Default	Boolean	เป็น default GPU ที่ใช้คำนวณหรือไม่?	Yes	NOT NULL

ใช้สำหรับจัดเก็บข้อมูลของ GPU ในระบบ เพื่อใช้ในการอ้างอิงและวิเคราะห์ประสิทธิภาพของการ์ดจอแต่ละรุ่น โดยประกอบด้วยรหัส GPU ชื่อรุ่นและยี่ห้อ รวมถึงค่าประสิทธิภาพการประมวลผล และขนาดหน่วยความจำ (Memory) ซึ่งข้อมูลทั้งหมดนี้ช่วยให้ระบบสามารถนำไปใช้ในการเปรียบเทียบและเลือกใช้งาน GPU ได้อย่างเหมาะสม โดยข้อมูลสเปกของ GPU อ้างอิงจากเอกสารเปรียบเทียบสเปกการ์ดจอของ NVIDIA และข้อมูล CUDA GPU Specifications ของ NVIDIA Developer Documentation [7]

3.2.3 แผนภาพการไหลของข้อมูล (Context Diagram)

แผนภาพบริบท (Context Diagram) แสดงภาพรวมการทำงานของระบบ OOPS! โดยแสดงความสัมพันธ์ระหว่างระบบกับผู้ใช้ภายนอก ได้แก่ Member, Admin, Superadmin, Google Authenticator, Email Service และ HIBP API เพื่อแสดงการรับส่งข้อมูลหลัก เช่น การสมัครสมาชิก การเข้าสู่ระบบ การยืนยันตัวตน การจัดเก็บข้อมูลลับ และการตรวจสอบความปลอดภัยของรหัสผ่าน

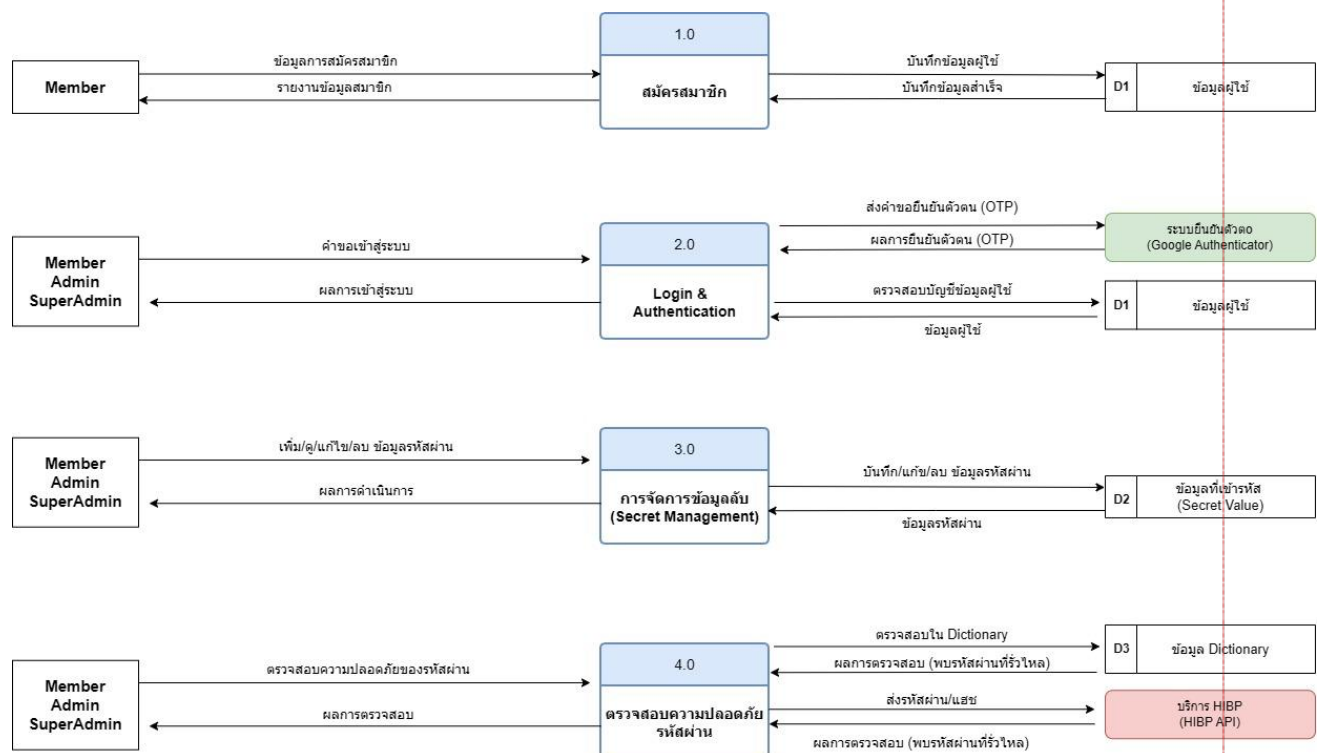


ภาพประกอบที่ 3.7 Context Diagram

3.2.4 Data Flow Diagram Level 1

รูปภาพฉบับจริง

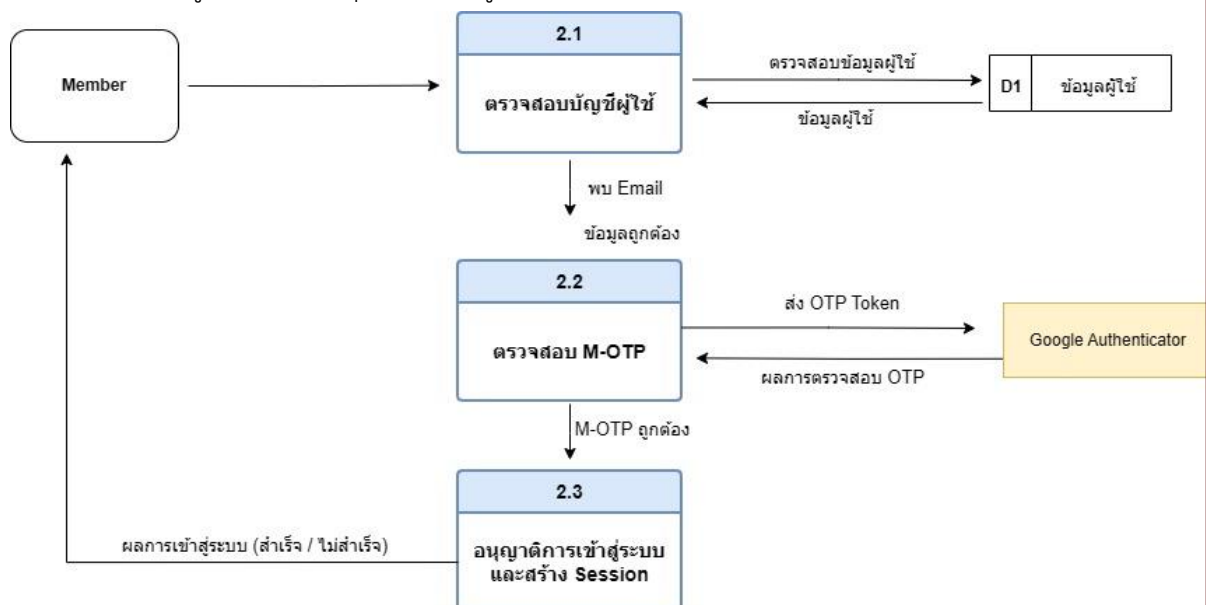
แผนภาพกระแสข้อมูลระดับที่ 1 (DFD Level 1) แสดงขั้นตอนการทำงานของระบบ OOPS! โดยแบ่งกระบวนการทำงานออกเป็นส่วนตัวต่าง ๆ ได้แก่ การสมัครสมาชิก การเข้าสู่ระบบ การจัดการข้อมูลลับ การตรวจสอบความปลอดภัยรหัสผ่าน พร้อมแสดงการเชื่อมโยงข้อมูลกับฐานข้อมูล (Data Store) และหน่วยงานภายนอกที่เกี่ยวข้อง



ภาพประกอบที่ 3.8 แผนภาพกระแสข้อมูลระดับที่ 1

3.2.5 Data Flow Diagram Level 2 – Login & Authentication

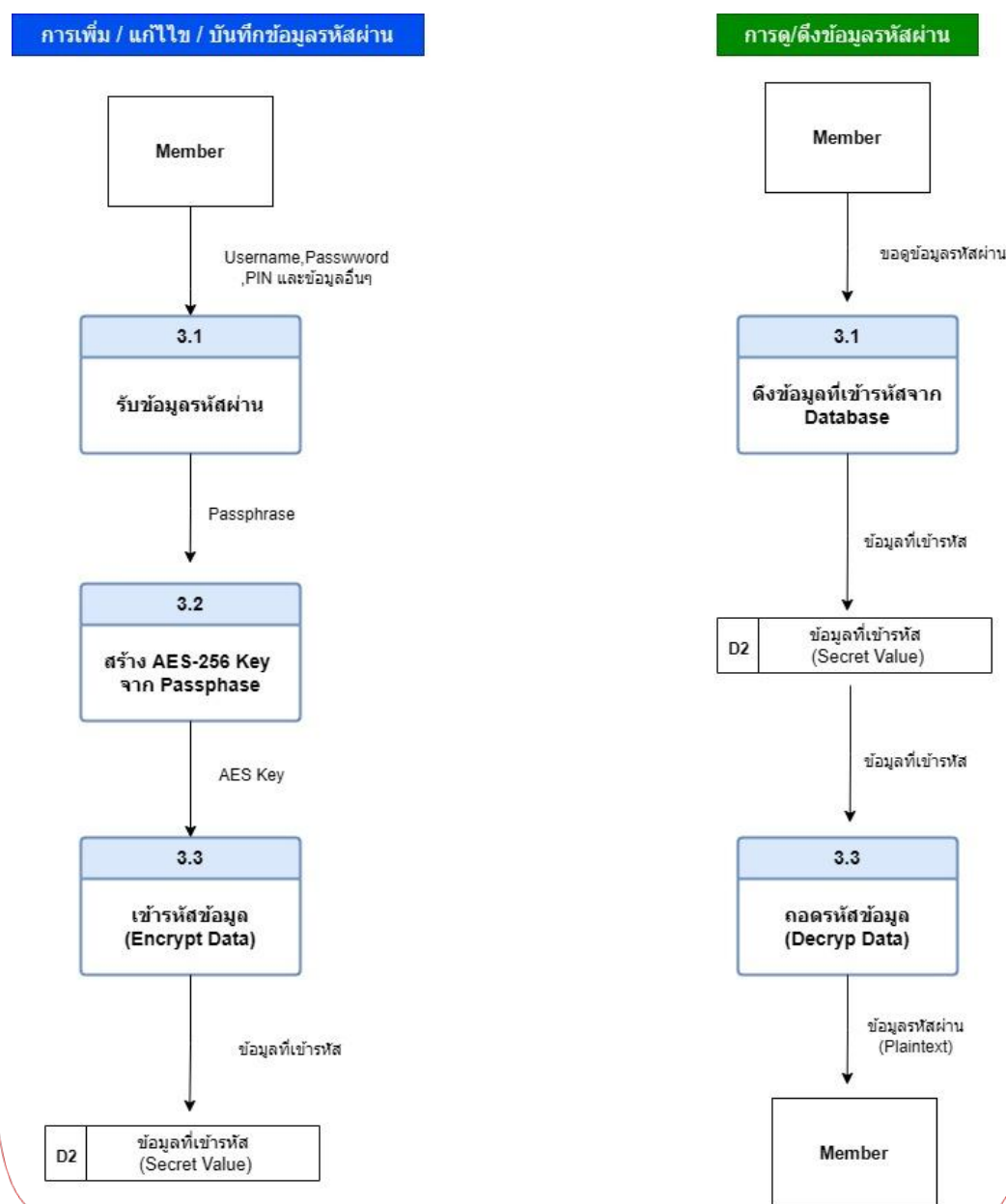
แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการเข้าสู่ระบบและยืนยันตัวตน แสดงรายละเอียดการตรวจสอบ Email การส่ง OTP การตรวจสอบ OTP และการตรวจสอบ M-OTP ผ่าน Google Authenticator เพื่อยืนยันตัวตนผู้ใช้งานก่อนอนุญาตให้เข้าสู่ระบบ ซึ่งช่วยเพิ่มความปลอดภัยในการใช้งานระบบ OOPS!



ภาพประกอบที่ 3.9 แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการเข้าสู่ระบบและยืนยันตัวตน

3.2.6 Data Flow Diagram Level 2 – Secret Management/Encryption

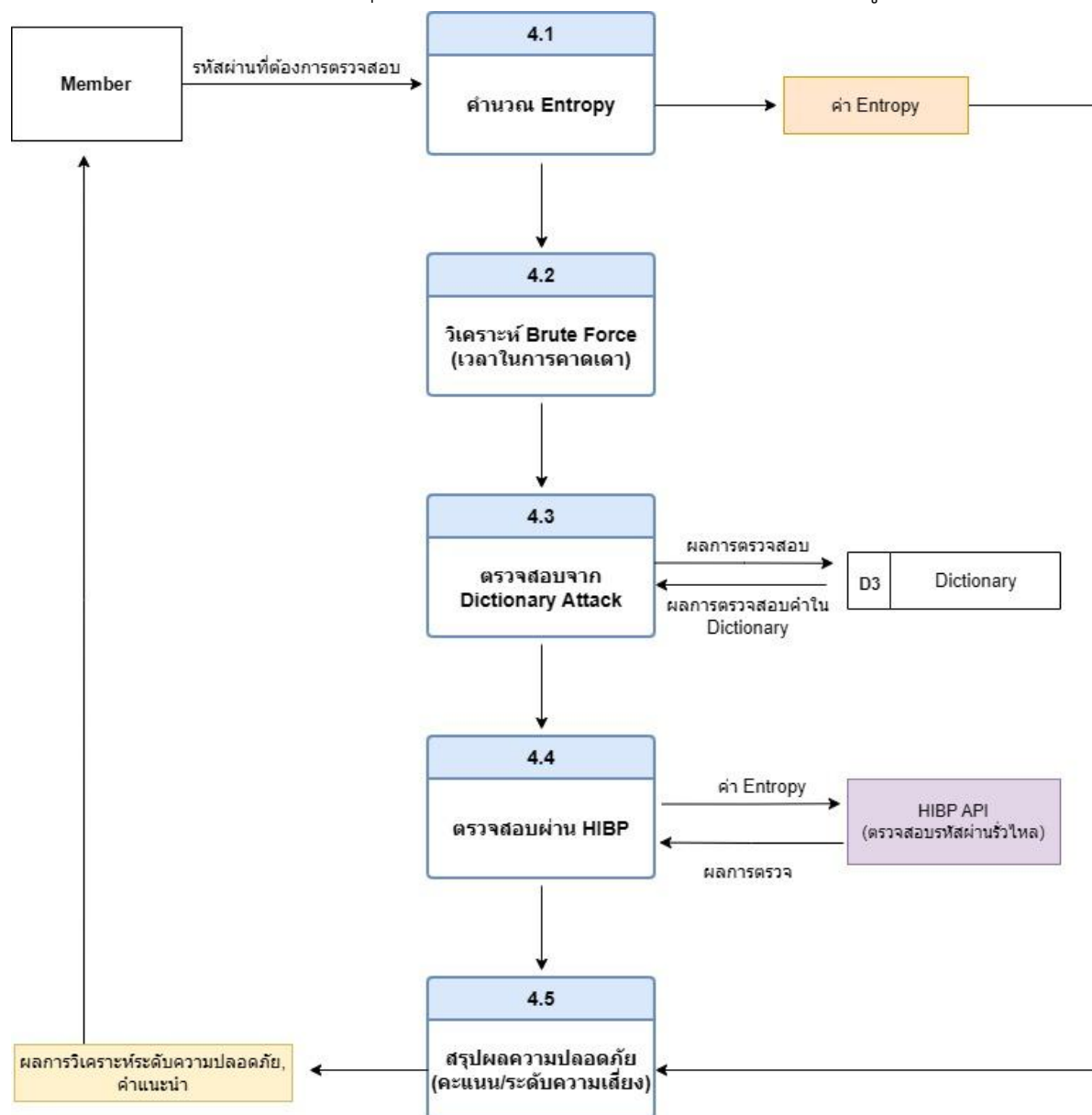
แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการจัดการข้อมูลลับ ผู้ใช้สามารถเพิ่ม แก้ไข ลบ หรือดูข้อมูลรหัสผ่านที่ต้องการจัดเก็บ ระบบจะเข้ารหัสข้อมูลก่อนบันทึกลงในฐานข้อมูลรหัสผ่าน (D2 Secret Database) เพื่อป้องกันการเข้าถึงโดยไม่ได้รับอนุญาต และเมื่อผู้ใช้งานขอข้อมูล ระบบจะถอดรหัสก่อนส่งกลับให้ผู้ใช้



ภาพประกอบที่ 3.10 แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการจัดการข้อมูลลับ

3.2.7 Data Flow Diagram Level 2 – Password Security Check

แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการตรวจสอบความปลอดภัยรหัสผ่าน ผู้ใช้ส่งรหัสผ่านที่ต้องการตรวจสอบเข้ามา ระบบจะวิเคราะห์ความแข็งแรงของรหัสผ่านด้วยการคำนวณ Entropy ประเมินการโจมตีแบบ Brute Force ตรวจสอบคำศัพท์จากฐานข้อมูล Dictionary และตรวจสอบว่ารหัสผ่านเคยรั่วไหลผ่าน HIBP API หรือไม่ จากนั้นจึงสรุปผลเป็นคะแนนความปลอดภัยและคำแนะนำแก่ผู้ใช้



ภาพประกอบที่ 3.11 แผนภาพกระแสข้อมูลระดับที่ 2 ของกระบวนการตรวจสอบความปลอดภัยรหัสผ่าน

3.3 Data Store

ตารางที่ 3.6 Data Store Description

ID	Data Store	Description	Data Structure
D1	เพิ่มข้อมูลสมาชิก	เก็บรายละเอียดข้อมูลสมาชิก	รหัสสมาชิก+ชื่อผู้ใช้+อีเมล+รหัสที่ได้มาจาก Google Authenticator+รหัสผ่าน+หมายเลขโทรศัพท์+บทบาท
D2	เพิ่มข้อมูลที่เข้ารหัส	เก็บรายละเอียดข้อมูลที่เข้ารหัส	รหัสข้อมูลลับ+รหัสผู้ใช้+ชื่อข้อมูลลับ+ค่าของข้อมูลลับ+รหัสระบบ
D3	เพิ่ม Dictionary	เก็บรายละเอียดข้อมูลรหัสผ่านที่เคยรั่วไหล	รหัสผ่านที่เคยรั่วไหล

3.4 External Entity Description

ตารางที่ 3.7 External Entity Description

Name	Description	Input Data Flow	Output Data Flow
User	ผู้ใช้งานทั่วไปที่สามารถเข้าถึงหน้าเว็บไซต์และใช้งานฟังก์ชันพื้นฐานของระบบได้ เช่น สมัครสมาชิก ตรวจสอบข้อมูลเบื้องต้น และเข้าสู่ระบบ	- ข้อมูลการสมัครสมาชิก - ข้อมูลรหัสผ่าน - ข้อมูลการร้องขอเข้าสู่ระบบ - ข้อมูลการตรวจสอบความปลอดภัยเบื้องต้น	- ข้อมูลสมาชิกที่ถูกบันทึกลงฐานข้อมูล - ข้อมูลรหัสผ่านที่ถูกเข้ารหัส (Encrypted Password) - ผลการสมัครสมาชิก - ผลการตรวจสอบข้อมูลการเข้าสู่ระบบ - การแจ้งเตือนสถานะการสมัครหรือเข้าสู่ระบบ
Member	ผู้ใช้งานที่สมัครสมาชิกและยืนยันตัวตนเรียบร้อยแล้ว สามารถใช้งานฟังก์ชันภายในระบบด้วยการยืนยัน	- ข้อมูลบัญชีผู้ใช้งาน - ข้อมูลการเข้าสู่ระบบด้วย Username/Password - รหัส M-OTP สำหรับยืนยันตัวตน - ข้อมูลคำร้องขอใช้งาน	- ผลการยืนยันตัวตนด้วย M-OTP - Token หรือ Session การใช้งานระบบ - ข้อมูลส่วนตัวของสมาชิก - ประวัติการเข้าใช้งานระบบ

Name	Description	Input Data Flow	Output Data Flow
	ตัวตนแบบ M-OTP และเข้าถึงข้อมูลส่วนตัวได้	ภายในระบบ - ข้อมูลการแก้ไขโปรไฟล์ ผู้ใช้งาน	- การแจ้งเตือนสถานะการเข้าสู่ระบบและความปลอดภัย
Admin	ผู้ดูแลระบบที่มีหน้าที่จัดการข้อมูลภายในแอปพลิเคชัน ตรวจสอบการทำงานของระบบ และดูรายงานการใช้งานของสมาชิก	- ข้อมูลการเข้าสู่ระบบของผู้ดูแลระบบ - ข้อมูลการจัดการสมาชิก (เพิ่ม แก้ไข ลบ) - ข้อมูลการตรวจสอบกิจกรรมภายในระบบ - ข้อมูลการจัดการรายงานและเนื้อหาภายในระบบ - คำสั่งตรวจสอบ Log การทำงาน	- รายงานสถิติการใช้งานระบบ - ผลการจัดการข้อมูลสมาชิก - ข้อมูล Log การเข้าใช้งานและกิจกรรมต่าง ๆ - ผลการอัปเดตหรือแก้ไขข้อมูลภายในระบบ - การแจ้งเตือนเหตุการณ์ผิดปกติภายในระบบ
Superadmin	ผู้ดูแลระบบสูงสุดที่มีสิทธิ์ควบคุมและจัดการทุกส่วนของระบบ รวมถึงกำหนดสิทธิ์การเข้าถึงของ User, Member และ Admin ตลอดจนตรวจสอบความปลอดภัยของระบบทั้งหมด	- ข้อมูลการเข้าสู่ระบบของ Superadmin - ข้อมูลการกำหนดสิทธิ์ผู้ใช้งาน (Role & Permission) - ข้อมูลการจัดการระบบทั้งหมด - ข้อมูลการกำหนดค่าความปลอดภัยของระบบ - ข้อมูลการตรวจสอบฐานข้อมูลและ Server - คำสั่งจัดการสิทธิ์ของ Admin และ Member	- ผลการกำหนดสิทธิ์การเข้าถึงระบบ - รายงานภาพรวมและสถิติทั้งหมดของระบบ - ข้อมูลการตรวจสอบความปลอดภัยและ Audit Log - ผลการจัดการบัญชีผู้ใช้งานและผู้ดูแลระบบ - การแจ้งเตือนเหตุการณ์ด้านความปลอดภัย - ข้อมูลสำรองและการกู้คืนระบบ (Backup & Recovery Status)

3.5 Data Flow Description and Data Structure

เป็นขั้นตอนการทำงานของระบบที่แสดงให้เห็นกระบวนการรับ-ส่งข้อมูลภายในระบบการจัดการผู้ใช้งานและการยืนยันตัวตน ซึ่งช่วยให้สามารถติดตาม ตรวจสอบ และวิเคราะห์การทำงานของระบบได้อย่างมี

ประสิทธิภาพ โดยกระบวนการนี้จะแสดงลำดับการไหลของข้อมูลตั้งแต่ผู้ใช้งานเข้าสู่ระบบ การสมัครสมาชิก การยืนยันตัวตนด้วย M-OTP การจัดการข้อมูลสมาชิก ตลอดจนการตรวจสอบและกำหนดสิทธิ์การเข้าถึงของ Admin และ Superadmin รวมถึงข้อมูลที่ระบบรับเข้า (Input) และข้อมูลที่ระบบส่งออก (Output) ในแต่ละขั้นตอน เพื่อให้ระบบสามารถทำงานได้อย่างถูกต้อง ปลอดภัย และรองรับการจัดการข้อมูลภายในระบบได้อย่างมีประสิทธิภาพ

ตารางที่ 3.8 Data Flow Description and Data Structure

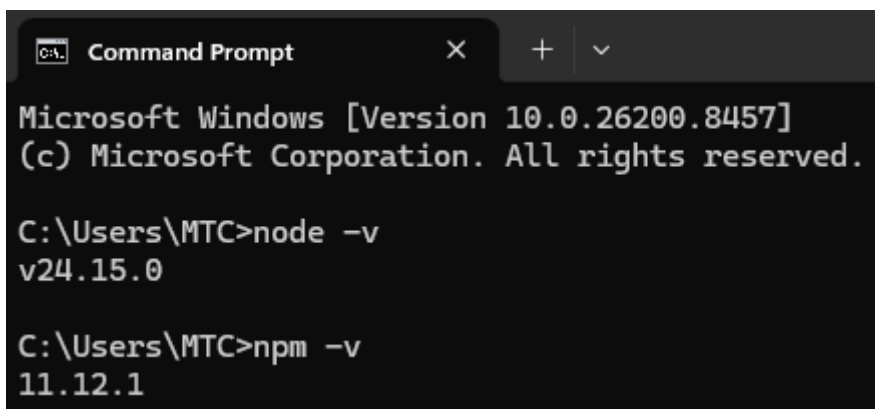
Name	Description	Source	Destination	Data Structure
ข้อมูลการสมัครสมาชิก	ข้อมูลที่ผู้ใช้งานกรอกเพื่อสร้างบัญชีใหม่ในระบบ	User	ระบบฐานข้อมูลสมาชิก	รหัสสมาชิก+ชื่อผู้ใช้+อีเมล+รหัสผ่าน+หมายเลขโทรศัพท์+บทบาท+วันที่สมัครสมาชิก
ข้อมูลการเข้าสู่ระบบ	ข้อมูลที่ใช้สำหรับตรวจสอบตัวตนก่อนเข้าใช้งานระบบ	User, Member	ระบบยืนยันตัวตน	ชื่อผู้ใช้+รหัสผ่าน+วันเวลาเข้าสู่ระบบ+หมายเลข IP
ข้อมูล M-OTP	ข้อมูลรหัส OTP สำหรับยืนยันตัวตนผู้ใช้งาน	ระบบยืนยันตัวตน	Member, Admin, Superadmin	รหัส OTP+รหัสอ้างอิง+เวลาหมดอายุ+สถานะการยืนยัน
ข้อมูลสมาชิก	ข้อมูลรายละเอียดของสมาชิกภายในระบบ	ฐานข้อมูล	Member, Admin	รหัสสมาชิก+ชื่อผู้ใช้+อีเมล+สิทธิ์ผู้ใช้งาน+สถานะบัญชี+รูปโปรไฟล์
ข้อมูลสิทธิ์การใช้งาน	ข้อมูลกำหนดสิทธิ์การเข้าถึงของผู้ใช้งานแต่ละประเภท	Superadmin	ระบบจัดการสิทธิ์	รหัสสมาชิก+ชื่อผู้ใช้+บทบาท+วันที่แก้ไข
ข้อมูลการจัดการผู้ใช้งาน	ข้อมูลที่ใช้เพิ่มแก้ไข หรือลบสมาชิกในระบบ	Admin, Superadmin	ฐานข้อมูล	รหัสผู้ใช้+สถานะผู้ใช้+ประวัติการแก้ไข+สถานะการลบข้อมูล

Name	Description	Source	Destination	Data Structure
ข้อมูลรายงานระบบ	ข้อมูลสรุปผลการทำงานและสถิติการใช้งานระบบ	ฐานข้อมูล	Admin, Superadmin	สถิติการใช้งาน+ประวัติการเข้าสู่ระบบ+บันทึกกิจกรรม
ข้อมูลการสำรองระบบ	ข้อมูลสำหรับการสำรองและกู้คืนข้อมูลของระบบ	Superadmin	Backup Server	ชื่อไฟล์+วันที่สำรองข้อมูล+สถานะการใช้งาน
ข้อมูลความปลอดภัยระบบ	ข้อมูลสำหรับตรวจสอบและป้องกันความปลอดภัยของระบบ	ระบบรักษาความปลอดภัย	Superadmin	สถานการณ์ใช้งาน+บันทึกกิจกรรม+เวลาในการตรวจสอบ
ข้อมูล Session การใช้งาน	ข้อมูลเซสชันการเข้าใช้งานของผู้ใช้ภายในระบบ	User, ระบบ	Session manager, ฐานข้อมูล	Session ID+รหัสผู้ใช้, เวลาที่เข้าสู่ระบบ, Expired Time, Session Status

3.6 การพัฒนาระบบ

3.6.1 การติดตั้ง Node.js

- เปิดเว็บไซต์สำหรับดาวน์โหลด Node.js ผ่านลิงก์ [Node.js Download] (https://nodejs.org/en/download?utm_source=chatgpt.com) เพื่อดาวน์โหลดตัวติดตั้ง
- เลือกเวอร์ชันของ Node.js ที่ต้องการติดตั้ง โดยแนะนำให้เลือกเวอร์ชัน LTS (Long Term Support) เนื่องจากมีความเสถียรและเหมาะสำหรับการพัฒนาระบบ
- เลือกระบบปฏิบัติการที่ใช้งาน เช่น Windows, macOS หรือ Linux จากนั้นกดดาวน์โหลดไฟล์ติดตั้ง
- เมื่อดาวน์โหลดเสร็จ ให้เปิดไฟล์ติดตั้ง (.msi สำหรับ Windows) จากนั้นทำการติดตั้งโปรแกรมตามขั้นตอนให้เสร็จสิ้น
- เมื่อติดตั้งสำเร็จแล้วให้ทำการตรวจสอบการติดตั้งโดยเปิดโปรแกรม Command Prompt หรือ Terminal แล้วพิมพ์คำสั่ง `node -v` เพื่อทำการตรวจสอบหมายเลขเวอร์ชันและคำสั่ง `npm -v` เพื่อตรวจสอบการติดตั้ง npm (Node Package Manager)



```

Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\MTC>node -v
v24.15.0

C:\Users\MTC>npm -v
11.12.1
  
```

ภาพประกอบที่ 3.12 การตรวจสอบหมายเลขเวอร์ชันและการติดตั้ง npm

- หากระบบแสดงหมายเลขเวอร์ชันของ Node.js และ npm แสดงว่าการติดตั้งสำเร็จ สามารถใช้งานได้

3.6.2 การใช้งาน Node.js

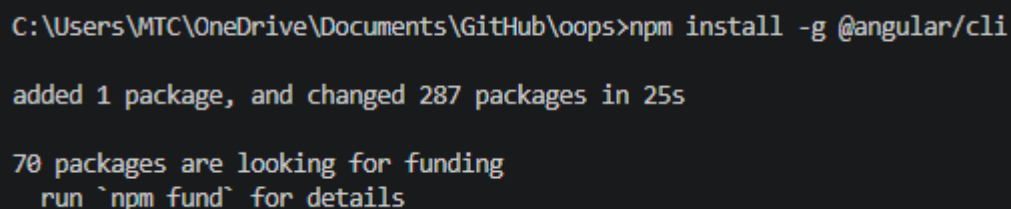
Node.js เป็นโปรแกรมสำหรับรันภาษา JavaScript บนฝั่งเซิร์ฟเวอร์โดยใช้รูปแบบการทำงานแบบ Asynchronous และ Event-driven ซึ่งช่วยให้ระบบสามารถประมวลผลและรองรับผู้ใช้งานจำนวนมากได้อย่างมีประสิทธิภาพ

ในการพัฒนาระบบ OOPS! ผู้พัฒนาได้นำ Node.js มาใช้ในการพัฒนาระบบฝั่ง Back-end สำหรับจัดการการเชื่อมต่อฐานข้อมูล การตรวจสอบสิทธิ์ผู้ใช้งาน การจัดการ OTP การเข้ารหัสข้อมูล และการสร้าง REST API เพื่อเชื่อมต่อกับ Front-end ของระบบ

- Modules Node.js : ใช้ระบบโมดูล (Modules) เพื่อแบ่งแยกการทำงานออกเป็นส่วนต่างๆ ช่วยให้สามารถพัฒนาและดูแลระบบได้ง่ายขึ้น เช่น โมดูลสำหรับสร้าง HTTP Server การจัดการไฟล์ (File System) และการจัดการ Path ของไฟล์
- npm (Node Package Manager) : เป็นเครื่องมือสำหรับติดตั้งและจัดการแพ็คเกจต่าง ๆ ที่ใช้ในโปรเจกต์ เช่น Express.js, Mongoose, JWT
- Asynchronous Programming : ช่วยให้ระบบสามารถทำงานหลายอย่างพร้อมกันได้ เช่น การอ่านไฟล์ การเชื่อมต่อฐานข้อมูล MongoDB หรือการส่ง OTP ผ่านอีเมล โดยไม่ทำให้ระบบหยุดรอผลลัพธ์
- Event-driven Architecture : ใช้ในการควบคุมการทำงานของระบบ โดยจะทำงานตามเหตุการณ์ที่เกิดขึ้น เช่น การส่ง Request จากผู้ใช้งาน หรือการตอบกลับข้อมูลจากฐานข้อมูล

3.6.3 การติดตั้ง Angular

- เปิด Command Prompt หรือ Terminal แล้วพิมพ์คำสั่ง `npm install -g @angular/cli` เพื่อทำการติดตั้ง

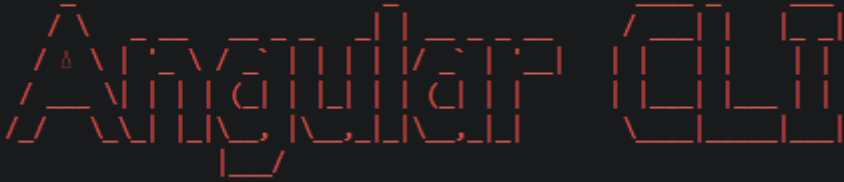


```
C:\Users\VMTC\OneDrive\Documents\GitHub\oops>npm install -g @angular/cli
added 1 package, and changed 287 packages in 25s
70 packages are looking for funding
run `npm fund` for details
```

ภาพประกอบที่ 3.13 การติดตั้ง Angular

- เมื่อการติดตั้งเสร็จสิ้น ให้ตรวจสอบเวอร์ชัน Angular CLI ด้วยคำสั่ง `ng version`

```
C:\Users\WTC\OneDrive\Documents\GitHub\oops>ng version
```



```

Angular CLI      : 21.2.12
Node.js         : 24.15.0
Package Manager : npm 11.12.1
Operating System : win32 x64

```

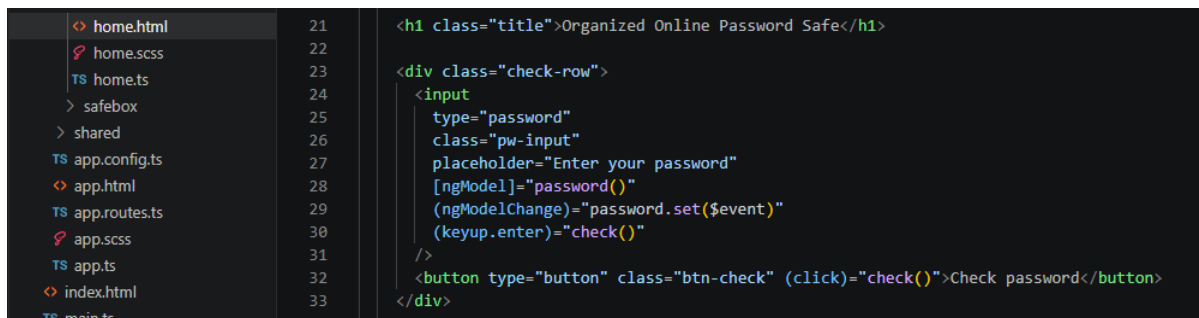
Package	Installed Version	Requested Version
typescript	5.9.3	~5.9.2

ภาพประกอบที่ 3.14 การตรวจสอบเวอร์ชัน Angular CLI ด้วยคำสั่ง `ng version`

- หากระบบแสดงข้อมูลเวอร์ชันของ Angular CLI แสดงว่าติดตั้งสำเร็จเรียบร้อยแล้ว
- ทำการสร้างโฟลเดอร์เพื่อใช้เก็บข้อมูลโปรเจกต์ด้วยคำสั่ง
 - `ng new` ชื่อโปรเจกต์ : สร้างโปรเจกต์
 - `cd` ชื่อโปรเจกต์ : เข้าสูโฟลเดอร์
 - `code . + enter` : เพื่อเปิด Visual Studio Code
- ใช้คำสั่ง `ng serve` เพื่อเปิดใช้งานเบราว์เซอร์

3.6.4 การใช้งาน Angular

ผู้พัฒนาได้นำ Angular มาใช้ในการออกแบบและพัฒนาหน้าจอของระบบ เช่น หน้าเข้าสู่ระบบ สมัครสมาชิก จัดการข้อมูลลับ และหน้าจัดการ OTP โดย Angular ทำหน้าที่เชื่อมต่อกับ REST API ของระบบ Back-end ที่พัฒนาด้วย Node.js และ Express.js เพื่อรับส่งข้อมูลระหว่างผู้ใช้งานและฐานข้อมูล



ภาพประกอบที่ 3.15 การใช้ Angular ในการออกแบบหน้าแรกของเว็บไซต์

บรรทัดที่ 21-33

โค้ดส่วนนี้เป็นหน้าแรกของเว็บไซต์เป็นตัวอย่างของการนำ Angular มาใช้งานในการออกแบบ ประกอบด้วยหัวข้อของระบบ ช่องกรอกรหัสผ่าน และปุ่มสำหรับตรวจสอบรหัสผ่าน ผู้ใช้งานสามารถกรอกรหัสผ่านลงในช่อง Input ซึ่งข้อมูลจะถูกเชื่อมกับตัวแปรภายในระบบผ่าน ngModel และเมื่อกดปุ่ม Enter หรือคลิกปุ่ม “Check password” ระบบจะเรียกใช้ฟังก์ชัน check() เพื่อทำการตรวจสอบรหัสผ่านตามเงื่อนไขที่กำหนด

3.6.5 การสมัครสมาชิก

การสมัครสมาชิกผู้ใช้งานต้องกรอก ชื่อเต็ม อีเมล และหมายเลขโทรศัพท์เพื่อรับ Email-OTP เพื่อใช้ในการสมาชิก

```

23 <form class="form" (ngSubmit)="submit(f)" #f="ngForm">
24   <h1 class="title">Create Account</h1>
25
26   <label class="field">
27     <span class="label">Full Name :</span>
28     <input type="text" name="fullName" ngModel required />
29   </label>
30   <label class="field">
31     <span class="label">Email :</span>
32     <input type="email" name="email" ngModel required />
33   </label>
34   <label class="field">
35     <span class="label">Phone :</span>
36     <input type="tel" name="phone" ngModel />
37   </label>
38
39   <label class="captcha">
40     <input type="checkbox" name="robot" ngModel />
41     <span>I'm not robot</span>
42   </label>
43
44   @if (error()) {
45     <p class="form-error">{{ error() }}</p>
46   }
47
48   <button type="submit" class="btn-submit" [disabled]="loading()">
49     {{ loading() ? 'กำลังส่ง...' : 'SIGN UP' }}
50   </button>
51 </form>

```

ภาพประกอบที่ 3.16 ฟอรั่มสมัครสมาชิก

บรรทัดที่ 23-51

โค้ดส่วนนี้เป็นฟอรั่มสมัครสมาชิกสำหรับรับข้อมูลผู้ใช้งานประกอบด้วย ชื่อ ผู้ใช้ อีเมล และเบอร์โทรศัพท์ โดยใช้ `ngModel` ในการเชื่อมข้อมูลกับระบบ มีการตรวจสอบข้อมูลที่จำเป็นด้วย `required` และเมื่อผู้ใช้งานกดปุ่ม `SIGN UP` ระบบจะเรียกใช้ฟังก์ชัน `submit(f)` เพื่อประมวลผลข้อมูลการสมัครสมาชิกต่อไป

```

1  import { Component, inject, signal } from '@angular/core';
2  import { FormsModule, NgForm } from '@angular/forms';
3  import { Router } from '@angular/router';
4  import { AuthService } from '../../data/auth.service';
5
6  @Component({
7    selector: 'app-register',
8    imports: [FormsModule],
9    templateUrl: './register.html',
10   styleUrls: ['./register.scss'],
11 })

```

ภาพประกอบที่ 3.17 การ Import & การกำหนดคุณสมบัติผ่าน Component

บรรทัดที่ 1-4

Import โมดูลและคลาสที่จำเป็นสำหรับการพัฒนาหน้าสมัครสมาชิก

- Component ใช้สำหรับสร้าง Component ของระบบ
- Inject เรียกใช้งาน Service ภายใน Component
- Signal ใช้จัดการข้อมูลแบบ Reactive
- FormsModule & NgForm ใช้สำหรับจัดการฟอร์มและข้อมูลที่ผู้ใช้งานกรอกภายในหน้าเว็บ
- Router ใช้สำหรับควบคุมการเปลี่ยนหน้าในระบบ
- AuthService ใช้จัดการข้อมูลเกี่ยวกับการสมัครสมาชิกและการเข้าสู่ระบบของผู้ใช้งาน

บรรทัดที่ 6-11

ใช้กำหนด Component ของ Angular

- selector ใช้ตั้งชื่อ Component สำหรับเรียกใช้งาน
- imports ใช้ระบุ Module ที่ต้องใช้
- templateUrl กำหนดไฟล์ HTML สำหรับแสดงผล
- styleUrls กำหนดไฟล์ SCSS สำหรับตกแต่งหน้าเว็บ


```

12 export class Register {
13     private auth = inject(AuthService);
14     private router = inject(Router);
15     loading = signal(false);
16     error = signal<string | null>(null);
17
18     async submit(form: NgForm) {
19         if (this.loading()) return;
20         const { fullName, email, phone } = form.value;
21         if (!fullName || !email) {
22             this.error.set('กรุณกรอกชื่อและอีเมล');
23             return;
24         }
25         this.loading.set(true);
26         this.error.set(null);
27         try {
28             const res = await this.auth.register(fullName, email, phone ?? '');
29             sessionStorage.setItem('oops_flow_email', email);
30             if (res.devOtp) sessionStorage.setItem('oops_dev_otp', res.devOtp);
31             this.router.navigateByUrl('/register/email-otp');
32         } catch (e: any) {
33             this.error.set(e?.error?.error ?? e?.message ?? 'API error');
34         } finally {
35             this.loading.set(false);
36         }
37     }
38 }
39

```

ภาพประกอบที่ 3.18 การตรวจสอบการสมัครสมาชิก

บรรทัดที่ 12-39

โค้ดส่วนนี้ใช้จัดการการสมัครสมาชิกของผู้ใช้งาน โดยตรวจสอบข้อมูลที่กรอก ส่งข้อมูลไปยัง AuthService และใช้ Router เปลี่ยนหน้าไปยังหน้ายืนยัน OTP หากข้อมูลถูกต้องจะส่งข้อมูลไปยังระบบสมัครสมาชิก บันทึกข้อมูลบางส่วนไว้ใน sessionStorage และเปลี่ยนไปยังหน้ายืนยัน OTP แต่หากเกิดข้อผิดพลาด ระบบจะแสดงข้อความแจ้งเตือนให้ผู้ใช้งานทราบ

```

1  <main class="page">
2    <div class="card">
3      <h1 class="logo">OOPS!</h1>
4      <p class="prompt">กรุณกรอก Email - OTP เพื่อยืนยันตัวตน</p>
5      @if (email()) {
6        <p class="meta">{{ email() }}</p>
7      }
8      <input
9        class="otp-input"
10       placeholder="Email-OTP"
11       [ngModel]="otp()"
12       (ngModelChange)="otp.set($event)"
13       (keyup.enter)="submit()"
14     />
15     @if (error()) {
16       <p class="form-error">{{ error() }}</p>
17     }
18     <button type="button" class="btn-submit" (click)="submit()" [disabled]="loading()">
19       {{ loading() ? 'กำลังตรวจ...' : 'SUBMIT' }}
20     </button>
21   </div>
22 </main>

```

ภาพประกอบที่ 3.19 กรอกอีเมล OTP

บรรทัดที่ 1-22

ผู้ใช้งานกรอกรหัส OTP ลงในช่อง Input และเมื่อกดปุ่ม SUBMIT หรือปุ่ม Enter ระบบจะเรียกใช้ฟังก์ชัน submit() เพื่อตรวจสอบ OTP นอกจากนี้ยังมีการแสดงข้อความผิดพลาดและปิดการใช้งานปุ่มชั่วคราวระหว่างการตรวจสอบข้อมูล (loading()) เพื่อป้องกันการส่งข้อมูลซ้ำ

```

12 export class EmailOtp {
13   private auth = inject(AuthService);
14   private router = inject(Router);
15   otp = signal('');
16   loading = signal(false);
17   error = signal<string | null>(null);
18   email = signal(sessionStorage.getItem('oops_flow_email') ?? '');
19
20   async submit() {
21     if (!this.email()) {
22       this.error.set('Session หมดอายุ กรุณา register ใหม่');
23       return;
24     }
25     if (this.otp().length === 0) return;
26     this.loading.set(true);
27     this.error.set(null);
28     try {
29       const res = await this.auth.verifyEmail(this.email(), this.otp());
30       sessionStorage.setItem('oops_flow_qr', res.qr);
31       sessionStorage.setItem('oops_flow_secret', res.secret);
32       this.router.navigateByUrl('/login/qr');
33     } catch (e: any) {
34       this.error.set(e?.error?.error ?? e?.message ?? 'API error');
35     } finally {
36       this.loading.set(false);
37     }
38   }
39 }

```

ภาพประกอบที่ 3.20 ตรวจสอบ Email-OTP

- | | |
|-----------------|---|
| บรรทัดที่ 12-18 | สร้าง Class สำหรับจัดการการยืนยันตัวตนด้วย Email-OTP และกำหนด export สำหรับคลาส |
| บรรทัดที่ 20-24 | ฟังก์ชันสำหรับส่ง OTP ไปตรวจสอบกับระบบโดยตรวจสอบว่ามีอีเมลอยู่ในระบบหรือไม่ หากไม่มีจะแสดงข้อความแจ้งเตือน |
| บรรทัดที่ 25-37 | ตรวจสอบว่าผู้ใช้งานกรอก OTP หรือไม่โดยส่งอีเมลและ OTP ไปตรวจสอบกับ Back-end บันทึกข้อมูล QR Code และ Secret ลงใน sessionStorage และเปลี่ยนหน้าไปยังหน้าสแกน QR Code |

```

1 <main class="page">
2   <div class="card">
3     <h2 class="heading">Setup Authenticator</h2>
4     <h1 class="logo">OOPS!</h1>
5
6     <p class="prompt">สแกน QR code นี้ด้วยแอปพลิเคชัน Google Authenticator</p>
7     <p class="prompt">แล้วกรอกรหัส 6 หลักด้านล่างเพื่อเข้าใช้งาน</p>
8
9     @if (qr()) {
10      <div class="qr"><img [src]="qr()" alt="OTP QR code" /></div>
11      <p class="secret">หรือกรอก secret: <code>{{ secret() }}</code></p>
12    } @else {
13      <p class="form-error">ไม่พบ QR - กรุณา register ใหม่</p>
14    }
15
16    <input
17      class="input"
18      placeholder="กรอกรหัส 6 หลักจาก Authenticator"
19      maxlength="6"
20      inputmode="numeric"
21      [ngModel]="token()"
22      (ngModelChange)="token.set($event)"
23      (keyup.enter)="confirm()"
24    />
25    @if (error()) {
26      <p class="form-error">{{ error() }}</p>
27    }
28    <button type="button" class="btn-confirm" (click)="confirm()" [disabled]="loading()"
29      | {{ loading() ? 'กำลังยืนยัน...' : 'ยืนยันและเข้าสู่ระบบ' }}
30    </button>
31
32    <a routerLink="/" class="btn-back">กลับไปหน้าแรก</a>
33  </div>
34 </main>

```

ภาพประกอบที่ 3.21 ยืนยันตัวตนด้วย Google Authenticator

บรรทัดที่ 1-34

ระบบจะแสดง QR Code ให้ผู้ใช้งานสแกนผ่านแอปพลิเคชัน Google Authenticator เพื่อสร้างรหัส OTP สำหรับเข้าสู่ระบบ หากไม่สามารถสแกน QR Code ได้ ระบบจะแสดง Secret Key เพื่อให้ผู้ใช้งานกรอกเองได้

หลังจากผู้ใช้งานสแกน QR Code แล้ว จะสามารถกรอกรหัส OTP 6 หลักลงในช่อง Input และเมื่อกดปุ่ม “ยืนยันและเข้าสู่ระบบ” หรือกดปุ่ม Enter ระบบจะเรียกฟังก์ชัน confirm() เพื่อตรวจสอบ OTP นอกจากนี้ยังมีการแสดงข้อความแจ้งข้อผิดพลาด และปิดการใช้งานปุ่มชั่วคราวระหว่างการตรวจสอบข้อมูลเพื่อป้องกันการส่งข้อมูลซ้ำ รวมถึงมีปุ่มสำหรับกลับไปยังหน้าแรกของระบบ

```

65 register(fullName: string, email: string, phone: string) {
66   return firstValueFrom(
67     this.http.post<OtpResp>(`${this.base}/register`, { fullName, email, phone }),
68   );
69 }
70
71 verifyEmail(email: string, code: string) {
72   return firstValueFrom(this.http.post<QrResp>(`${this.base}/verify-email`, { email, code }));
73 }
74
75 // — First-time setup: confirm Authenticator —————
76 async confirmTotp(email: string, token: string) {
77   const res = await firstValueFrom(
78     this.http.post<TokenResp>(`${this.base}/confirm-totp`, { email, token }),
79   );
80   this.persist(res.token, res.user);
81   return res;
82 }

```

ภาพประกอบที่ 3.22 การกระบวนกรยืนยันตัวตนของผู้ใช้งาน

บรรทัดที่ 65-82

เป็นส่วนของ Service สำหรับจัดการกระบวนกรยืนยันตัวตนของผู้ใช้งาน ประกอบด้วย การลงทะเบียนผู้ใช้ (register) การยืนยันอีเมล (verifyEmail) และการยืนยันตัวตนแบบ Two-Factor Authentication (confirmTotp)

โดยแต่ละฟังก์ชันจะส่ง HTTP POST ไปยัง API ของระบบพร้อมข้อมูลที่เกี่ยวข้อง เช่น ชื่อผู้ใช้ อีเมล เบอร์โทรศัพท์ หรือรหัสยืนยัน จากนั้นใช้ firstValueFrom เพื่อแปลง Observable เป็น Promise สำหรับรองรับการทำงานแบบ async/await และเมื่อการยืนยันสำเร็จ ระบบจะทำการบันทึก token และข้อมูลผู้ใช้งานใน storage เพื่อใช้ในการเข้าสู่ระบบต่อไป.

3.6.6 การเข้าสู่ระบบ

ในการเข้าสู่ระบบจะต้องกรอก Email และ M-OTP เพื่อใช้ในการเข้าสู่ระบบ

```

21 <input
22   class="input"
23   placeholder="Enter email"
24   type="email"
25   [ngModel]="email()"
26   (ngModelChange)="email.set($event)"
27 />
28 @if (mode() === 'admin') {
29   <input
30     class="input"
31     placeholder="Password"
32     type="password"
33     [ngModel]="password()"
34     (ngModelChange)="password.set($event)"
35   />
36 }
37 <input
38   class="input"
39   placeholder="M-OTP (6 หลัก)"
40   maxLength="6"
41   inputmode="numeric"
42   [ngModel]="motp()"
43   (ngModelChange)="motp.set($event)"
44   (keyup.enter)="submit()"
45 />

```

ภาพประกอบที่ 3.23 กรอกข้อมูลสำหรับการเข้าสู่ระบบ

- | | |
|-----------------|--|
| บรรทัดที่ 21-27 | ใช้สร้างช่องกรอกอีเมลของผู้ใช้งานโดยเชื่อมข้อมูลอีเมลระหว่างหน้าเว็บกับระบบ Angular |
| บรรทัดที่ 28-35 | ใช้ตรวจสอบว่าเป็นโหมดผู้ดูแลระบบ (Admin) หรือไม่หากใช่จะมีช่องกรอกรหัสผ่านเพิ่มเติมเข้ามา |
| บรรทัดที่ 37-45 | ใช้สร้างช่องกรอกรหัส M-OTP จำนวน 6 หลักและกำหนดให้กรอกได้ไม่เกิน 6 ตัวและแสดงคีย์บอร์ดตัวเลขบนอุปกรณ์มือถือ ทำการเชื่อมข้อมูล M-OTP กับระบบ Angular เมื่อผู้ใช้งานกดปุ่ม Enter ระบบจะเรียกฟังก์ชัน submit() เพื่อเข้าสู่ระบบ |

```

47   @if (error()) {
48     <p class="form-error">{{ error() }}</p>
49   }
50
51   <div class="actions">
52     <button type="button" class="btn-submit" (click)="submit()" [disabled]="loading()"
53       | {{ loading() ? 'กำลังเข้าสู่ระบบ...' : 'SUBMIT' }}
54     </button>
55   </div>
56
57   <div class="links">
58     <a routerLink="/reset/email">Forgot M-OTP?</a>
59     <a routerLink="/contact-admin">Contact admin</a>
60   </div>
61   <p class="register-link">
62     ยังไม่มีบัญชี? <a routerLink="/register">สมัครสมาชิก</a>
63   </p>
64 </div>
65 </main>

```

ภาพประกอบที่ 3.24 หน้าเข้าสู่ระบบ

บรรทัด 47-65

ใช้สำหรับแสดงข้อความผิดพลาด ปุ่มเข้าสู่ระบบ และลิงก์ต่าง ๆ ภายในหน้า Login โดยระบบจะตรวจสอบว่ามีข้อผิดพลาดหรือไม่ผ่าน error() แล้วแสดงข้อความแจ้งเตือนให้ผู้ใช้งานทราบ จากนั้นมีปุ่ม SUBMIT สำหรับส่งข้อมูลเข้าสู่ระบบ

```

23 toggleMode() {
24   this.mode.set(this.mode() === 'member' ? 'admin' : 'member');
25   this.error.set(null);
26 }
27
28 async submit() {
29   if (!this.email() || !this.motp()) {
30     this.error.set('กรุณกรอกอีเมลและ M-OTP');
31     return;
32   }
33   if (this.mode() === 'admin' && !this.password()) {
34     this.error.set('กรุณกรอกรหัสผ่าน');
35     return;
36   }
37   this.loading.set(true);
38   this.error.set(null);
39   try {
40     if (this.mode() === 'admin') {
41       await this.auth.loginAdmin(this.email(), this.password(), this.motp());
42     } else {
43       await this.auth.loginMember(this.email(), this.motp());
44     }
45     this.router.navigateByUrl('/safebox');
46   } catch (e: any) {
47     this.error.set(e?.error?.error ?? e?.message ?? 'API error');
48   } finally {
49     this.loading.set(false);
50   }
51 }

```

ภาพประกอบที่ 3.25 คลาส Login

บรรทัด 23-25

ฟังก์ชัน toggleMode() ใช้สลับโหมดระหว่าง Member และ Admin พร้อมล้างข้อความข้อผิดพลาดเดิม

บรรทัด 28-51

ใช้ตรวจสอบความถูกต้องของข้อมูลก่อนเข้าสู่ระบบโดยตรวจสอบข้อมูลเข้าสู่ระบบตามโหมดที่เลือก หากเป็น Admin จะใช้รหัสผ่านร่วมกับ M-OTP แต่ Member ใช้เพียงอีเมลและ M-OTP และเมื่อเข้าสู่ระบบสำเร็จ ระบบจะเปลี่ยนไปยังหน้า


```

76  async confirmTotp(email: string, token: string) {
77      const res = await firstValueFrom(
78          this.http.post<TokenResp>(`${this.base}/confirm-totp`, { email, token }),
79      );
80      this.persist(res.token, res.user);
81      return res;
82  }

```

ภาพประกอบที่ 3.26 การยืนยันตัวตนแบบ Two-Factor Authentication (2FA)

บรรทัดที่ 76-82

ระบบจะรับค่าอีเมลและโทเคน (token) ที่ผู้ใช้กรอกจากแอปยืนยันตัวตน จากนั้นส่งข้อมูลไปยัง API ผ่าน HTTP POST ไปยัง endpoint /confirm-totp เมื่อได้รับการตอบกลับสำเร็จ ระบบจะทำการบันทึกข้อมูลผู้ใช้และ JWT token ลงในหน่วยเก็บข้อมูลภายในแอปผ่านฟังก์ชัน persist() เพื่อใช้ในการเข้าสู่ระบบในครั้งถัดไป

```

85  requestEmailOtp(email: string) {
86      return firstValueFrom(
87          this.http.post<OtpResp>(`${this.base}/login/request-email-otp`, { email }),
88      );
89  }
90
91  async loginMember(email: string, token: string) {
92      const res = await firstValueFrom(
93          this.http.post<TokenResp>(`${this.base}/login/member`, { email, token }),
94      );
95      this.persist(res.token, res.user);
96      return res;
97  }
98
99  async loginAdmin(email: string, password: string, token: string) {
100      const res = await firstValueFrom(
101          this.http.post<TokenResp>(`${this.base}/login/admin`, { email, password, token }),
102      );
103      this.persist(res.token, res.user);
104      return res;
105  }

```

ภาพประกอบที่ 3.27 Service สำหรับจัดการกระบวนการเข้าสู่ระบบ

บรรทัดที่ 85-89

การขอ OTP สำหรับเข้าสู่ระบบผ่านอีเมล

บรรทัดที่ 91-97

การเข้าสู่ระบบของผู้ใช้ทั่วไป (loginMember)

บรรทัดที่ 99-105

การเข้าสู่ระบบของผู้ดูแลระบบ (loginAdmin)

แต่ละฟังก์ชันจะส่งข้อมูลไปยัง API ผ่าน HTTP POST พร้อมข้อมูลที่ จากนั้นใช้ firstValueFrom เพื่อรองรับการทำงานแบบ async/await และเมื่อการเข้าสู่ระบบสำเร็จ ระบบจะทำการบันทึก token และข้อมูลผู้ใช้ผ่านฟังก์ชัน persist() เพื่อใช้ในการยืนยันตัวตนในครั้งถัดไป

3.6.7 การรีเซ็ต M-OTP

```

8      <input
9        class="input"
10       placeholder="Enter email"
11       type="email"
12       [ngModel]="email()"
13       (ngModelChange)="email.set($event)"
14       (keyup.enter)="submit()"
15     />
16     @if (error()) {
17       <p class="form-error">{{ error() }}</p>
18     }
19     <button class="btn-submit" (click)="submit()" [disabled]="loading()">
20       {{ loading() ? 'กำลังส่ง...' : 'submit' }}
21     </button>
22
23     <a routerLink="/login" class="link">Forget email</a>
24   </div>
25 </main>

```

ภาพประกอบที่ 3.28 กรอกอีเมลเพื่อขอรีเซ็ตรหัสยืนยันตัวตน

บรรทัดที่ 8-25

Reset M-OTP สำหรับให้ผู้ใช้งานกรอกอีเมลเพื่อขอรีเซ็ตรหัสยืนยันตัวตน โดยระบบจะแสดงข้อความแนะนำการใช้งานและมีช่องกรอกอีเมลที่ใช้ ngModel เชื่อมข้อมูลกับตัวแปร email() เมื่อผู้ใช้งานกดปุ่ม Enter หรือปุ่ม submit ระบบจะเรียกใช้ฟังก์ชัน submit() เพื่อส่งคำขอรีเซ็ตไปยังระบบ

```

12 export class ResetEmail {
13     private auth = inject(AuthService);
14     private router = inject(Router);
15     email = signal('');
16     loading = signal(false);
17     error = signal<string | null>(null);
18
19     async submit() {
20         if (!this.email()) return;
21         this.loading.set(true);
22         this.error.set(null);
23         try {
24             const res = await this.auth.resetMotpRequest(this.email());
25             sessionStorage.setItem('oops_flow_email', this.email());
26             if (res.devOtp) sessionStorage.setItem('oops_dev_otp', res.devOtp);
27             this.router.navigateByUrl('/reset/otp');
28         } catch (e: any) {
29             this.error.set(e?.error?.error ?? e?.message ?? 'API error');
30         } finally {
31             this.loading.set(false);
32         }
33     }

```

ภาพประกอบที่ 3.29 คลาสรีเซตอีเมล

บรรทัดที่ 12-33

เป็นคลาส ResetEmail สำหรับรีเซต M-OTP ผ่านอีเมล โดยใช้ AuthService ติดต่อกับระบบ และ Router สำหรับเปลี่ยนหน้าเว็บ ระบบใช้ signal() เก็บข้อมูลอีเมล สถานะการโหลด และข้อความข้อผิดพลาด

ฟังก์ชัน submit() ใช้ส่งคำขอรีเซต M-OTP เมื่อกรอกอีเมลสำเร็จ ระบบจะบันทึกข้อมูลลง sessionStorage และเปลี่ยนไปยังหน้า /reset/otp เพื่อกรอกรหัส OTP หากเกิดข้อผิดพลาด ระบบจะแสดงข้อความแจ้งเตือนผ่าน error()

```

20   async submit() {
21     if (!this.email() || !this.otp()) return;
22     this.loading.set(true);
23     this.error.set(null);
24     try {
25       await this.auth.resetMotpVerify(this.email(), this.otp());
26       this.router.navigateByUrl('/reset/new-motp');
27     } catch (e: any) {
28       this.error.set(e?.error?.error ?? e?.message ?? 'API error');
29     } finally {
30       this.loading.set(false);
31     }
32   }
33 }

```

ภาพประกอบที่ 3.30 คลาสรีเซ็ต OTP สำหรับตรวจสอบรหัสในขั้นตอนการรีเซ็ต M-OTP

บรรทัดที่ 20-33

ใช้ส่งอีเมลและรหัส OTP ไปตรวจสอบกับระบบ หากข้อมูลถูกต้อง ระบบจะเปลี่ยนไปยังหน้า /reset/new-motp เพื่อกำหนด M-OTP ใหม่ แต่หากเกิดข้อผิดพลาด ระบบจะแสดงข้อความแจ้งเตือนผ่าน error() และปิดสถานะการโหลดเมื่อการทำงานเสร็จสิ้น

```

23   async ngOnInit() {
24     if (!this.email()) {
25       this.error.set('Session หมดอายุ');
26       this.loading.set(false);
27       return;
28     }
29     try {
30       const res = await this.auth.resetMotpNew(this.email());
31       this.qr.set(res.qr);
32       this.secret.set(res.secret);
33     } catch (e: any) {
34       this.error.set(e?.error?.error ?? e?.message ?? 'API error');
35     } finally {
36       this.loading.set(false);
37     }
38   }

```

ภาพประกอบที่ 3.31 คลาสรีเซ็ต New M-OTP

บรรทัดที่ 23-38

หากพบอีเมลระบบจะเรียก auth.resetMotpNew() เพื่อสร้าง QR Code และ Secret Key ใหม่สำหรับตั้งค่า M-OTP จากนั้นนำข้อมูลไปเก็บใน qr และ secret เพื่อแสดงบนหน้า

```

40   async confirm() {
41       if (this.token().length !== 6) {
42           this.error.set('กรอกรหัส 6 หลัก');
43           return;
44       }
45       this.confirming.set(true);
46       this.error.set(null);
47       try {
48           await this.auth.loginMember(this.email(), this.token());
49           sessionStorage.clear();
50           this.router.navigateByUrl('/safebox');
51       } catch (e: any) {
52           this.error.set(e?.error?.error ?? e?.message ?? 'API error');
53       } finally {
54           this.confirming.set(false);
55       }
56   }
57 }

```

ภาพประกอบที่ 3.32 ฟังก์ชัน confirm สำหรับตรวจสอบรหัส OTP จาก Google Authenticator

บรรทัดที่ 40-57

ระบบจะตรวจสอบก่อนว่ารหัสที่กรอกมีครบ 6 หลักหรือไม่ หากไม่ครบจะแสดงข้อความแจ้งเตือนผ่าน error()

เมื่อข้อมูลถูกต้อง ระบบจะกำหนดสถานะ confirming เป็น true และเรียก auth.loginMember() เพื่อตรวจสอบรหัส OTP กับระบบ หากยืนยันสำเร็จ ระบบจะล้างข้อมูลใน sessionStorage และเปลี่ยนหน้าไปยัง /safebox

```

108 resetMotpRequest(email: string) {
109   return firstValueFrom(
110     this.http.post<OtpResp>(`${this.base}/reset-motp/request`, { email }),
111   );
112 }
113 resetMotpVerify(email: string, code: string) {
114   return firstValueFrom(
115     this.http.post<{ ok: true }>(`${this.base}/reset-motp/verify`, { email, code }),
116   );
117 }
118 resetMotpNew(email: string) {
119   return firstValueFrom(this.http.post<QrResp>(`${this.base}/reset-motp/new`, { email }));
120 }

```

ภาพประกอบที่ 3.33 Service สำหรับจัดการกระบวนการรีเซ็ตรหัส M-OTP

บรรทัดที่ 108-112	ขั้นตอนการขอรีเซ็ต (resetMotpRequest)
บรรทัดที่ 113-117	การยืนยันรหัสที่ส่งไปยังอีเมล (resetMotpVerify)
บรรทัดที่ 118-120	การสร้างค่า M-OTP ใหม่ (resetMotpNew)

แต่ละฟังก์ชันจะส่งคำขอไปยัง API ผ่าน HTTP POST พร้อมข้อมูลที่จำเป็น เช่น อีเมลและรหัสยืนยัน จากนั้นใช้ firstValueFrom เพื่อรองรับการทำงานแบบ async/await ของ Angular และรับผลลัพธ์กลับมาใช้งานต่อในระบบ.

3.6.8 ตรวจสอบความแข็งแกร่งของรหัสผ่านด้วยค่า Entropy

ระบบนี้มีการประเมินความแข็งแกร่งของรหัสผ่านโดยใช้ค่าเอนโทรปี (Entropy) ซึ่งคำนวณผ่าน CheckerService ในระบบ Angular เพื่อวัดระดับความยากในการคาดเดารหัสผ่าน โดยค่าที่ได้จะถูกนำมาแสดงผลให้ผู้ใช้งานทราบถึงความปลอดภัยของรหัสผ่านที่ป้อนเข้าไป

แม้ค่าเอนโทรปีไม่มีค่าตายตัวที่เป็นมาตรฐานสากล แต่โดยทั่วไปสามารถแบ่งช่วงการประเมินได้ดังนี้:

- 0-39 bits -> อยู่ในระดับ “น้อย” ไม่ปลอดภัย
- 40-79 bits -> อยู่ในระดับ “ปานกลาง” มีความปลอดภัยในระดับหนึ่ง
- 80 bits ขึ้นไป -> อยู่ในระดับ “สูง” ปลอดภัย

```

12 export class Entropy {
13   private checker = inject(CheckerService);
14   password = signal('');
15   result = signal<EntropyResult | null>(null);
16   loading = signal(false);
17   error = signal<string | null>(null);
18
19   async check() {
20     if (!this.password()) {
21       this.result.set(null);
22       return;
23     }
24     this.loading.set(true);
25     this.error.set(null);
26     try {
27       const res = await this.checker.entropy(this.password());
28       this.result.set(res);
29     } catch (e: any) {
30       this.error.set(e?.message ?? 'API error');
31     } finally {
32       this.loading.set(false);
33     }
34   }
35 }

```

ภาพประกอบที่ 3.34 คลาส Entropy

บรรทัดที่ 12-35

ระบบจะตรวจสอบว่ามีการกรอกรหัสผ่านหรือไม่ จากนั้นส่งค่าไปคำนวณ entropy พร้อมจัดการสถานะกำลังโหลดและข้อผิดพลาด และนำผลลัพธ์ที่ได้มาแสดงผลในหน้าจอ

```

10 export function calculateEntropy(password: string) {
11   let pool = 0;
12   if (/[0-9]/.test(password)) pool += 10;
13   if (/[a-z]/.test(password)) pool += 26;
14   if (/[A-Z]/.test(password)) pool += 26;
15   if (/[^A-Za-z0-9]/.test(password)) pool += 32;
16
17   if (pool === 0 || password.length === 0) {
18     return { entropy: 0, level: 'น้อย' as const, safe: false, pool };
19   }
20
21   const entropy = Math.round(password.length * Math.log2(pool));
22
23   // Thresholds (from spec): <40 weak, 40-79 medium, >=80 strong
24   let level: 'น้อย' | 'ปานกลาง' | 'สูง';
25   let safe: boolean;
26   if (entropy >= 80) {
27     level = 'สูง';
28     safe = true;
29   } else if (entropy >= 40) {
30     level = 'ปานกลาง';
31     safe = true;
32   } else {
33     level = 'น้อย';
34     safe = false;
35   }
36
37   return { entropy, level, safe, pool };
38 }

```

ภาพประกอบที่ 3.35 ฟังก์ชัน calculateEntropy()

บรรทัดที่ 10-15

กำหนดค่าตัวแปร pool เพื่อใช้เก็บขนาดของชุดอักขระ (Character Pool) จากนั้นจะทำการตรวจสอบว่ารหัสผ่านมีการใช้งานอักขระประเภทใดบ้างได้แก่

- ตัวเลข (0-9)
- ตัวอักษรพิมพ์เล็ก (a-z)
- ตัวอักษรพิมพ์ใหญ่ (A-Z)
- อักขระพิเศษ (Special Characters)

บรรทัดที่ 17-19

ตรวจสอบว่ารหัสผ่านว่างเปล่าหรือไม่ หรือมีค่าขนาดชุดอักขระเท่ากับ 0 หรือไม่ หากเป็นกรณีดังกล่าวระบบจะกำหนดค่า Entropy เป็น 0 พร้อมระบุระดับความปลอดภัยเป็น “น้อย” และกำหนดสถานะความปลอดภัย (safe) เป็น false เพื่อแสดงว่ารหัสผ่านไม่มีความปลอดภัย

บรรทัดที่ 21

กำหนดสูตรการคำนวณ

บรรทัดที่ 24-38

เมื่อได้ค่า Entropy แล้ว ระบบจะทำการจัดระดับความปลอดภัยของรหัสผ่านตามเกณฑ์ที่กำหนดไว้ในโครงงาน

สุดท้าย ระบบจะส่งผลลัพธ์กลับประกอบด้วยค่า Entropy ระดับความปลอดภัย (level) สถานะความปลอดภัย (safe) และขนาดของชุดอักขระ (pool) เพื่อนำไปใช้แสดงผลและวิเคราะห์ความแข็งแรงของรหัสผ่านภายในระบบต่อไป



3.6.9 Dictionary Attack

Dictionary Attack คือเทคนิคการโจมตีรหัสผ่านโดยใช้ชุดคำหรือรายการรหัสผ่านที่มีการรวบรวมไว้ล่วงหน้า มาทดลองเข้าสู่ระบบทีละคำให้ใช้เวลาให้น้อยกว่าและมีประสิทธิภาพมากในกรณีที่ผู้ตั้งรหัสผ่านที่เป็นคำทั่วไปหรือรหัสผ่านที่ถูกใช้งานซ้ำบ่อยครั้ง

ในระยะแรกของการพัฒนาโครงการ ผู้จัดทำมีแผนที่จะใช้ฐานข้อมูลรหัสผ่าน Rockyou2024 สำหรับการทดสอบการโจมตีแบบ Dictionary Attack เนื่องจากเป็นฐานข้อมูลที่มีการรวบรวมรหัสผ่านจำนวนมากและยังทันสมัย

หลังจากการศึกษาและทดลองใช้งานจริง พบว่าฐานข้อมูลดังกล่าวมีขนาดใหญ่มาก โดยมีข้อมูลรหัสผ่านมากกว่า 9,948,575,739 รายการ (ประมาณ 9.9 พันล้านรหัสผ่าน) และมีขนาดไฟล์รวมมากกว่า 100 GB ส่งผลให้ต้องใช้ทรัพยากรของระบบในระดับสูง ทั้งในด้านพื้นที่จัดเก็บข้อมูล หน่วยความจำ (RAM) และกำลังประมวลผลของเครื่องคอมพิวเตอร์

นอกจากนี้ การประมวลผลข้อมูลในลักษณะ Big Data จำเป็นต้องอาศัยโครงสร้างพื้นฐานที่มีประสิทธิภาพสูง เช่น เครื่องแม่ข่าย (Server) ที่มีสเปกสูง ระบบจัดเก็บข้อมูลขนาดใหญ่ รวมถึงอาจมีค่าใช้จ่ายเพิ่มเติมในการใช้บริการ Cloud Computing หรือบริการฐานข้อมูลออนไลน์ เพื่อรองรับการประมวลผลข้อมูลจำนวนมาก ซึ่งเกินขอบเขตด้านงบประมาณและทรัพยากรที่เหมาะสมสำหรับโครงการระดับปริญญาโท

ด้วยข้อจำกัดดังกล่าว ผู้จัดทำจึงได้ปรับเปลี่ยนแนวทางการดำเนินงาน โดยเลือกใช้ฐานข้อมูล Rockyou.txt แทน เนื่องจากเป็นชุดข้อมูลรหัสผ่านที่ได้รับความนิยมและถูกนำมาใช้อย่างแพร่หลายในงานด้าน Cybersecurity และการทดสอบเจาะระบบ (Penetration Testing) โดยข้อมูลภายในไฟล์มีรหัสผ่านอยู่ประมาณ 14,344,391 คำ (ณ วันที่ 27 พฤษภาคม 2569) เพราะมีขนาดข้อมูลที่เหมาะสม สามารถประมวลผลได้รวดเร็ว และเพียงพอต่อการจำลองสถานการณ์การโจมตีแบบ Dictionary Attack ในระดับที่สอดคล้องกับวัตถุประสงค์ของโครงการ

ในการทดลองครั้งนี้ได้เลือกใช้ฐานข้อมูลรหัสผ่าน Rockyou.txt เนื่องจากเป็นชุดข้อมูลรหัสผ่านที่ได้รับความนิยมและถูกนำมาใช้อย่างแพร่หลายในงานด้าน Cybersecurity และการทดสอบเจาะระบบ (Penetration Testing) โดยข้อมูลภายในไฟล์มีรหัสผ่านอยู่ประมาณ 14,344,391 คำ (ณ วันที่ 27 พฤษภาคม 2569) ซึ่งเป็นข้อมูลรหัสผ่านจริงที่รั่วไหลจากเหตุการณ์ข้อมูลหลุดของเว็บไซต์ RockYou ในปี ค.ศ. 2009

พร้อมกันนี้ ผู้จัดทำได้นำ API ของ Have I Been Pwned (HIBP) มาใช้ร่วมในการตรวจสอบรหัสผ่านเพื่อเพิ่มประสิทธิภาพและความทันสมัยในการตรวจสอบข้อมูลรหัสผ่านที่เคยรั่วไหลโดย HIBP เป็นบริการที่รวบรวมข้อมูลจากเหตุการณ์ข้อมูลรั่วไหล (Data Breach) จากหลากหลายแหล่งทั่วโลก ทำให้สามารถตรวจสอบได้ว่ารหัสผ่านที่ผู้ใช้งานตั้งเคยปรากฏอยู่ในฐานข้อมูลที่ถูกเปิดเผยหรือไม่ซึ่งมีข้อดีคือสามารถเข้าถึง

ข้อมูลที่มีการอัปเดตอยู่เสมอ ลดภาระในการจัดเก็บฐานข้อมูลขนาดใหญ่ภายในระบบ และช่วยลดต้นทุนด้านทรัพยากรของเครื่องคอมพิวเตอร์ได้อย่างมีประสิทธิภาพ อีกทั้งการเรียกใช้งานผ่าน API ยังช่วยให้ระบบสามารถตรวจสอบข้อมูลได้รวดเร็ว มีความยืดหยุ่นในการพัฒนา และง่ายต่อการต่อยอดในอนาคต

ทั้งนี้ ขั้นตอนการนำ API ของ Have I Been Pwned (HIBP) มาใช้งานร่วมกับระบบ ผู้จัดทำมีแผนที่จะพัฒนาและดำเนินการในลำดับถัดไป (Project 2) เนื่องจากจำเป็นต้องมีการออกแบบระบบเพิ่มเติมทั้งในด้านการเชื่อมต่อ API การจัดการความปลอดภัยของข้อมูล และการเพิ่มประสิทธิภาพในการประมวลผลข้อมูลผ่านเครือข่าย

ในขั้นตอนนี้ผู้จัดทำจะมุ่งเน้นการพัฒนาระบบตรวจสอบรหัสผ่านโดยอ้างอิงจากฐานข้อมูล Rockyou.txt เป็นหลัก เพื่อใช้ในการศึกษากระบวนการทำงานของการโจมตีแบบ Dictionary Attack และวิเคราะห์ความปลอดภัยของรหัสผ่านเบื้องต้น ซึ่งมีขั้นตอนสำคัญในการดำเนินงานดังต่อไปนี้

```

12 export class Dictionary {
13   private checker = inject(CheckerService);
14   password = signal('');
15   selected = signal<string[]>(['rockyou']);
16   result = signal<DictionaryResult | null>(null);
17   loading = signal(false);
18   error = signal<string | null>(null);
19
20   dicts = [
21     { id: 'rockyou', label: 'Rockyou.txt' },
22   ];
23
24   toggle(id: string) {
25     const cur = this.selected();
26     if (cur.includes(id)) {
27       if (cur.length > 1) this.selected.set(cur.filter((n) => n !== id));
28     } else {
29       this.selected.set([...cur, id]);
30     }
31   }
32
33   async check() {
34     if (!this.password()) {
35       this.result.set(null);
36       return;
37     }
38     this.loading.set(true);
39     this.error.set(null);
40     try {
41       const res = await this.checker.dictionary(this.password(), this.selected());
42       this.result.set(res);
43     } catch (e: any) {
44       this.error.set(e?.message ?? 'API error');
45     } finally {
46       this.loading.set(false);
47     }
48   }
49 }

```

ภาพประกอบที่ 3.36 คลาส Dictionary

บรรทัดที่ 12-18

จัดเก็บคำรหัสผ่านที่ผู้ใช้งานกรอกผ่านตัวแปร password และกำหนดฐานข้อมูลเริ่มต้นเป็น rockyou ผ่านตัวแปร selected นอกจากนี้ยังมีการจัดเก็บผลลัพธ์ของการตรวจสอบไว้ใน result รวมถึงสถานะการทำงานของระบบ เช่น loading สำหรับแสดงสถานะระหว่างประมวลผล และ error สำหรับจัดเก็บข้อความแจ้งเตือนเมื่อเกิดข้อผิดพลาดระหว่างการเรียกใช้งาน API

บรรทัดที่ 20-22

กำหนดรายการฐานข้อมูล Dictionary ที่สามารถเลือกใช้งานได้ โดยในโครงงานนี้กำหนดให้ใช้ Rockyoutxt เป็นฐานข้อมูลหลักในการตรวจสอบรหัสผ่าน

บรรทัดที่ 24-31

ฟังก์ชัน toggle() ทำหน้าที่จัดการการเลือกหรือยกเลิกฐานข้อมูลที่ต้องการใช้งาน โดยระบบจะตรวจสอบว่าฐานข้อมูลที่เลือกมีอยู่ในรายการแล้วหรือไม่ หากมีอยู่แล้วยังเหลือฐานข้อมูลอื่นให้เลือก ระบบจะทำการลบออกจากรายการ แต่หากยังไม่มีอยู่ในรายการ ระบบจะเพิ่มฐานข้อมูลดังกล่าวเข้าไปสำหรับการตรวจสอบรหัสผ่าน

บรรทัด 33-48

ทำหน้าที่ตรวจสอบรหัสผ่านที่ผู้ใช้งานกรอกเข้าสู่ระบบ โดยเริ่มจากตรวจสอบว่ามีการกรอกรหัสผ่านหรือไม่ หากไม่มีข้อมูลระบบจะยกเลิกการทำงานและล้างผลลัพธ์เดิม จากนั้นระบบจะกำหนดสถานะ loading เพื่อแสดงว่ากำลังประมวลผลและเรียกใช้งานเมธอด dictionary() จาก CheckerService เพื่อส่งรหัสผ่านและฐานข้อมูลที่เลือกไปตรวจสอบ เมื่อระบบได้รับผลลัพธ์กลับมาจะทำการจัดเก็บข้อมูลไว้ใน result เพื่อแสดงผลแก่ผู้ใช้งาน แต่หากเกิดข้อผิดพลาดระหว่างการทำงาน ระบบจะจัดเก็บข้อความข้อผิดพลาดไว้ใน error และเมื่อสิ้นสุดกระบวนการ ระบบจะปิดสถานะการโหลดข้อมูลโดยกำหนด loading กลับเป็น false อีกครั้ง

```

10  ✓ function loadRockyou(): Set<string> {
11    if (cache) return cache;
12    const filePath = join(__dirname, '..', 'data', 'rockyou-top.txt');
13    const text = readFileSync(filePath, 'utf-8');
14    cache = new Set(
15      text
16        .split('\n')
17        .map((s) => s.trim())
18        .filter((s) => s.length > 0),
19    );
20    return cache;
21  }

```

ภาพประกอบที่ 3.37 ฟังก์ชัน LoadRockyou

บรรทัดที่ 10-21

ตรวจสอบว่ามีข้อมูลอยู่ใน cache หรือไม่ หากมีอยู่แล้วจะส่งค่ากลับทันที แต่หากยังไม่มี ระบบจะทำการระบุตำแหน่งของไฟล์ rockyou-top.txt จากนั้นอ่านข้อมูลภายในไฟล์ด้วยรูปแบบ UTF-8 แล้วนำข้อมูลแต่ละบรรทัดมาแยกเป็นรหัสผ่านแต่ละรายการ พร้อมทั้งตัดช่องว่างที่ไม่จำเป็นออก และกรองข้อมูลที่เป็นค่าว่างทิ้ง ก่อนนำข้อมูลทั้งหมดไปจัดเก็บในรูปแบบ Set เพื่อเพิ่มความรวดเร็วในการค้นหาว่ารหัสผ่านระหว่างการตรวจสอบ

```

10  ✓ function loadRockyou(): Set<string> {
11    if (cache) return cache;
12    const filePath = join(__dirname, '..', 'data', 'rockyou-top.txt');
13    const text = readFileSync(filePath, 'utf-8');
14    cache = new Set(
15      text
16        .split('\n')
17        .map((s) => s.trim())
18        .filter((s) => s.length > 0),
19    );
20    return cache;
21  }
22
23  ✓ export function dictionaryCheck(password: string, dicts: string[] = ['rockyou']) {
24    const results: Record<string, boolean> = {};
25    let foundAny = false;
26
27    ✓ if (dicts.includes('rockyou')) {
28      const set = loadRockyou();
29      const found = set.has(password) || set.has(password.toLowerCase());
30      results['rockyou'] = found;
31      if (found) foundAny = true;
32    }
33
34    ✓ return {
35      safe: !foundAny,
36      leaked: foundAny,
37      results,
38      dictsChecked: dicts,
39    };
40  }

```

ภาพประกอบที่ 3.38 ฟังก์ชัน dictionaryCheck()

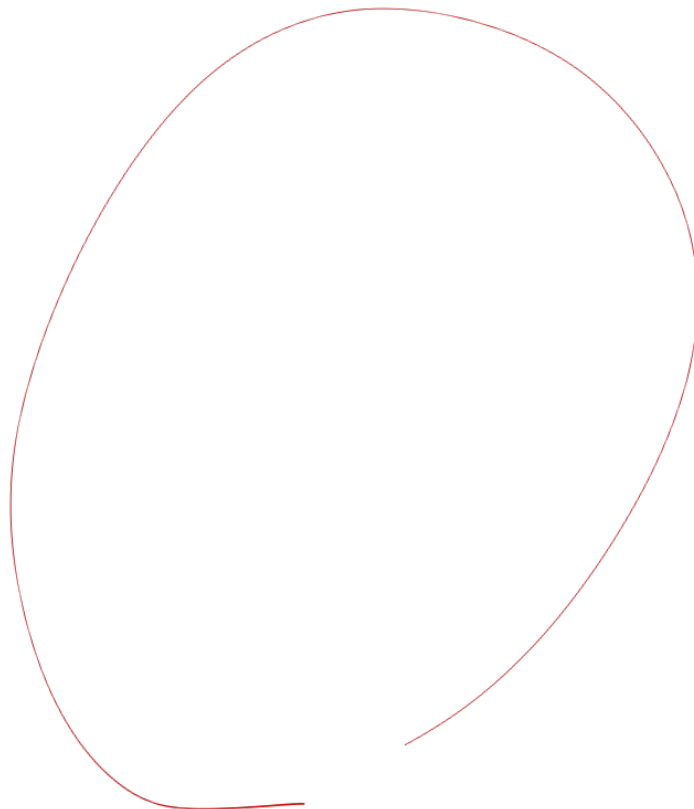
บรรทัดที่ 23-40

dictionaryCheck() ทำหน้าที่ตรวจสอบว่ารหัสผ่านที่ผู้ใช้งานกรอกมีอยู่ในฐานข้อมูล Dictionary หรือไม่

ระบบจะสร้างตัวแปร results สำหรับจัดเก็บผลลัพธ์ของแต่ละฐานข้อมูล และใช้ตัวแปร foundAny สำหรับตรวจสอบว่าพบรหัสผ่านในฐานข้อมูลหรือไม่ จากนั้นระบบจะตรวจสอบว่ามีการเลือกใช้งานฐานข้อมูล rockyou หรือไม่ หากมีระบบจะเรียกใช้ฟังก์ชัน loadRockyou() เพื่อโหลดข้อมูล

รหัสผ่านเข้าสู่ระบบ แล้วทำการตรวจสอบว่ารหัสผ่านที่ผู้ใช้งานกรอกตรงกับข้อมูลในฐานข้อมูลหรือไม่ ทั้งในรูปแบบตัวอักษรปกติและตัวอักษรขนาดเล็ก (Lowercase) เพื่อเพิ่มความยืดหยุ่นในการตรวจสอบ

หากพบรหัสผ่านในฐานข้อมูล ระบบจะกำหนดค่าใน results['rockyou'] เป็น true และเปลี่ยนสถานะ foundAny เป็น true เพื่อระบุว่ารหัสผ่านดังกล่าวมีความเสี่ยงหรือเคยปรากฏอยู่ในฐานข้อมูลรหัสผ่านที่ไม่ปลอดภัย



3.6.10 GPU Attack

GPU Attack คือการโจมตีรหัสผ่านด้วยการใช้พลังประมวลผลของ GPU เพื่อทดลองเดารหัสผ่านจำนวนมากในเวลาอันรวดเร็ว ทำให้รหัสผ่านที่มีความซับซ้อนต่ำสามารถถูกถอดรหัสได้ง่ายขึ้น

ผู้จัดทำจึงพัฒนาหน้าทดสอบ GPU Attack ด้วย Angular Framework เพื่อให้ผู้ใช้สามารถกรอกรหัสผ่านและเลือกประเภทของ GPU สำหรับจำลองการโจมตี โดยระบบจะคำนวณและแสดงผลระยะเวลาที่คาดว่าจะใช้ในการแคร็กรหัสผ่านแบบ Real-time ดังตัวอย่างโค้ดต่อไปนี้

```

12 export class GpuAttack {
13   private checker = inject(CheckerService);
14   password = signal('');
15   gpu = signal('NVIDIA RTX5090');
16   result = signal<GpuAttackResult | null>(null);
17   loading = signal(false);
18   error = signal<string | null>(null);
19
20   gpus = [
21     'NVIDIA RTX5090',
22     'NVIDIA RTX5080',
23     'NVIDIA RTX5070',
24     'NVIDIA RTX5060',
25     'NVIDIA RTX4090Ti',
26     'NVIDIA RTX4060',
27     'NVIDIA RTX4050',
28     'NVIDIA RTX3060',
29   ];
30
31   async check() {
32     if (!this.password()) {
33       this.result.set(null);
34       return;
35     }
36     this.loading.set(true);
37     this.error.set(null);
38     try {
39       const res = await this.checker.gpuAttack(this.password(), this.gpu());
40       this.result.set(res);
41     } catch (e: any) {
42       this.error.set(e?.message ?? 'API error');
43     } finally {
44       this.loading.set(false);
45     }
46   }
47 }

```

ภาพประกอบที่ 3.39 คลาส GpuAttack

บรรทัดที่ 13-18

ภายในคลาสมีการใช้ `signal()` เพื่อจัดเก็บข้อมูลต่าง ๆ ได้แก่ รหัสผ่าน (password), รุ่นของ GPU (gpu), ผลลัพธ์การคำนวณ (result), สถานะการโหลด (loading) และข้อความผิดพลาด (error) ซึ่งช่วยให้ข้อมูลสามารถอัปเดตบนหน้าเว็บแบบ Real-time ได้

บรรทัดที่ 20-47

ตัวแปร `gpus` ถูกกำหนดเป็นรายการรุ่นของการ์ดจอที่ผู้ใช้สามารถเลือกใช้ ในการจำลองการโจมตีรหัสผ่าน

ฟังก์ชัน `check()` จะทำหน้าที่ตรวจสอบว่าผู้ใช้กรอกรหัสผ่านหรือไม่ หากมีข้อมูล ระบบจะส่งรหัสผ่านและรุ่น GPU ไปยัง `CheckerService` ผ่านเมธอด `gpuAttack()` เพื่อคำนวณเวลาที่ใช้ในการแคร็กรหัสผ่าน จากนั้นจะแสดงผลลัพธ์กลับมายังผู้ใช้ หากเกิดข้อผิดพลาด ระบบจะแสดงข้อความแจ้งเตือนทันที *แจ้งเตือน*

```

8  export const GPU_RATES: Record<string, number> = {
9    // hashes per second (rough order-of-magnitude vs. NTLM)
10   'NVIDIA RTX5090': 380e9,
11   'NVIDIA RTX5080': 240e9,
12   'NVIDIA RTX5070': 160e9,
13   'NVIDIA RTX5060': 110e9,
14   'NVIDIA RTX4090Ti': 300e9,
15   'NVIDIA RTX4090': 286e9,
16   'NVIDIA RTX4060': 60e9,
17   'NVIDIA RTX4050': 38e9,
18   'NVIDIA RTX3060': 27e9,
19 };

```

ภาพประกอบที่ 3.40 GPU_RATES

บรรทัดที่ 8-19

ใช้กำหนดความเร็วในการประมวลผลของ GPU แต่ละรุ่น โดยค่าที่เก็บไว้คือ จำนวนครั้งในการทดลองแฮชรหัสผ่านต่อวินาที (Hashes per Second) เช่น NVIDIA RTX5090 มีความเร็วประมาณ 380e9 hashes ต่อวินาที ซึ่งหมายถึงสามารถทดลองรหัสผ่านได้ประมาณ 380 พันล้านครั้งต่อวินาที


```

21 function poolSize(password: string): number {
22     let pool = 0;
23     if (/[0-9]/.test(password)) pool += 10;
24     if (/[a-z]/.test(password)) pool += 26;
25     if (/[A-Z]/.test(password)) pool += 26;
26     if (/^[A-Za-z0-9]/.test(password)) pool += 32;
27     return pool;
28 }
29
30 function formatDuration(seconds: number): string {
31     if (seconds < 1) return 'น้อยกว่า 1 วินาที';
32     if (seconds < 60) return `${Math.round(seconds)} วินาที`;
33     if (seconds < 3600) return `${Math.round(seconds / 60)} นาที`;
34     if (seconds < 86400) return `${Math.round(seconds / 3600)} ชั่วโมง`;
35     if (seconds < 31_536_000) return `${Math.round(seconds / 86400)} วัน`;
36     const years = seconds / 31_536_000;
37     if (years > 1e9) return `${(years / 1e9).toExponential(2)} พันล้านปี`;
38     if (years > 1e6) return `${Math.round(years / 1e6)} ล้านปี`;
39     return `${Math.round(years)} ปี`;
40 }

```

ภาพประกอบที่ 3.41 ฟังก์ชัน poolSize & formatDuration

- บรรทัดที่ 21-28 ใช้สำหรับคำนวณขนาดของชุดอักขระภายในรหัสผ่าน โดยระบบจะตรวจสอบว่ารหัสผ่านประกอบด้วยตัวเลข ตัวอักษรพิมพ์เล็ก ตัวอักษรพิมพ์ใหญ่ หรืออักขระพิเศษหรือไม่ จากนั้นนำจำนวนอักขระของแต่ละประเภทมารวมกัน เพื่อนำไปใช้คำนวณความเป็นไปได้ของรหัสผ่านในการโจมตีแบบ Brute-force
- บรรทัดที่ 30-40 ใช้สำหรับแปลงค่าระยะเวลาจากหน่วยวินาทีให้อยู่ในรูปแบบที่อ่านเข้าใจง่าย เช่น วินาที นาที ชั่วโมง วัน หรือปี เพื่อช่วยให้ผู้ใช้สามารถมองเห็นระยะเวลาที่คาดว่าจะใช้ในการแคร็กรหัสผ่านได้ชัดเจนมากยิ่งขึ้น

```

42 export function gpuAttack(password: string, gpu: string) {
43   if (!password) {
44     return { time: '', seconds: 0, safe: false, gpu, rate: 0 };
45   }
46
47   const rate = GPU_RATES[gpu] ?? GPU_RATES['NVIDIA RTX4090'];
48   const pool = poolSize(password);
49   if (pool === 0) {
50     return { time: '0 วินาที', seconds: 0, safe: false, gpu, rate };
51   }
52
53   // Worst-case combinations: pool^length, average half of that.
54   // Cap exponent to avoid Infinity for very long passwords.
55   const exp = Math.min(password.length, 64);
56   const combinations = Math.pow(pool, exp) / 2;
57   const seconds = combinations / rate;
58
59   const safe = seconds > 3600 * 24 * 365; // > 1 year considered safe
60
61   return {
62     time: formatDuration(seconds),
63     seconds,
64     safe,
65     gpu,
66     rate,
67   };
68 }
69
70 export const SUPPORTED_GPUS = Object.keys(GPU_RATES);

```

ภาพประกอบที่ 3.42 ฟังก์ชันสำหรับคำนวณระยะเวลาที่คาดว่าจะใช้ในการแคร็กรหัสผ่าน

บรรทัดที่ 42-68

ระบบจะรับค่ารหัสผ่านและรุ่นของ GPU จากผู้ใช้ จากนั้นนำข้อมูลความเร็วของ GPU จากตัวแปร GPU_RATES มาคำนวณร่วมกับขนาดของชุดอักขระ (poolSize) เพื่อหาจำนวนความเป็นไปได้ของรหัสผ่านแบบ Brute-force

ระบบจะคำนวณจำนวน combinations ของรหัสผ่าน และนำมาหารด้วยอัตราการประมวลผลของ GPU เพื่อประมาณระยะเวลาในการแคร็ก จากนั้นใช้ฟังก์ชัน formatDuration() แปลงเวลาให้อ่าน พร้อมประเมินว่ารหัสผ่านมีความปลอดภัยหรือไม่ หากใช้เวลาเกิน 1 ปีในการแคร็กจะถือว่ามีความปลอดภัย

บรรทัดที่ 70

ใช้สำหรับดึงรายชื่อ GPU ที่ระบบรองรับ เพื่อนำไปแสดงในหน้าเลือกอุปกรณ์สำหรับจำลองการโจมตีรหัสผ่านต่อไป

3.6.11 AES 256 GCM

โครงการนี้เลือกใช้ Scrypt Hash Function ในการจัดเก็บรหัสผ่านของผู้ใช้งานภายในฐานข้อมูล เนื่องจาก Scrypt เป็น Salted Hash Function ประเภท Memory-Hard Function ที่ถูกออกแบบมาเพื่อเพิ่มความปลอดภัยและลดความเสี่ยงจากการโจมตีแบบ Rainbow Table Attack และ Brute Force Attack โดยเฉพาะการโจมตีที่ใช้หน่วยประมวลผลกราฟิก (GPU) ในการคำนวณค่า Hash จำนวนมากพร้อมกัน

แม้ว่า GPU จะมีจำนวน Processors มากกว่า CPU ทำให้สามารถคำนวณค่า Hash ได้รวดเร็วกว่า แต่ Scrypt ถูกออกแบบมาให้ใช้หน่วยความจำ (Memory) จำนวนมากระหว่างการประมวลผล ส่งผลให้ผู้โจมตีจำเป็นต้องใช้ RAM ปริมาณสูงในการถอดรหัสผ่าน หากต้องการเพิ่มประสิทธิภาพในการโจมตีจำเป็นต้องเพิ่มหน่วยความจำให้กับ GPU ซึ่งมีต้นทุนสูง ทำให้การโจมตีทำได้ยากและไม่คุ้มค่า

ก่อนหน้านี้ระบบจำนวนมากนิยมใช้ Bcrypt ซึ่งเป็น Salted Hash Function สำหรับจัดเก็บรหัสผ่าน อย่างไรก็ตาม Scrypt มีจุดเด่นด้านการป้องกันการโจมตีด้วย GPU ได้ดีกว่า เนื่องจากมีคุณสมบัติแบบ Memory-Hard ทำให้เหมาะสมสำหรับระบบที่ต้องการความปลอดภัยของรหัสผ่านในระดับสูง

นอกจากนี้ยังมี Argon2id ซึ่งเป็นอัลกอริทึมสำหรับ Password Hashing ที่ได้รับความนิยมและถูกแนะนำโดยองค์กร OWASP เนื่องจากมีประสิทธิภาพด้านความปลอดภัยสูงเช่นเดียวกัน อย่างไรก็ตาม Argon2id มีความซับซ้อนในการกำหนดค่า Parameters มากกว่า Scrypt ทั้งในด้าน Memory Cost, Time Cost และ Parallelism ดังนั้นโครงการปริญญาโทฉบับนี้จึงเลือกใช้ Scrypt เนื่องจากมีความเหมาะสมต่อการพัฒนาและสามารถนำไปประยุกต์ใช้งานได้ง่ายกว่า

```

7  function getKey(): Buffer {
8      if (cachedKey) return cachedKey;
9      const k = process.env.VAULT_MASTER_KEY;
10     if (!k) {
11         throw new Error('VAULT_MASTER_KEY env var is required for vault encryption');
12     }
13     if (k.length === 44) {
14         cachedKey = Buffer.from(k, 'base64');
15     } else if (k.length === 64 && /^[0-9a-f]+$/.test(k)) {
16         cachedKey = Buffer.from(k, 'hex');
17     } else {
18         cachedKey = scryptSync(k, 'oops-vault-salt-v1', 32);
19     }
20     return cachedKey;
21 }

```

ภาพประกอบที่ 3.43 ฟังก์ชัน getKey

บรรทัดที่ 7-21

ใช้สำหรับดึงและเตรียม Encryption Key ที่ใช้ในการเข้ารหัสข้อมูล โดยระบบจะตรวจสอบก่อนว่ามีการเก็บ key ไว้ในตัวแปร cachedKey แล้วหรือไม่ หากมีอยู่แล้วจะเรียกใช้งานทันทีเพื่อลดการประมวลผลซ้ำซ้อน

จากนั้นระบบจะดึงค่าจากตัวแปรสภาพแวดล้อม VAULT_MASTER_KEY หากไม่พบค่าจะเกิดข้อผิดพลาดเพื่อป้องกันการดำเนินงานโดยไม่มีคีย์เข้ารหัส

ในกรณีที่คีย์ถูกเก็บในรูปแบบ Base64 หรือ Hex ระบบจะทำการแปลงค่าให้อยู่ในรูปแบบ Buffer แต่หากเป็นข้อความทั่วไป ระบบจะใช้ scryptSync() เพื่อสร้างกุญแจเข้ารหัสขนาด 32 bytes สำหรับใช้งานกับอัลกอริทึม AES-256-GCM ต่อไป

```

23 export function encrypt(plaintext: string): string {
24     const iv = randomBytes(12);
25     const cipher = createCipheriv(ALGO, getKey(), iv);
26     const enc = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final()]);
27     const tag = cipher.getAuthTag();
28     return Buffer.concat([iv, tag, enc]).toString('base64');
29 }
30
31 export function decrypt(packed: string): string {
32     const buf = Buffer.from(packed, 'base64');
33     const iv = buf.subarray(0, 12);
34     const tag = buf.subarray(12, 28);
35     const enc = buf.subarray(28);
36     const decipher = createDecipheriv(ALGO, getKey(), iv);
37     decipher.setAuthTag(tag);
38     return Buffer.concat([decipher.update(enc), decipher.final()]).toString('utf8');
39 }

```

ภาพประกอบที่ 3.44 ฟังก์ชัน encrypt&decrypt

บรรทัดที่ 23-29

ใช้สำหรับเข้ารหัสข้อมูลข้อความธรรมดาด้วยอัลกอริทึม AES-256-GCM โดยระบบจะสร้างค่า IV (Initialization Vector) แบบสุ่มเพื่อเพิ่มความปลอดภัย จากนั้นนำข้อมูลไปเข้ารหัสและสร้าง Authentication Tag สำหรับตรวจสอบความถูกต้องของข้อมูล ก่อนรวมข้อมูลทั้งหมดและแปลงเป็นรูปแบบ Base64 เพื่อจัดเก็บหรือส่งต่อได้ง่าย

บรรทัดที่ 31-39

ใช้สำหรับถอดรหัสข้อมูลที่ถูกรหัสกลับมาเป็นข้อความเดิม โดยระบบจะทำการแยกค่า IV, Authentication Tag และข้อมูลที่ถูกรหัสออกจากกัน จากนั้นใช้กุญแจลับเดียวกันในการถอดรหัสและตรวจสอบความถูกต้องของข้อมูล เพื่อให้มั่นใจว่าข้อมูลไม่ถูกแก้ไขระหว่างการจัดเก็บหรือส่งผ่านระบบ

เอกสารอ้างอิง

- [1] D. p. manager, “Dashlane,” [ออนไลน์]. Available: <https://www.dashlane.com/>.
- [2] T.-r. p. m. f. p. & b. use, “1password,” [ออนไลน์]. Available: <https://1password.com/>.
- [3] m. & a. p. Securely store, “NordPass,” [ออนไลน์]. Available: <https://nordpass.com/>.
- [4] R. p. manager, “RoboForm,” [ออนไลน์]. Available: <https://www.roboform.com/>.
- [5] “RockYou2024 | 10 billion passwords leaked, how to check yours,” Webopedia, 5 July 2024. [ออนไลน์]. Available: https://www.webopedia.com/technology/rockyou2024-data-leak-explained/?utm_source=chatgpt.com. [%1 ที่เข้าถึง 7 May 2026].
- [6] T. Hunt, “Have I Been Pwned,” Have I Been Pwned, [ออนไลน์]. Available: <https://haveibeenpwned.com/>. [%1 ที่เข้าถึง 12 May 2026].
- [7] NVIDIA, “GeForce Graphics Cards Compare,” [ออนไลน์]. Available: https://www.nvidia.com/en-us/geforce/graphics-cards/compare/?utm_source=chatgpt.com. [%1 ที่เข้าถึง 7 5 2026].
- [8] R. D. L. Explained. [ออนไลน์]. Available: https://www.webopedia.com/technology/rockyou2024-data-leak-explained/?utm_source=chatgpt.com.

ประวัติย่อผู้จัดทำ

~~ประวัติย่อผู้จัดทำโครงการ~~

ประวัติย่อผู้จัดทำโครงการ

ชื่อ ชื่อสกุล มลธิชา ศรีดาจันทร์
วัน เดือน ปีเกิด 26 สิงหาคม 2546
สถานที่เกิด อำเภอขุนหาญ จังหวัดศรีสะเกษ
ที่อยู่ที่สามารถติดต่อได้ 163 หมู่ 3 ตำบลขุนหาญ อำเภอขุนหาญ จังหวัดศรีสะเกษ
โทรศัพท์มือถือ 0943067913
อีเมล

ประวัติการศึกษา ปัจจุบัน ปริญญาตรี (ระบบปกติ)

วิทยาศาสตร์บัณฑิต (วท.บ.) วิทยาการคอมพิวเตอร์

มหาวิทยาลัยมหาสารคาม

มัธยมศึกษาตอนปลาย โรงเรียนขุนหาญวิทยาสรรค